JOB-SWITCH HANDBOOK

From zero to the interview

Sahil Siddiqui
https://www.linkedin.com/in/sahil-siddiqui-8416b8195/

This book teaches arrays and matrices in JavaScript. Each chapter uses only ideas from earlier chapters. You start by indexing a list. You finish by filling a grid in the order the cells depend on.

One learning path · Stay, then Then, then a second pass

00 The array
You become able to index, sort, and tell a subarray from a subsequence.

01 Read the problem
You become able to classify a prompt in four lines before any code.

02 One pass
You become able to answer each index from a running summary.

03 Two loops
You become able to enumerate every pair and every subarray.

04 Hash map
You become able to replace an inner search with a lookup.

05 Pointers from the ends
You become able to discard one end of a sorted range.

06 Same-direction pointers
You become able to overwrite an array with a read index and a write index.

07 Sliding window
You become able to grow and shrink one contiguous band.

08 Prefix sums
You become able to answer a range sum, including when negatives appear.

09 Difference array
You become able to add a value on many ranges, then read the array once.

10 A left pass and a right pass
You become able to combine two directional summaries.

11 Kadane
You become able to take the best contiguous sum, even if every value is negative.

12 Sort, then scan
You become able to pay n log n so pairs and intervals have an order.

13 Place each value
You become able to put a value at the index where it belongs.

14 Binary search
You become able to search an index, or search the answer itself.

15 Monotonic stack
You become able to find the next greater element in one pass.

16 Greedy scans
You become able to track a reach, a jump layer, or a tank.

17 Recognition
You become able to name the chapter from the wording.

18 The matrix
You become able to index rows, columns, diagonals, and four directions.

19 Walk the matrix
You become able to spiral, transpose, and rotate.

20 In-place marks
You become able to store the next state inside the cell.

21 2D prefix
You become able to sum any rectangle, then collapse a matrix to a 1D map.

22 Sorted matrix
You become able to binary-search a flat index, or walk a staircase.

23 Regions
You become able to flood a region and run a multi-source search.

24 Filling order

You become able to fill a cell only after the cells it depends on.

25 Four weeks

You become able to order the practice and sit a mixed mock.

BEFORE CHAPTER 0

How to study this book

Read one chapter at a time. Close the page and rewrite the template before you solve that chapter's problems.

How a chapter is built

Each pattern chapter has nine parts. They always come in the same order, so you know where to look.

The first part says what the pattern is. It states the idea in plain words, before any code. The second part says what you already know. It names the earlier chapter this one uses. If you cannot rewrite that earlier template from a blank file, do that rewrite before you start.

The third part is a worked example, and it comes before the code. This is the part to read slowly. The example takes a small input and walks it one step at a time. You see which position you are on, what you update, and what is stored when the step ends. The code later is a compact form of this walk. If you skip the walk, you will not see why one line has to run before the next.

The fourth part is how to recognise the pattern. It lists the signs in a prompt that point to this chapter, and it names the lookalike that belongs in another chapter. The fifth part is headed What to say out loud. Those are two to four sentences you can say in an interview before you write code.

The sixth part is the JavaScript template. The lines with it say what the important lines do. Read those lines beside the example. The seventh part is time and extra memory. Time is how the number of steps grows as the list gets longer. Extra memory is space you use besides the input. A few variables are a small cost. A second copy of the whole list is a different cost, and the chapter says so when that happens.

The eighth part is mistakes. These are the errors this pattern invites. After you rewrite the template, check your code against that list. The ninth part is a practice ladder. It is the problem list, already in order. Start at the top.

Some chapters contain more than one template. Each template still has an example, the code, and a cost. You read each block the same way. The worked example is still the part to read slowly.

The three lanes

Every practice problem sits in one lane. The three lanes are **Stay**, **Then**, and **Second pass**. The lane is printed on the problem. It tells you when that problem is your work.

A Stay problem drills the template you just rewrote. Stay is required. A Stay problem is never premium-only. Premium means LeetCode locks the problem statement behind a paid account, so a free account cannot open it. This book does not put a locked problem in Stay.

A Then problem is the same chapter with one extra demand. You might handle a carry, a count, or a wrap from the last position back to the first. You do Then after Stay. Then is still first-pass work.

A Second pass problem is a harder use of the same idea. Second pass waits until the path is finished. The path is chapter 0 through chapter 25. Leave these problems until that path is done.

First pass means Stay plus Then. You finish both before you open the next chapter. Second pass is a later round. It starts the week after you finish chapter 24. Chapter 25 is the calendar and the mock. It is not a new template.

A paid problem sometimes uses the same pattern as the chapter. That problem is a premium cousin. The chapter names the cousin in one sentence. The sentence tells you the twist. The cousin is not required work. You can finish this book without a paid account.

What to do on a study day

Work on one chapter for the day. Do these steps in order.

Read the chapter once. Include the worked example. Follow each step of the example on paper, with the same numbers the chapter uses. Read the example before you study the template.

Close the page. Open a blank file. Rewrite the template from memory. Write the whole template before you open the chapter again.

Open the chapter and check the update order. The update order is which variable changes on which line, and which of those lines runs first. If your order does not match, close the page and write the template again. Repeat until the update order matches.

Solve the Stay problems in the listed order. Finish Stay before you open the next chapter.

The Then problems come after Stay. Solve them before you leave the chapter. Stay plus Then is the first pass for that chapter. Second pass is not part of this day.

When you are stuck

Set a timer for 25 minutes. Use that time to name the pattern and to write the first lines of the template. Test a plan on a small input, even if you expect the plan to fail. A failed plan tells you which fact you missed.

If the 25 minutes end and you still have no correct approach, stop. Go back to the chapter. Reread only the section headed How you recognise it, and the section headed What to say out loud. Do not reread the whole chapter. Do not open a solution.

You are training recognition, not transcription. Recognition means you look at a new prompt and name the pattern. Transcription means you copy steps from a solution that is already open. Copying does not teach the name. In the interview the prompt is new, and no solution sits beside you. After those two sections, close the page and try the problem again from a blank file.

A 45-minute interview

A 45-minute interview has four beats. Do them in order. Say each beat out loud while you work.

First, restate the input and the output. Say what the function receives. Say what it must return. Say the limits in the prompt: how long the list can be, and whether a value can be missing, repeated, or negative. If you cannot say this in a few sentences, you do not know the problem yet. Wait to write code until you can.

Second, say the pattern name and the brute-force cost. Brute force means the direct plan that checks every possibility, such as every pair or every subarray. Say that plan's cost out loud. A common cost is $O(n^2)$. $O(n^2)$ means the work grows like n times n. If the list can be very long, say that this plan is too slow, and name the pattern you will use instead.

Third, write the template. While you write, narrate the invariant. An invariant is a fact that stays true every time the loop finishes a step. Point at the variables that fact uses. You might say that a running total covers every position from the first value through the value you just read, and that the same fact is true again after the next step. Say that fact. Leave the brackets and the semicolons unspoken.

Fourth, test four inputs before you say you are done. Use four inputs the constraints allow. The four are an empty list, one element, duplicates, and negatives. If a constraint forbids one of them, say that out loud, and replace it with a case the prompt does allow. Walk each input by hand. Check the position you read and the value you return. After those four checks, you may say you are done.

JavaScript you will trip on

The code in this book is the JavaScript you write in an interview. Six language facts can still make a correct pattern fail. Read them before chapter 0. An index is a position in a list. The first index is 0.

1. **Sort needs a comparator.** A comparator is a function you pass to sort. It tells sort which of two values comes first. Take the list [10, 2, 1]. If you call sort and you pass no function, JavaScript turns the values into strings. String order is dictionary order. The character 1 comes before the characters 10, and the characters 10 come before the character 2, because the first character decides. The list becomes [1, 10, 2]. That is not number order. When the values are numbers, write `sort((a, b) => a - b)`. A negative result puts `a` first. The list then becomes [1, 2, 10].
2. **Map versus object.** An object key is a string. If you use a number as a key, JavaScript stores the characters of that number, not the number itself. A stored index of 0 is also falsy. Falsy means a test treats the value as false. So `if (obj[need])` is false when the stored index is 0. You miss index 0, and you think the key is absent. The key is there. The test is wrong. Use a Map when the key is a number, and when the stored value might be 0. `Map.has` tells you the key is present even when the value is 0.
3. **Negative remainder.** In JavaScript, `-1 % 5` is `-1`, not `4`. The `%` operator keeps the sign of the number on the left. A remainder you use as an index has to sit in the range from 0 up to one less than the divisor. The repair is `((x % k) + k) % k`. For `x = -1` and `k = 5`, the inner step is `-1 + 5`, which is 4, and `4 % 5` is 4. Chapter 8 uses this repair.
4. **Shared row array.** `Array(m).fill(Array(n).fill(0))` looks like a grid of zeros. It is not `m` separate rows. `fill` puts the same inner array in every row. A write on row 0 shows up on row 1, because every row is that same array. Chapter 18 shows the fix. Use `Array.from`, and build a new array for each row inside the function you pass. Each row is then its own array. A write on row 0 stays on row 0.
5. **Do not walk an array with `for...in`.** `for...in` walks keys. It does not walk values. The keys are strings, so you get the characters 0 and 1, not the numbers 0 and 1. A prototype can show up in that walk. A prototype is a shared object behind the array. If a property was added there, the loop can visit it. Use a counted loop when you need the index: `for (let i = 0; i < nums.length; i++)`. Use `for...of` when you only need each value.
6. **`shift` and `unshift` inside a loop are not free.** `shift` removes the front value and slides every later value one place left. `unshift` inserts at the front and slides every value one place right. One call walks every later value. Do that n times and the cost is $O(n^2)$, the same n-times-n cost named above. A head index is the cheap version of removing from the front. You leave the array in place. You keep an integer at the front, and you add one to that integer when the front value is done. The next read starts at the new integer.

Links

Every practice link is a real LeetCode problem. The line shows the number and the official title, which is the title on the LeetCode page. The address has this shape: leetcode.com/problems/<slug>/. The slug is the short name in the address. It is lowercase, and a hyphen stands where the title has a space.

You can check the shape on a problem from chapter 0. Problem 1929 is Concatenation of Array. Its address is leetcode.com/problems/concatenation-of-array/. The number, the title, and the slug all name that same problem. If the page title does not match the ladder, you are on the wrong problem.

The difficulty tag matches the site. LeetCode rates a problem Easy, Medium, or Hard, and this book uses that same rating. The book does not retag a Hard problem as Easy.

The lane sits next to the difficulty, and the two tags answer different questions. The difficulty tag is LeetCode's rating. The lane is this book's order: Stay, Then, or Second pass. Open the ladder from the top. Solve in that order.

Carry forward. Chapter 0 is the list itself. An index is a position. The first index is 0. If the list holds n values, the last index you may read is $n - 1$. The next index, n, is past the end. The same chapter shows what $O(n^2)$ costs when n is 100,000. $O(n^2)$ means the work grows like n times n. 100,000 times 100,000 is 10 billion steps. At a rough speed of 100 million simple steps a second, that is about 100 seconds. A judge stops a solution before then. Two full loops are too slow for a list that long.

CHAPTER 00 · BUILDS ON NOTHING BUT COUNTING

The array in JavaScript

An index is a position. The first is 0, the last is n − 1, and a subarray has no gaps.

What an array is

An array is a row of values stored in order. You reach a value by its index. An index is the position number of a slot. The first index is 0, not 1. If the array holds n values, the last index you may read is n − 1. `nums.length` is n. `nums[nums.length]` is one past the end. JavaScript gives you `undefined` there. It does not throw.

Take `[10, 20, 30]`. Here n is 3. Index 0 holds 10. Index 1 holds 20. Index 2 holds 30. Index 3 is not a slot. A loop that uses `i <= nums.length` reads that missing slot. On this array the bad read is `nums[3]`.

Subarray, subsequence, subset

Interviews use three words for a part of an array, and the words are not interchangeable. A subarray is a contiguous slice: the chosen values sit next to each other, and their order stays. A subsequence keeps that order, but gaps are allowed. A subset is only a selection: order does not matter, and the empty selection counts. Take **[2, 7, 3]**.

WORD	RULE	THE LIST
Subarray	Contiguous. Order stays.	[2], [2,7], [2,7,3], [7], [7,3], [3]
Subsequence	Order stays. Gaps allowed.	The six subarrays, plus [2,3]
Subset	A selection. Order does not matter.	{2,3} is one subset. Empty counts. 8 subsets.

The six subarrays are every contiguous block. There are $n \times (n + 1) / 2$ of them. Here n is 3, so there are 6. **[2, 3]** skips 7, so it is not a subarray. It is a subsequence, because 2 still comes before 3 in the original row. Every subarray is a subsequence. The extra subsequences are the ones with gaps.

A subset does not care about order. {2, 3} is the same subset as {3, 2}. With three distinct values you can include or exclude each one, so there are $2^3 = 8$ subsets, and one of those eight is empty. If a prompt says subarray and you allow a gap, you will count pairs the problem does not want.

In place

In place means a handful of extra variables, not a second array whose length grows with n. Building an answer array of length n is allowed by many problems, and that build is not in place. The usual in-place contract changes the input array and returns a length k. The first k slots hold the answer. The caller reads only those slots. The rest may be garbage.

Cost, in one sitting

Write the cost before you write the loops. Interview arrays often have n up to 1e5, sometimes 1e6. A modern judge runs very roughly 1e8 simple operations a second. n^2 at n = 1e5 is 1e10 operations. That is about a hundred seconds. It will time out. n log n at the same n is about 1.7e6 operations. That is fine.

N UP TO	FITS	DOES NOT FIT
100	n^3, sometimes 2^n with pruning	n^4
1,000	n^2	n^3
1e5	n log n, or n	n^2
1e6	n, or n log n if the work inside is tiny	n^2

Read the table as a gate. When n is about 100, three nested loops can finish, and a pruned exponential search sometimes finishes. Four nested loops do not. When n is about 1,000, two loops fit and three do not. When n is 1e5, you need one pass, or one pass after a sort. Two full nested loops time out. When n is 1e6, even the work inside an n log n loop has to stay small.

Empty and one element

Empty and one element are part of the template, not a special puzzle. If you read `nums[0]`, decide what an empty array returns before you write the loop. A single element has no neighbor. A loop that looks at index i and index i − 1 must start at 1, or the read of index −1 is `undefined`. If a single value is a valid run by itself, record a run of length 1. On the input `[1]`, a consecutive-ones count that never updates on a lone 1 returns 0, and the right answer is 1.

The calls you will actually make

You will add and remove at the ends, and you will sort. `push` and `pop` touch the back. `shift` and `unshift` touch the front, and every later cell slides. The front operations are the expensive ones.

SORT IS LEXICOGRAPHIC UNTIL YOU PASS A COMPARATOR.

```
const a = [10, 2, 1];
a.push(7);     // end, O(1) amortized. a is [10, 2, 1, 7]
a.pop();       // end, O(1). a is [10, 2, 1]
a.shift();     // front, O(n). everyone slides left
a.unshift(4);  // front, O(n). everyone slides right

const buggy = [10, 2, 1];
buggy.sort();                 // lexicographic: [1, 10, 2]
const fixed = [10, 2, 1];
fixed.sort((x, y) => x - y);  // numeric: [1, 2, 10]
```

`push(7)` puts 7 at the end in amortized constant time. The array grows sometimes, and those rare copies are spread across many pushes. `pop()` removes that end in constant time. `shift()` removes index 0, and every remaining value moves one slot left, so one shift is O(n). `unshift(4)` slides everyone right to open index 0. It is also O(n). If you `shift` inside a loop, n times, the cost is about n^2. On n = 1e5 that times out.

The default `sort()` does not compare numbers. It turns each value into a string and sorts those strings in dictionary order. Dictionary order means the characters decide, from the left. Take `[10, 2, 1]`. The strings are "10", "2", and "1". "1" comes before "10" because both start with the character 1, and the shorter string ends first. "10" comes before "2" because the character 1 comes before the character 2. The array becomes `[1, 10, 2]`. Ten is sitting between 1 and 2.

Any time the values are numbers, pass a comparator. The form you write is `(a, b) => a - b`. If a is 10 and b is 2, the result is 8, which is positive, so a is placed after b. If the result is negative, a comes first. If the result is 0, those two values keep their current order. The block above uses the names `x` and `y`. The subtraction is the same comparison. The same bug hides when you sort intervals later. Sort by `a[0] - b[0]`, not by the default string order.

Two names can be one array

An array is an object. A variable holds a reference, which is a name for that one array. Assignment copies the name. It does not copy the slots.

ASSIGNMENT COPIES THE REFERENCE.

```
const a = [1, 2];
const b = a;
b[0] = 9;          // a[0] is 9 as well
```

```
const copy = a.slice();
copy[0] = 0;       // a is unchanged
```

`a` starts as `[1, 2]`. `b = a` copies the reference, so `b` and `a` name the same row. `b[0] = 9` writes into that row. `a[0]` is 9 as well. `slice()` copies one level: `copy` is a new array holding the same numbers. `copy[0] = 0` changes the copy. `a` stays `[9, 2]`. The spread `[...a]` is the same kind of copy. It is shallow. Shallow means one level. If a cell holds another array, that inner array is still shared.

That fact becomes a real bug when you build a matrix. `Array(m).fill(Array(n).fill(0))` builds one inner array and stores that same reference in every row. Change one row and every row changes. Chapter 18 finishes the bug and shows the repair. Until then, remember the rule from this block: `slice` or `[...a]` copies one level, and each row of a matrix needs its own inner array.

Plus one, because digits are an array

You can store a number as an array of digits, with the most significant digit at index 0. Plus one adds 1 to the last digit. When a digit is 9, that slot becomes 0 and the 1 carries left.

Take `[1, 2, 9]`. Start at the last index, i = 2. The digit is 9. 9 is not less than 9, so you cannot add 1 in that slot. Write 0 and keep the carry. The array is now `[1, 2, 0]`. Move to i = 1. The digit is 2, and 2 is less than 9. Add 1. The slot becomes 3. Return at once. The answer is `[1, 3, 0]`. Index 0 stays 1, because the carry died at index 1. You never look at a slot after the return.

Take `[9, 9]`. At i = 1 the digit is 9, so write 0 and carry. The array is `[9, 0]`. At i = 0 the digit is 9 again, so write 0 and carry. The array is `[0, 0]`. The next index would be −1, so the loop ends, and the carry is still alive. There is no digit left to receive it. `unshift(1)` puts that 1 at the front and slides the old digits right once. The answer is `[1, 0, 0]`. You do that extra slide at most once, so the whole function is still linear in the number of digits.

PLUS ONE. [9, 9] MUST BECOME [1, 0, 0].

```
function plusOne(digits) {
  for (let i = digits.length - 1; i >= 0; i--) {
    if (digits[i] < 9) {
      digits[i] += 1;
      return digits;
    }
    digits[i] = 0;
  }
  digits.unshift(1);
  return digits;
}
```

The loop starts at the last index and walks toward 0. That direction is the carry. The test `digits[i] < 9` happens before any write of 0. When the test passes, you add 1 and return, so a later slot is left alone. When the test fails, you write 0 and the loop itself is the carry. If the loop finishes, `unshift(1)` is the leading digit. Doing that `unshift` on every step would be a different, slower algorithm. Here it runs only after every digit was 9.

```
[1, 2, 9]
i=2: 9, write 0, carry on
i=1: 2, write 3, return [1, 3, 0]

[9, 9]
both become 0, loop ends, unshift 1 → [1, 0, 0]
```

One pass from the right. O(n) time. The final unshift moves n digits once, which is still O(n).

A rotated sorted array, counted by drops

A non-decreasing array can be cut, and the front piece can be moved to the back. That shape is a rotation. It has at most one drop. A drop is a place where a value is greater than the next value. The wrap from the last index back to index 0 counts as a pair.

Take `[3, 4, 5, 1, 2]`. There are five values, so the last index is 4. Start a drop count at 0. Index 0 holds 3 and the next cell holds 4. 3 is not greater than 4, so the count stays 0. Index 1 holds 4 and the next holds 5. Still no drop. Index 2 holds 5 and the next holds 1. 5 is greater than 1, so the count becomes 1. That pair is the cut. Index 3 holds 1 and the next holds 2. No drop. Index 4 holds 2. The next index is (4 + 1) mod 5, which is 0, and that cell holds 3. 2 is not greater than 3, so the wrap is not a second drop. The count is 1. One drop is allowed, so this array passes.

Now take `[3, 5, 1, 4, 2]`. Index 1 holds 5 and the next holds 1, which is a drop. Index 3 holds 4 and the next holds 2, which is another drop. Two drops means this is not a rotation of a non-decreasing array. The function returns false. Equal neighbors are not drops, so `[1, 1, 1]` has a count of 0 and passes. A fully sorted array also has a count of 0 and passes. The test is `drops <= 1`, which allows zero or one. You will not binary-search this shape until chapter 14. Counting the drops is enough here.

AT MOST ONE DESCENT, INCLUDING THE WRAP.

```
function check(nums) {
  let drops = 0;
  for (let i = 0; i < nums.length; i++) {
    const next = nums[(i + 1) % nums.length];
    if (nums[i] > next) drops += 1;
  }
  return drops <= 1;
}
```

`drops` starts at 0 so a sorted array can still succeed. The loop index runs from 0 through n – 1. `(i + 1) % nums.length` is what makes the last step read index 0. You add 1 only when `nums[i] > next`. A tie is allowed, because non-decreasing includes equals. The return uses `<=`. If you required `drops === 1`, the input `[1, 2, 3]` would fail, and it is already sorted.

Mistakes

- On `[10, 2, 1]`, a loop condition `i <= nums.length` reads index 3. That read is `undefined`. A later numeric step becomes NaN, or a comparison quietly does nothing.
- On `[10, 2, 1]`, `sort()` with no comparator returns `[1, 10, 2]`. Any later scan that assumes 10 is the largest value is wrong.
- Building a second array of length n and calling the algorithm in place is the wrong cost. The extra memory grows with n. In place means a handful of variables.
- `for...in` on an array walks keys, and those keys are strings. A property sitting on the prototype can show up as a fake index. Use an index loop or `for...of`.
- Calling `shift` once per element on an array of length n slides the tail n times. The input that shows the cost is a long row, n = 1e5. The time is about n^2, and the judge times out.

Practice

Stay problems are the mechanics of indexes, copies, and counts. Then problems are those same mechanics with a carry, a digit count, or a wrap.

Stay before chapter 1. Then before you leave the chapter.

1929. Concatenation of Array EASY STAY

Build an array of length 2n. ans[i] and ans[i + n] both read nums[i].

1920. Build Array from Permutation EASY STAY
ans[i] = nums[nums[i]]. Two reads. Do not confuse the index with the value.

1470. Shuffle the Array EASY STAY
The array is x1..xn y1..yn. Place them as x1,y1,x2,y2 by writing index 2*i and 2*i+1.

1512. Number of Good Pairs EASY STAY
A pair is two indices i < j with the same value. Count frequencies, then add c*(c-1)/2 for each count.

1431. Kids With the Greatest Number of Candies EASY THEN
One pass for the max, a second pass to compare candies[i] + extraCandies with that max.

1295. Find Numbers with Even Number of Digits EASY THEN
Count digits by dividing by 10, or by the length of the decimal string. Zero is one digit.

66. Plus One EASY THEN
The template above. The only new digit is a leading 1 when every digit was 9.

989. Add to Array-Form of Integer EASY THEN
Add k from the right. The carry is Math.floor(sum / 10), and k shrinks the same way.

1752. Check if Array Is Sorted and Rotated EASY THEN
Count strict drops, including the pair (last, first). The answer is drops <= 1.

REWRITE FROM MEMORY

Close the page and write the comparator (a, b) => a - b, the plus-one loop on [9, 9], and the six subarrays of [2, 7, 3] from memory.

Carry forward. Chapter 1 uses index, subarray, and in place to classify a prompt before you choose a loop.

CHAPTER 01 · BUILDS ON CHAPTER 0

How to read an array problem

Four lines before any code: the input, the output, whether the answer is contiguous or in place, and what the brute force costs.

You can already name an index, a subarray, and an in-place write. An index is a position, and the first one is 0. A subarray is a contiguous slice. In place means a handful of extra variables, not a second array of length n. The next skill is classification. You refuse to code until four lines on paper tell you which later chapter the prompt belongs to.

The 25-minute habit lives in the study chapter, the page before chapter 0. When you are stuck, that page tells you to set a timer and then reread the recognition for the chapter you already chose. This chapter does not use that timer. The skill here is the choice itself. You name the shape. You do not reach for a clever loop yet.

The four lines

Write four lines before any code. Each line is one question. Chapter 3 is the brute force those lines point at. Chapters 4 through 16 exist because the fourth line often says the brute force is too slow. You do not need those chapters memorized today. You need the four lines, so the chapter number is a conclusion.

Input. What is n, roughly? Are values sorted? Can they be negative? Can they be zero? n is the length. If n can be 1e5, two nested loops will not finish, which you already priced in chapter 0. If the values are sorted, a later chapter can throw away one side after a single comparison. If negatives are allowed, a running total can fall and then recover, and a band that only shrinks can miss an answer. Zero matters when you count digits, multiply signs, or divide. Write the answers you actually have, including "negatives are forbidden" when the prompt says so.

Output. A boolean, an index, a length, a count of subarrays, or a rebuilt array? A boolean often means you only need to know that something has been seen. An index means you must store a position, and index 0 is a real position, not a missing one. A length paired with an in-place write means the caller reads a prefix of the same array. A prefix is the contiguous block from index 0 through some index. A count of subarrays means you are visiting slices, not only pairs. A rebuilt array of length n is extra memory the problem asked for. Building it is not a failure, and it is not in place.

Shape. Does the answer have to be contiguous? Does it have to be in place? Contiguous means a subarray: the values sit next to each other and the order stays. A subsequence keeps order and allows gaps. A subset ignores order. If the prompt says subarray and you allow a gap, the loops are wrong before they start. If it says in place, a second array of length n breaks the contract even when the values are right. Chapter 0 fixed these words. Use them in this line.

Brute cost. If you tried every pair, every subarray, or every possible answer, what is the cost, and does the constraint allow it? Pairs are about n^2. Subarrays with a fresh sum inside are also about n^2. Trying every possible speed one by one is often too slow, even when a later chapter can search those speeds in order. Write the Big-O before you nest a loop. If that cost fits the table from chapter 0, the brute force is the answer. If it does not fit, the four lines have told you why a later chapter exists. They have not told you to guess its name from a keyword.

How a prompt points at a later chapter

After the four lines, you still say the direction in a full sentence. You are not choosing syntax. You are naming the family the lines imply.

- If the value you need at each index is a sum, a count, a minimum, or a maximum of what you have already seen, you are heading toward one pass.

- If the answer is one contiguous band and a running total is enough to accept or reject that band, you are heading toward one pass, a window, or a prefix.
- If any pair will do, order does not matter, and the values are not sorted, you are heading toward a map, or toward a sort.
- If the array is already sorted and each comparison lets you throw away one side, you are heading toward two indexes that move inward.
- If n is a few hundred and the prompt asks you to count every pair or every triple, you are heading toward two loops, and that cost is allowed.
- If the prompt asks for the minimum speed, capacity, or day such that a plan works, you are heading toward a search over the answer, not a search over an index.

Use chapter 2 when the answer at index i needs only the current value and a short summary of one side.

Use chapter 3 when n is small and every pair, every triple, or every subarray is cheap enough to write directly.

Use chapter 7 when every value that the condition cares about is positive, the answer is a contiguous band, and shrinking the left end is what repairs the band.

Use chapter 8 when you count subarrays with a given sum and a negative value can repair a sum you have already walked past.

Use chapter 14 when you are searching a speed, a capacity, or a day, and once a candidate works every larger candidate also works.

Use chapter 15 when each index wants the nearest later value that is strictly greater, such as the next warmer day.

What to do with the eight drills

For each prompt below, write the chapter number you think it will land in, and one sentence that quotes your four lines. Do not write code. The answer key is in chapter 17. Leave that chapter closed until all eight guesses are on paper. If you open it first, you skip the skill this chapter is teaching. After each guess, check it against the brute-cost line. If that cost already fits, do not reach for a later chapter just because the later chapter sounds advanced.

1. Longest subarray whose sum is at most K. Every value is positive.
2. Count the subarrays whose sum is exactly K. Values may be negative.
3. Two indices in a sorted array add to a target.
4. Remove duplicates from a sorted array, in place.
5. For each day, how many days until a strictly warmer temperature.
6. How many islands are in a grid of land and water.
7. Add a value on many index ranges, then read the final array.
8. The minimum eating speed so the piles are finished within H hours.

The wording above is the whole prompt you get. You fill in n and the shape yourself, the way the four lines ask. A good sentence names contiguous or in place when the prompt used that idea, and it names a cost. Chapter 17 will show the landing chapter. Your job in this chapter is to be able to defend the guess from the four lines.

Mistakes

- The failure starts when a prompt looks like one you have seen, and you code before the four lines. A familiar word such as maximum sends you into a stock loop. On a later input whose values may be negative, that loop tracks a minimum price and returns 0, while the asked answer was a different kind of maximum.

- Calling every contiguous problem a sliding window fails on a count of subarrays whose sum is exactly K when negatives are allowed. A window that only shrinks cannot be repaired by a later negative, so it under-counts. Chapter 7 and chapter 8 exist so you can tell those prompts apart.
- Guessing the chapter from one keyword fails in both directions. The word maximum by itself is not a reason to use the best-subarray-sum loop. The word sorted by itself is not a reason to binary-search. The four lines are what decide.

REWRITE FROM MEMORY

Close the page and write the four lines, then classify the eight drills again without looking at chapter 17.

Carry forward. Chapter 2 is the first algorithm: one pass, where the answer at i depends only on the current value and a summary of what you have already seen.

CHAPTER 02 · BUILDS ON CHAPTER 1

One pass and running state

The answer at i is a function of the current value and a summary of the prefix you have already walked.

What this pattern is

One pass means you visit each index once. You walk left to right, or you walk right to left. You do not start the array over. A running state is a small summary of the indexes you have already visited. The summary might be a sum, a count, a minimum, or a maximum. At the current index you may use the value in that cell and the summary. You may not rescan the earlier cells.

An index is the position of a cell. The first index is 0. A prefix is the contiguous block from index 0 through the index you have just finished. An invariant is a sentence about your variables that stays true every time you finish an index. Here the invariant is: the summary describes exactly the prefix you have walked, and the answer at this index is a function of that summary and the current value.

Picture a walk down a street of shops. You do not memorize every price. You remember the cheapest price so far, which is one number. At the next shop you ask one question with that memory, and only then do you update the memory. The array is the street. The running state is the note in your pocket.

What you already know

Chapter 0 gave you indexes, the cost table, and the meaning of in place. Chapter 1 gave you four lines so you can name this family before you code. This chapter adds the first algorithm. The answer at i depends only on `nums[i]` and a summary of one side. If the summary would have to drop a cell in the middle and remember it again later, you are past this chapter.

A worked example

Take the array `[1, 2, 3, 4, 5]`. It holds five values, so the last index is 4. Index 0 holds 1. You keep one number, called sum, and it starts at 0. That 0 is the sum of the empty prefix, before any cell is included. At index 0 you add 1, so sum becomes 1, and you record 1. You add before you record, because this cell has to be inside the sum you store. At index 1 the value is 2. Sum becomes 3, and you record 3. You do not read index 0 again, because that 1 is already inside sum. At index 2 the value is 3 and sum becomes 6. At index 3 the value is 4 and sum becomes 10. At index 4 the value is 5 and sum becomes 15. The recorded array is `[1, 3, 6, 10, 15]`.

After each record, sum equals the last recorded value, and the recorded list is exactly the prefix sums of the cells you have walked. Nothing to the right of the current index has been added. If you recorded sum before adding the current value, index 0 would store 0, and the final 15 would never be written. On the smaller input `[1, 2]` that swapped order returns `[0, 1]` instead of `[1, 3]`.

```
[1, 2, 3, 4, 5]
index 0: sum 1, record [1]
index 1: sum 3, record [1, 3]
index 2: sum 6, record [1, 3, 6]
index 3: sum 10, record [1, 3, 6, 10]
index 4: sum 15, record [1, 3, 6, 10, 15]
```

How you recognise it

- The value you need at i is a sum, a count, a minimum, or a maximum of the prefix you have walked, and you do not need the exact earlier index that produced it.
- One buy and one later sell can be answered from the cheapest price seen so far, as long as you sell against the old minimum.

- A run of equal values, or of strictly increasing values, resets when the rule breaks, and you only keep the current length and the best length.
- The answer at i is the greatest value strictly to the right, so one walk from the last index can carry that best value.
- A balance of the left sum and the right sum becomes arithmetic once a first pass has computed the total.
- If the correct summary would have to forget a cell that fell out of a moving band, this chapter is the wrong tool.

Use chapter 7 when the answer is one contiguous band and dropping the left end is what repairs the band.

Use chapter 11 when you want the best sum of any contiguous segment and a negative prefix should be thrown away.

Use chapter 15 when you need the nearest greater value, not the greatest value anywhere to the right.

What to say out loud

I walk each index once. When I finish index i, the summary matches the prefix I have walked, and it does not include anything past i. The answer uses that summary together with the current value. If the answer needs the old summary, I use the old summary before I change it.

The template

THE SUMMARY IS THE SUM SO FAR.

```
function runningSum(nums) {
  const out = [];
  let sum = 0;
  for (const x of nums) {
    sum += x;
    out.push(sum);
  }
  return out;
}
```

The invariant is: after each push, `out` holds the prefix sums of the values already seen, and `sum` equals the last of those prefix sums. `out` starts empty and `sum` starts at 0, which is the empty prefix. Inside the loop you add `x` and only then push. That order is the off-by-one. Pushing first stores the previous prefix in the current slot and drops the final sum on the floor. The loop has no index, so you cannot mis-count a bound. You push once per value, so the output length equals the input length. An empty input never enters the loop and returns an empty array.

Highest altitude is this same loop on a list of gains. The altitude starts at 0, which counts as a place you have touched. Add each gain into a running sum, and keep the largest altitude you have seen, including the start. Take gains `[-5, 1, 5, 0, -7]`. The altitudes are 0, then −5, then −4, then 1, then 1, then −6. The best is 1. The final altitude is −6, and that final number is not the answer. A negative gain can pull you down after the best point.

A run that resets

Take `[1, 1, 0, 1, 1, 1]`. You want the longest contiguous block of ones. `run` is the current block. `best` is the longest block finished so far. Both start at 0. At index 0 the value is 1, so run becomes 1 and best becomes 1. At index 1 the value is 1, so run becomes 2 and best becomes 2. At index 2 the value is 0. A zero ends the block, so run goes back to 0. Best stays 2, because the earlier block is still the champion. At index 3 the value is 1 and run becomes 1. Best stays 2, because 1 is not longer. At index 4 run becomes 2. At index 5 run becomes 3, and best becomes 3. The answer is 3. The zero threw away the current run only.

On the input `[1]` the same steps set run to 1 and best to 1. A single 1 is a real block. If best stayed 0 until a zero arrived, this input would return 0.

THE SUMMARY IS THE CURRENT RUN. A ZERO THROWS IT AWAY.

```
function findMaxConsecutiveOnes(nums) {
  let best = 0;
  let run = 0;
  for (const x of nums) {
    if (x === 1) {
      run += 1;
      if (run > best) best = run;
    } else {
      run = 0;
    }
  }
  return best;
}
```

On a 1 you add to `run` first, then you compare with `best`. On a 0 you set `run` to 0 and you do not touch `best`. Updating best before the increment would store a length that is one short. Resetting best on a zero would forget the earlier champion. The invariant is: best is the longest run entirely inside the prefix you have finished, and run is the length of the unfinished block that touches the current index, or 0 if the current value broke it.

```
[1, 1, 0, 1, 1, 1]
run 1, 2, then 0 resets, then 1, 2, 3
best = 3
```

The same loop changes only the reset rule. A longest strict increase resets the run to 1, not to 0, when the value does not grow, because the current cell still starts a new block of length 1. A single element is length 1. Consecutive characters is the same run, compared on letters. A large group remembers the start index of the current character, and when the run ends you record that start and end if the length is at least 3. A mountain is a strict climb and then a strict descent. The peak cannot be index 0 or the last index, and the walk has to finish on the last index. A monotonic check keeps two flags, still non-decreasing and still non-increasing. An equal neighbor leaves both flags alive. The sign of a product is not a multiplication. A zero returns 0. Otherwise you count the negatives, and the sign is the parity of that count.

Best day to sell

Take prices `[7, 1, 5, 3, 6, 4]`. There are six days. You may buy once and sell once, and the sell index must be later than the buy index. `minPrice` is the cheapest earlier price. It starts at infinity, so no fake buy exists yet. `best` starts at 0, because skipping the trade is allowed. At index 0 the price is 7. Profit against infinity is not a gain, so best stays 0. Only after that check does 7 become the minimum. At index 1 the price is 1. Profit is $1 - 7$, a loss, so best stays 0. Then 1 is cheaper than 7, so the minimum becomes 1. You used the old minimum, 7, before you stored 1. At index 2 the price is 5. Profit is $5 - 1 = 4$, so best becomes 4. The price 5 is not a new minimum. At index 3 the price is 3. Profit is 2, which does not beat 4. At index 4 the price is 6. Profit is $6 - 1 = 5$, so best becomes 5. At index 5 the price is 4. Profit is 3, and best stays 5. The answer is 5. That sale buys at index 1 and sells at index 4.

PROFIT USES YESTERDAY'S MINIMUM. THEN THE MINIMUM MAY MOVE.

```
function maxProfit(prices) {
  let minPrice = Infinity;
  let best = 0;
  for (const price of prices) {
    const profit = price - minPrice;
    if (profit > best) best = profit;
    if (price < minPrice) minPrice = price;
  }
  return best;
}
```

The invariant is: `best` is the best sale on a day you have already finished, and `minPrice` is the cheapest price strictly before any sale you might make tomorrow. Infinity at the start means the first day has no earlier buy. The profit line reads `minPrice` before the update line writes it. That order is the whole algorithm. The off-by-one is not an index bound. It is those two `if` statements. If the minimum moves first, today is both the buy and the sell, and today's profit is 0.

```
[1, 2]
correct: price 1, profit is −Infinity so best stays 0, then min = 1.
         price 2, profit = 1, best = 1.

broken, if you write the min first:
price 1, min = 1, profit = 0
price 2, min = 2, profit = 0
answer 0, and you can never sell above the min you just stored.
```

The greatest value on the right

Sometimes the summary lives to the right of i. Take `[17, 18, 5, 4, 6, 1]`. You replace each cell with the greatest value strictly to its right. The last cell has nothing to its right, so its answer is −1. Start `best` at −1, and start the index at 5. Save 1, write −1 into index 5, then let best become 1 because 1 is greater than −1. At index 4 save 6, write 1, then best becomes 6. At index 3 save 4, write 6, and leave best at 6 because 4 is smaller. At index 2 save 5, write 6, and leave best at 6. At index 1 save 18, write 6, then best becomes 18. At index 0 save 17, write 18, and leave best at 18 because 17 is smaller. The array is `[18, 6, 6, 6, 1, −1]`.

This is the greatest value anywhere to the right. It is not the nearest greater value. A smaller neighbor can still be the nearest greater one, and one running maximum cannot remember that wait. That wait is chapter 15.

REPLACE, THEN CONSIDER THE OLD VALUE AS A CANDIDATE.

```
function replaceElements(arr) {
  let best = -1;
  for (let i = arr.length - 1; i >= 0; i--) {
    const cur = arr[i];
    arr[i] = best;
    if (cur > best) best = cur;
  }
  return arr;
}
```

You save `cur` before the write. Then you store `best` into the cell. Only then may `cur` raise `best`. If you raised `best` first, the cell would be replaced by a maximum that includes itself. The last index would become its own value instead of −1. If you wrote the cell before saving `cur`, the old value would be gone. On `[17, 18, 5, 4, 6, 1]` the 1 at index 5 has to be saved before −1 overwrites it, or `best` never becomes 1 and every earlier answer shifts. The loop bound is `i >= 0`. The step `i--` is what moves you left. `best` starts at −1 so the rightmost write has a defined answer.

```
[17, 18, 5, 4, 6, 1]
from the right, best starts at -1
1 → write -1, best = 1
6 → write 1, best = 6
4 → write 6
5 → write 6
18 → write 6, best = 18
17 → write 18
result [18, 6, 6, 6, 1, -1]
```

The pivot index

A pivot index has an equal sum on both sides. The pivot value itself belongs to neither side. You cannot know the right-hand sum until you know the total, so the first walk only adds. Take `[1, 7, 3, 6, 5, 6]`. The total is 28. `left` starts at 0. At index 0, right is 28 − 0 − 1 = 27. 0 is not 27, so now left becomes 1. At index 1, right is 28 − 1 − 7 = 20. Not equal, so left becomes 8. At index 2, right is 28 − 8 − 3 = 17. Not equal, so left becomes 11. At index 3, right is 28 − 11 − 6 = 11. The two sides match, so you return 3. You return on the match, before left absorbs 6. Indexes 4 and 5 are never needed once 3 has matched. If no index matches, you return −1.

THE TOTAL TURNS ONE FUTURE SUM INTO ARITHMETIC.

```
function pivotIndex(nums) {
  let total = 0;
  for (const x of nums) total += x;
  let left = 0;
  for (let i = 0; i < nums.length; i++) {
    const right = total - left - nums[i];
    if (left === right) return i;
    left += nums[i];
  }
  return -1;
}
```

The first loop finishes the total before any comparison. In the second loop the order is fixed. Compute `right` from the old `left` and from `nums[i]`. Compare. Only then add `nums[i]` into `left`. The pivot is in neither sum, which is why it is subtracted out of the right-hand side and not yet added to the left. If you add first, index 3 on this input has left 17 and right 5, the match disappears, and you return −1. An empty array skips both loops and returns −1. A one-element array has left 0 and right 0, so index 0 is a pivot.

```
[1, 7, 3, 6, 5, 6], total 28
i=0 left 0, right 27, no, left becomes 1
i=1 left 1, right 20, no, left becomes 8
i=2 left 8, right 17, no, left becomes 11
i=3 left 11, right 11, return 3
```

The total is the sum of the whole array. The right-hand sum is that total, minus the left prefix, minus the pivot cell. This is the bridge into prefix sums. Chapter 8 will name the prefix array. Here two numbers are enough.

Time and extra memory

Every walk in this chapter visits each index a constant number of times, so the time is O(n). Extra memory is O(1), except the running sum, which allocates the output array the problem asks for.

Mistakes

- If you update the minimum price before the profit, the input `[1, 2]` returns 0. Day 0 stores 1 and records profit 0. Day 1 stores 2 before the subtraction, so the profit is 0 again. The sale of 2 against the old minimum 1 is never computed. The right answer is 1.
- If a run-length answer stays 0 until a reset, the input `[1]` returns 0. A single 1 is a run of length 1. The best has to move on that lone 1.
- If you overwrite a cell and then read it as the greatest value on the right, the input `[17, 18, 5, 4, 6, 1]` loses the 1 at the last index. Best never becomes 1, and the written answers are wrong. Save the old value, write the answer, then consider the old value.
- If you add `nums[i]` into the left sum before the comparison, the input `[1, 7, 3, 6, 5, 6]` misses index 3. The pivot belongs to neither side. Adding 6 first makes the left side 17 and the right side 5, so you return −1.
- If the running sum pushes before it adds, the input `[1, 2]` returns `[0, 1]`. Each slot holds the previous prefix, and the last sum is never stored.

Practice

Stay before chapter 3. Then before you leave the chapter.

1480. Running Sum of 1d Array EASY STAY
The running-sum template. out[i] is the sum of the prefix ending at i.

485. Max Consecutive Ones EASY STAY
Reset the run on 0. Track the best run. Empty input returns 0.

121. Best Time to Buy and Sell Stock EASY STAY
One buy, one sell, sell after buy. Profit before you move the minimum.

1299. Replace Elements with Greatest Element on Right Side EASY STAY
Walk from the right. The last answer is -1.

724. Find Pivot Index EASY STAY
Total first, then left versus total - left - nums[i]. Return the leftmost index.

1732. Find the Highest Altitude EASY THEN
Running sum of the gains. The best altitude includes the start at 0.

674. Longest Continuous Increasing Subsequence EASY THEN
Reset the run to 1 when the value does not strictly increase. A single element is length 1.

1446. Consecutive Characters EASY THEN
The same run, compared on characters instead of bits.

830. Positions of Large Groups EASY THEN
When a run of the same character ends, record [start, end] if the length is at least 3.

896. Monotonic Array EASY THEN
Two flags: still non-decreasing, still non-increasing. Equals keep both flags alive.

941. Valid Mountain Array EASY THEN
Strict climb, then strict descent. The peak cannot be the first or the last index.

1822. Sign of the Product of an Array EASY THEN
A zero returns 0. Otherwise the sign is the parity of the negative count. Do not multiply.

1991. Find the Middle Index in Array EASY THEN
The same balance as the pivot index. Left sum equals right sum.

REWRITE FROM MEMORY

Rewrite the stock loop and the right-to-left replacement from a blank file, and keep the old minimum in use until the profit line has run.

Carry forward. Chapter 3 drops the summary and writes both loops, so you can see the $O(n^2)$ scan that later chapters replace.

CHAPTER 03 · BUILDS ON CHAPTER 2

Two loops

When n is small, write the pairs or the subarrays directly. Reset the inner sum at every new start.

What this pattern is

Two loops means you name two indexes, and the inner index runs across the partners of the outer index. You use this when you truly need every pair, or every subarray, and n is small enough that n^2 fits. A pair is two indexes i and j with i before j. A subarray is a contiguous slice: you pick a start, then you extend the end one cell at a time. There is no running summary that can replace those visits. The visits are the algorithm.

An index is a position, and the first one is 0. An invariant here is small. When the outer index moves, the inner sum goes back to 0, so the new start does not inherit the previous start. Picture checking every window of a short shelf by hand. The shelf is short, so touching every window is honest work. When the shelf grows to 1e5 items, this chapter is the thing you time out on, and a later chapter has to replace it.

What you already know

Chapter 2 answered an index from a summary of one side. This chapter is what you write when a summary is the wrong tool and the cost table from chapter 0 says n^2 still fits. You already know a subarray has no gaps. This chapter adds the double loop that lists them, and the habit of resetting the inner sum at every new start.

A worked example

Take `nums = [1, 2, 1, 2, 1]` and `k = 3`. There are five values, so there are $5 \times 6 / 2 = 15$ subarrays, and the loops will touch all 15. Start i at 0 and set sum back to 0. j walks from 0. Sum becomes 1, then 3, which matches, so the count becomes 1. That subarray is indexes 0 through 1. Sum then becomes 4, then 6, then 7. None of those later sums is 3. Now i moves to 1. This is the step people skip. Sum is set back to 0 before j starts at 1. If you forgot that reset, sum would still be 7 and every later total would be nonsense. From i = 1 the sums are 2, then 3. The second 3 is indexes 1 through 2, and the count becomes 2. The longer sums from this start are 5 and 6.

i moves to 2, and sum is 0 again. j starts at 2. The first sum is 1. Then j = 3 makes sum 3, indexes 2 through 3, and the count becomes 3. The last extension is 4. i moves to 3. Sum starts at 0. Index 3 alone is 2. Indexes 3 through 4 sum to 3, and the count becomes 4. i moves to 4. The only sum is 1. The count stays 4. The four hits are `[1, 2]` at the start, `[2, 1]` next, `[1, 2]` starting at index 2, and `[2, 1]` starting at index 3.

A pair loop is a different inner start. j begins at i + 1, so you never compare a value with itself, and you never count (j, i) again after you have counted (i, j). The code below is the subarray version, which is why j begins at i and the sum is reset there.

```
nums = [1, 2, 1, 2, 1], k = 3
i=0 sums 1, 3 hit, 4, 6, 7
i=1 sums 2, 3 hit, 5, 6
i=2 sums 1, 3 hit, 4
i=3 sums 2, 3 hit
i=4 sums 1
count = 4
subarrays in length n: n * (n + 1) / 2
n = 5 → 15, and the loops visit all 15
```

How you recognise it

- n is a few hundred, or the prompt says to count every pair, and the cost table says n^2 fits.
- You need each pair of indexes with the left index strictly before the right index.
- You need each subarray, meaning each start i and each end j that is i or to the right of i.

- Nothing is sorted, there is no complement you can store, and no end of a window is safe to discard.
- Three indexes with i before j before k, and n at most about 100, is the same idea with one more loop.
- If the inner question is only "have I already seen one special value", a later chapter removes the inner loop.

Use chapter 4 when the inner loop only asks whether you have already seen the complement of the current value.

Use chapter 8 when n is large and you count subarrays whose sum is k, including when negatives are allowed.

Use chapter 15 when the inner scan looks for the next smaller or greater value to the right of i.

What to say out loud

I will name the cost before I nest the loops. For pairs, the inner index starts at i + 1. For subarrays, the inner sum goes back to 0 at every new start, and the inner index starts at i. If that cost does not fit, I stop and I do not pretend the nested loops are the final answer.

The template

RESET THE SUM AT EVERY START. THIS IS THE ALGORITHM CHAPTER 8 REPLACES.

```
function countSubarraysWithSum(nums, k) {
  let count = 0;
  for (let i = 0; i < nums.length; i++) {
    let sum = 0;
    for (let j = i; j < nums.length; j++) {
      sum += nums[j];
      if (sum === k) count += 1;
    }
  }
  return count;
}
```

The invariant is: when j moves, `sum` is the sum of indexes i through j, and `count` includes every earlier subarray whose sum was k. `let sum = 0` sits inside the outer loop, which is the reset. The inner loop starts at `j = i`, so a single cell is a subarray. You add `nums[j]` before you compare with k. Comparing first would test the previous end. The off-by-one is the inner start. `j = 0` would revisit cells left of i and would count the same slice many times. `j = i + 1` would skip every one-cell subarray.

Final prices with a discount, solved in this chapter, is the pair shape. For each i, scan j to the right for the first price that is less than or equal to `prices[i]`. Subtract that later price when you find it. The same question becomes a stack in chapter 15, because "the next smaller price" is the next-smaller problem. Write the pair version until you can feel which inner scans are only searching for a neighbor.

Time and extra memory

Pairs are O(n²) and subarrays visited this way are O(n²), with O(1) extra memory. Three nested index loops are O(n³). At n = 100 that is about a million steps and usually fits. At n = 1e5, n² does not.

Mistakes

- If the inner pair loop starts at j = 0, the input `[1, 2]` counted as pairs with i before j becomes two visits, (0, 1) and (1, 0). You double-count, or you use a value twice when you forget to skip i === j.
- If the subarray sum is not reset when i moves, the input `[1, 1, 1]` with k = 2 fails after the first start. The first start ends with sum 3. The second start adds onto 3 and never sees the subarray `[1, 1]` that begins at index 1. The count comes out short.
- Reaching for a map on an input of length 50 still computes an answer, and it hides the double loop this chapter is teaching. The nested loops are the algorithm while n² fits. Use them until the fourth line of chapter 1 says they do not fit.

Practice

Stay before chapter 4. Then before you leave the chapter.

1365. How Many Numbers Are Smaller Than the Current Number EASY STAY
For each i, count j where nums[j] < nums[i]. n is at most 500. The pair loops are the point.

1588. Sum of All Odd Length Subarrays EASY STAY
Enumerate subarrays, add the sum when the length is odd. Later you can also count how many odd-length subarrays cover each index.

1534. Count Good Triplets EASY THEN
Three loops, i < j < k, and three absolute-value checks. n is at most 100.

1475. Final Prices With a Special Discount in a Shop EASY THEN
For each i, the first j > i with prices[j] <= prices[i]. Brute force now. The stack version is chapter 15.

REWRITE FROM MEMORY

From a blank file, rewrite the double loop that counts subarrays with sum k, including the reset of sum inside the outer loop.

Carry forward. Chapter 4 takes the inner question "have I seen the complement?" and answers it with a map.

CHAPTER 04 · BUILDS ON CHAPTERS 2 AND 3

Hash map

The inner loop asked whether you have seen the other value that finishes the pair. Look that up, then insert.

What this pattern is

A hash map stores a value under a key so you can ask for that key later without scanning. In this chapter the key is usually a number you have already seen. The stored thing is an index, a count, or a list. A complement is the other value that finishes a pair. If the target is T and the current value is x, the complement is T − x. The inner loop of chapter 3 often asked only "have I seen the complement?" The map answers that in expected constant time.

An index is a position, and 0 is a real index. An invariant is a sentence that stays true at the start of each loop step. Here it is: the map holds values from indexes strictly before i, and the current value is not in the map yet. You look up the complement first. You insert the current value after that lookup. Picture a coat check. You ask whether the matching ticket is already on the board. You hang your own coat after the question, so you cannot match a coat with itself.

What you already know

Chapter 2 kept a summary of one side. Chapter 3 answered "have I seen this?" by scanning. This chapter keeps the question and replaces the scan with a lookup. You still walk the array once. The new tool is the map, and the new rule is the order: lookup, then insert.

A worked example

Take `nums = [8, 4, 5, 1, 7, 2]` and target 10. There are six values. Use a Map. At index 0 the value is 8, so the complement is 2. The map is empty, so there is no hit. Only then do you store 8 at index 0. At index 1 the value is 4 and the complement is 6. 6 is absent, so you store 4 at index 1. At index 2 the value is 5 and the complement is 5. 5 is not in the map yet, so this 5 does not pair with itself. You store 5 at index 2. At index 3 the value is 1 and the complement is 9. Miss, then store 1 at index 3. At index 4 the value is 7 and the complement is 3. Miss, then store 7 at index 4. At index 5 the value is 2 and the complement is 8. 8 is in the map at index 0. You return `[0, 5]`.

Index 0 is a real index in that answer. A plain object would store the key as a string, and a truthiness check treats a stored 0 as missing. `if (obj[need])` is false when the stored index is 0, so this same pair would be skipped. `Map.has` does not care that the index is 0.

The insert-after rule shows up on a one-element input. Take `[3]` with target 6. The complement is 3. The map is empty, so there is no hit. Then you insert 3 at index 0. You do not return `[0, 0]`. One 3 is not used twice. If you inserted before the lookup, the map would already contain 3, the lookup would hit, and you would pair the element with itself. Take `[3, 3]` with target 6 as the allowed version of a repeated value. Index 0 misses and stores 3 at 0. Index 1 looks up 3, hits index 0, and returns `[0, 1]`. Those are two different indexes.

```
[8, 4, 5, 1, 7, 2], target 10
i=0 need 2, miss, store 8 at 0
i=1 need 6, miss, store 4 at 1
i=2 need 5, miss, store 5 at 2
i=3 need 9, miss, store 1 at 3
i=4 need 3, miss, store 7 at 4
i=5 need 8, hit index 0, return [0, 5]

[3], target 6: need 3, map empty, then store. Not [0, 0].
object bug: if (obj[need]) is false when the stored index is 0
```

How you recognise it

- The array is not sorted, and the question at i is whether the complement target − x has already appeared.
- You need a count of each value, the last index of a value, or the list of items that share one key.
- You need the longest run of consecutive numbers, and the order of the array does not matter.
- A repeated value should return true as soon as the set already contains it.
- Two strings are anagrams when one frequency map built from the first string is consumed exactly by the second.
- If the array is sorted and each comparison can throw away an end, a map is extra memory you do not need.

Use chapter 5 when the array is sorted and moving one end replaces the map.

Use chapter 3 when n is small enough that the nested loops are the whole answer.

Use chapter 8 when the key you look up is an earlier prefix sum, not the value itself.

What to say out loud

I look up the complement before I insert the current value, so a value is never paired with itself. I use a Map, because index 0 is a real index and an object check would treat it as missing. The map holds only indexes I have already finished.

The template

MAP, NOT AN OBJECT. LOOKUP, THEN INSERT.

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);
  }
  return [];
}
```

`seen` is a Map. The loop index starts at 0 and stops before `nums.length`, so every real index is visited and index n is not. `need` is the complement. `seen.has(need)` runs before `seen.set`. That order is the off-by-one that matters. Set first, and a value whose double is the target matches itself. The return gives the stored index and then i, which are two different positions. If the loop ends, there was no pair, and you return an empty array. A frequency update later in the chapter is `freq.set(x, (freq.get(x) ?? 0) + 1)`. `??` replaces only `undefined`. A Set is the same idea when you store membership and nothing else.

The map can store more than the complement's index. An index of the complement is this template, inserted after the lookup. A last index answers "contains duplicate II": if this value was seen at j and i − j is at most k, return true, then store i. A frequency answers anagrams, good pairs, and intersections that must respect duplicate copies. Intersection II walks one array's counts down as you accept values from the other. A group stores a list under a shared key.

Groups that share a key

Take `["eat", "tea", "tan", "ate", "nat", "bat"]`. The key of a word is its characters sorted and joined. Index 0 is "eat", and the key is "aet". The map has no list for "aet" yet, so you create one and push the original "eat". Index 1 is "tea". Its key is also "aet", so you push "tea" onto that same list. Index 2 is "tan", key "ant", a new list. Index 3 is "ate", key "aet", pushed onto the first list. Index 4 is "nat", key "ant". Index 5 is "bat", key

"abt", a new list. The lists are `["eat", "tea", "ate"]`, `["tan", "nat"]`, and `["bat"]`. A Map keeps insertion order, so the groups come out in the order of their first word.

THE SORTED SPELLING IS THE KEY. THE ORIGINAL SPELLING IS THE VALUE.

```
function groupAnagrams(strs) {
  const groups = new Map();
  for (const s of strs) {
    const key = s.split("").sort().join("");
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(s);
  }
  return [...groups.values()];
}
```

You build the key first. You create the list only when `has` says it is missing. You push `s`, the original spelling, after the list exists. `split` makes a new array of characters, so the sort does not rewrite `s`. If you sorted `s` itself and pushed that, the returned words would be the keys. The spread at the end copies the lists out of the map. An empty input returns an empty list of groups.

Longest consecutive run

Put every value in a set. Duplicates collapse, because a set stores each number once. A run should be counted once, from its start. The start is a value x whose predecessor x − 1 is absent. Take `[100, 4, 200, 1, 3, 2]`. 100 has no 99, so it starts. The next value 101 is absent, and the length is 1. Best becomes 1. 4 does not start, because 3 is in the set. You skip it. 200 has no 199, so it starts a run of length 1. Best stays 1. 1 has no 0, so it starts. You walk 1, then 2, then 3, then 4. The next value 5 is absent. The length is 4, and best becomes 4. 3 does not start, because 2 is present. 2 does not start, because 1 is present. The answer is 4.

If you started a walk at every number, the run 1, 2, 3, 4 would be walked from 1, from 2, from 3, and from 4. On a long run of length n the inner walks cost about n^2. Starting only when x − 1 is absent visits each number at most once inside the inner walk. The outer loop still looks at each number once. The total stays linear.

ONLY A NUMBER WITH NO PREDECESSOR STARTS A RUN.

```
function longestConsecutive(nums) {
  const set = new Set(nums);
  let best = 0;
  for (const x of set) {
    if (set.has(x - 1)) continue;
    let y = x;
    while (set.has(y)) y += 1;
    if (y - x > best) best = y - x;
  }
  return best;
}
```

The set is built before the loop, so membership tests do not change it. The `continue` is the start rule, and it happens before `y` moves. The while loop steps `y` past the end of the run. The length is `y - x`, not `y - x + 1`, because `y` has already moved one past the last present value. An empty set never enters the loop, and best stays 0. The off-by-one people write is `y - x + 1` after that loop, which reports one too many.

```
[100, 4, 200, 1, 3, 2]
100 starts a run of length 1 (99 is absent)
4 does not start (3 is present)
200 starts a run of length 1
1 starts: 1, 2, 3, 4 → length 4
3 and 2 do not start
best = 4
```

Majority in one vote

Problem 169 promises that one value occurs more than n/2 times. You can count with a map. Boyer–Moore does it with two variables, a candidate and a vote count. Take `[2, 2, 1, 1, 1, 2, 2]`. Votes start at 0, so the first 2 becomes the candidate and the vote becomes 1. The next 2 raises the vote to 2. The next value is 1, and the vote falls to 1. The next 1 brings the vote to 0. The following 1 finds a vote of 0, so it becomes the candidate, and the vote becomes 1. The next value is 2, which cancels that vote back to 0. The last 2 finds a vote of 0, becomes the candidate, and leaves the vote at 1. The candidate is 2, which is the majority.

Pairs of "this candidate, some other value" cancel. The promised majority has more than half the seats, so it cannot be cancelled away. You do not need a second pass while the promise holds. If a follow-up removes the promise, you count the candidate afterward and check that count yourself.

169 PROMISES A MAJORITY, SO THE CANDIDATE IS THE ANSWER. NO SECOND PASS.

```
function majorityElement(nums) {
  let candidate = 0;
  let votes = 0;
  for (const x of nums) {
    if (votes === 0) candidate = x;
    votes += x === candidate ? 1 : -1;
  }
  return candidate;
}
```

When `votes` is 0, you set the candidate to `x` before the vote changes. The same `x` then adds 1, because it now matches the candidate. If you added the vote first, a 0 vote would subtract 1 from a stale candidate and you would skip the assignment. The promise is why the return needs no count. Problem 169 does not ask for the count.

```
[2, 2, 1, 1, 1, 2, 2]
2 votes 1
2 votes 2
1 votes 1
1 votes 0
1 becomes candidate, votes 1
2 votes 0
2 becomes candidate, votes 1
answer 2
```

Every value that beats n/3

Problem 229 asks for every value that occurs more than n/3 times. There are at most two such values. The problem does not promise that a survivor is actually that frequent, so a second pass counts the survivors. Take `[1, 1, 1, 3, 3, 2, 2, 2]`. n is 8, and n/3 is about 2.67, so a value needs a count of at least 3. The first 1 takes candidate a, because a's count is 0. The next two 1s raise a's count to 3. The first 3 takes candidate b. The next 3 raises b's count to 2. The first 2 matches neither candidate, and both counts are nonzero, so both counts fall: a goes to 2 and b goes to 1. The next 2 cancels again: a goes to 1 and b goes to 0. The last 2 finds b's count at 0, so b becomes 2 and b's count becomes 1.

The second pass counts the real copies. a is 1 and occurs three times. b is 2 and occurs three times. Both beat n/3, so both are returned. The cancelled 3 does not survive as a candidate.

The second pass is the part you cannot skip. Take `[1, 2, 3]`. n is 3, so a count must be greater than 1. The 1 takes a, the 2 takes b, and the 3 cancels both counts to 0. The variables a and b still hold 1 and 2. A second pass finds one copy of each. One is not greater than 1, so the answer is empty. If you returned the survivors without counting, you would return values that are not frequent enough.

TWO CANDIDATES. THE SECOND PASS IS REQUIRED.

```
function majorityElement(nums) {
  let a = 0;
  let b = 0;
  let ca = 0;
  let cb = 0;
  for (const x of nums) {
    if (x === a) ca += 1;
    else if (x === b) cb += 1;
    else if (ca === 0) { a = x; ca = 1; }
    else if (cb === 0) { b = x; cb = 1; }
    else { ca -= 1; cb -= 1; }
  }
  ca = 0;
  cb = 0;
  for (const x of nums) {
    if (x === a) ca += 1;
    else if (x === b) cb += 1;
  }
  const out = [];
  if (ca > nums.length / 3) out.push(a);
  if (cb > nums.length / 3) out.push(b);
  return out;
}
```

The first loop tests equality with a, then equality with b, then an empty count, and only then a cancel. That order matters. If you tested an empty count before equality, a live candidate could be overwritten by a value you should have counted. After the first loop you set both counts back to 0 before the second walk. The second walk is a real count, not another vote. You push a value only when its real count is greater than n/3. The division is numeric, so you compare with `nums.length / 3` and you do not round the threshold down by hand.

Top K frequent values can sit in buckets. The bucket index is the frequency, and the bucket holds the values with that frequency. Walk the buckets from the high end until you have k values. A heap also works. The bucket version is linear after the frequency map. 4Sum II splits four arrays into two pairs. Map every sum of a pair from the first two arrays, then ask the map for the negation of each pair from the last two. You add the stored count, because many pairs can share a sum.

Time and extra memory

A pass of expected constant-time lookups is expected O(n) time and O(n) extra memory for the map or the set. The promised majority vote is O(n) time and O(1) extra memory. Majority II adds a second count, still O(n) time and O(1) extra memory.

Mistakes

- Inserting before the lookup on `[3]` with target 6 returns a pair that uses 3 twice. The map already holds 3 when you ask for 3, so you return `[0, 0]`. Lookup first. The map is empty on that single element.
- Writing `if (seen[need])` on an object fails when the stored index is 0. On `[8, 4, 5, 1, 7, 2]` with target 10, the hit is index 0. The object check is false, and you miss `[0, 5]`. Use `Map.has`.
- Starting a consecutive run at every number still finds 4 on `[100, 4, 200, 1, 3, 2]`, and the time becomes quadratic on a long run such as `[1, 2, 3, ..., n]`. Start only when x − 1 is absent.
- Skipping the second pass on majority II fails on `[1, 2, 3]`. The cancelled candidates still sit in the variables, and returning them reports values whose counts are not above n/3. The second pass returns an empty list.
- Using the vote variable as a count for problem 169 is a different question. 169 does not ask for the count. If a follow-up removes the promise, count the candidate in a second pass before you trust it.

Practice

Stay before chapter 5. Then before you leave the chapter. Majority II is a second pass.

1. Two Sum EASY STAY
Lookup the complement, then insert. Exactly one answer exists. You may not use the same element twice.

217. Contains Duplicate EASY STAY
A set of values seen so far. A repeat returns true.

242. Valid Anagram EASY STAY
One frequency map built from s and consumed by t, or two maps compared. Lengths must match.

128. Longest Consecutive Sequence MEDIUM STAY
Start a run only when x − 1 is absent. Duplicates disappear inside the set.

169. Majority Element EASY STAY
The vote. The promise means you return the candidate.

219. Contains Duplicate II EASY THEN
Store the last index. The test is i − j <= k.

349. Intersection of Two Arrays EASY THEN
A set of the first array. Emit a value from the second only once.

350. Intersection of Two Arrays II EASY THEN
Frequencies. Each accepted copy consumes one count.

1207. Unique Number of Occurrences EASY THEN
Map value to count, then a set of those counts. The set must be the same size as the map.

2215. Find the Difference of Two Arrays EASY THEN
Two sets. A value belongs to the first answer list when it is missing from the second set.

49. Group Anagrams MEDIUM THEN
Key by the sorted characters. The original strings stay intact inside the lists.

347. Top K Frequent Elements MEDIUM THEN
Frequency map, then buckets by count, then walk from the highest count.

454. 4Sum II MEDIUM THEN
Map sums of A[i] + B[j]. For each C[k] + D[l], add the count of the negation.

229. Majority Element II MEDIUM SECOND PASS
At most two candidates, then a real count. Keep a value only when count > n/3.

REWRITE FROM MEMORY
Rewrite two sum and the consecutive-run start rule from a blank file, with the lookup before the insert.

Carry forward. Chapter 5 uses two indexes on a sorted array, so the complement search no longer needs a map.

CHAPTER 05 · BUILDS ON CHAPTERS 3 AND 4

Two pointers from the ends

On a sorted array, or on a palindrome, each step can throw away one end. A pointer here is an index you store in a variable.

What this pattern is

A pointer, the first time this chapter uses the word, is an index stored in an integer. It is not a memory address. Two pointers from the ends means two indexes. `l` starts at 0, the first index. `r` starts at the last index, n − 1. You are on a sorted array, or you are comparing the two ends of a string. Each step throws away one end forever.

The unchecked pairs are the ones that still sit between `l` and `r`. An invariant is a sentence that stays true every time you are about to move. Here it is: every pair you have not checked still has both ends inside that range, and the move you are about to make throws away only pairs that cannot be the answer. Picture a sorted row of cards on a table. Your left hand holds the smallest remaining card. Your right hand holds the largest. If the sum is too small, the left card can never work with anything still on the table, so the left hand steps in.

What you already know

Chapter 3 could test every pair with two loops. Chapter 4 could find a complement with a map. A complement is the other value that finishes the pair. If the array is sorted, the complement has a direction, and you can drop the map. This chapter adds the two ends and the rule for which end moves.

A worked example

Take the sorted array `[1, 2, 3, 5, 8, 11]` and target 10. There are six values. `l` starts at index 0, value 1. `r` starts at index 5, value 11. The sum is 12, which is too large. The right value is too big even when it is paired with the smallest remaining partner, so you move `r` to 4. Now the right value is 8. The sum is 1 + 8 = 9, which is too small. The left value is too small even when it is paired with the largest remaining partner, so you move `l` to 1. The left value is 2 and the right value is still 8. The sum is 10. You stop.

The problem wants 1-based indexes, so you return `[l + 1, r + 1]`, which is `[2, 5]`. Returning `[1, 4]` would be the 0-based pair, and it is the wrong answer for this problem. You never let `l` and `r` land on the same index. One index is not a pair. If the sum had stayed wrong until the ends met, you would return an empty array. This problem promises exactly one solution, so the equal-sum line does hit.

```
[1, 2, 3, 5, 8, 11], target 10
l=0 r=5 sum 12, too big, r = 4
l=0 r=4 sum 9, too small, l = 1
l=1 r=4 sum 10, return [2, 5]
```

How you recognise it

- The array is sorted, or you are allowed to sort it and you do not need the original indexes afterward.
- You compare the two ends: a sum against a target, a container's walls, the larger square, or a pair of characters.
- Each step can throw away one end forever, because every pair that used that end is impossible or useless.
- A sum that is too small can only be repaired by moving the left index right, on a non-decreasing array.
- A sum that is too large can only be repaired by moving the right index left.
- If both indexes are only allowed to move right, you are in a later chapter.

Use chapter 4 when the values are not sorted and you still need the complement in a map.

Use chapter 6 when both indexes move right and one of them writes into the array.

Use chapter 7 when both indexes move right to bound one contiguous band, and you are not throwing an end away forever.

What to say out loud

Every unchecked pair is still between l and r. If the sum is too small I move l right, because this left value cannot reach the target with any remaining partner. If the sum is too large I move r left. I stop when l and r meet, because one index is not a pair.

The template

TOO SMALL MOVES L. TOO LARGE MOVES R. THIS PROBLEM'S ANSWER IS 1-BASED.

```
function twoSum(numbers, target) {
  let l = 0;
  let r = numbers.length - 1;
  while (l < r) {
    const sum = numbers[l] + numbers[r];
    if (sum === target) return [l + 1, r + 1];
    if (sum < target) l += 1;
    else r -= 1;
  }
  return [];
}
```

The invariant is the live range: any pair that could still hit the target uses an index from `l` through `r`. `l` starts at 0 and `r` starts at the last index. The loop condition is `l < r`. When they are equal you would read one cell twice. You compute the sum before you move. Equality returns `[l + 1, r + 1]`. The added 1 is this problem's 1-based answer, and it is the off-by-one people drop. A sum below the target increments `l` only. Any other sum decrements `r` only. You do not move both ends on a miss. Moving both would skip the pair that uses the new left value with the old right value.

The container

Take heights `[1, 8, 6, 2, 5, 4, 8, 3, 7]`. There are nine lines, indexed 0 through 8. Width is the distance between the indexes. Height is the shorter wall. `l` starts at 0, height 1. `r` starts at 8, height 7. The area is 1 × 8 = 8. The left wall is shorter, so you move `l` to 1. Now the walls are 8 and 7. The width is 7. The area is 7 × 7 = 49, and best becomes 49. The right wall is shorter, so you move `r` to 7. The new right height is 3. The area is 3 × 6 = 18, which does not beat 49. The right wall is still shorter, so `r` moves to 6. The walls are 8 and 8. The area is 8 × 5 = 40. The walls are equal, so the code moves `r`, because the left wall is not strictly shorter. You keep going. Later areas are 16, 15, 4, and 6. None of them beats 49. The answer is 49.

Width only shrinks as the ends move in. Suppose the left wall is the shorter one. You have just measured this pair. Every other pairing that still uses this same left wall uses some index strictly between the two ends. That inner index is closer, so the width is smaller. The height of that inner pairing is the minimum of the left wall and the inner height. A minimum is at most the left wall, so the height is at most this left wall. Every inner pairing of this left wall has smaller width and height at most that wall. None of those inner pairs can beat the area you just measured. You move the left index, and you never use that short wall again.

If the right wall is shorter, the same argument says every inner pairing of that right wall is worse, and you move `r` left. If the walls are equal, that shared height still cannot get better while the width shrinks, so moving either end is safe. The code moves the right end in the equal case, because it tests whether the left wall is strictly shorter. If you moved the taller wall at the start of this array, you would leave the short left wall in place and skip the pair of index 1 and index 8. That pair holds area 49, and you would not get it back.

MOVE THE SHORTER WALL ONLY.

```
function maxArea(height) {
  let l = 0;
  let r = height.length - 1;
  let best = 0;
  while (l < r) {
    const h = Math.min(height[l], height[r]);
    const area = h * (r - l);
    if (area > best) best = area;
    if (height[l] < height[r]) l += 1;
    else r -= 1;
  }
  return best;
}
```

You measure the area before you move. `h` is the shorter wall, and the width is `r - l`, not `r - l + 1`. The slots between the walls are the water, and the distance between indexes is the width. After the measurement, you move only the shorter side. The `else` covers a shorter right wall and an equal pair, and in both of those cases `r` decreases. You do not move both ends in one step. The loop stops when `l` and `r` meet, because a container needs two distinct indexes.

```
[1, 8, 6, 2, 5, 4, 8, 3, 7]
l=0 (1), r=8 (7), area 8, left is shorter, l = 1
l=1 (8), r=8 (7), area 49, right is shorter, r = 7
keep going. 49 is the best.
moving the taller wall first would skip this pair and never recover it.
```

Squares, written from the back

A sorted array of negatives and positives has its largest squares at the ends, because squaring turns a large negative into a large positive. Take `[-4, -1, 0, 3, 10]`. There are five values. The output has five slots, and the write index starts at 4, the last slot. `l` is 0 and holds −4. `r` is 4 and holds 10. The absolute values are 4 and 10. 10 is larger, so you write 100 at index 4 and move `r` to 3. Now the ends are −4 and 3. Absolute values 4 and 3. 4 is larger, so you write 16 at index 3 and move `l` to 1. The ends are −1 and 3. 3 is larger, so you write 9 at index 2 and move `r` to 2. The ends are −1 and 0. 1 is larger, so you write 1 at index 1 and move `l` to 2. Both ends now sit on 0. You write 0 at index 0. The output is `[0, 1, 9, 16, 100]`.

THE NEXT OUTPUT CELL IS THE LARGER OF THE TWO END SQUARES.

```
function sortedSquares(nums) {
  const n = nums.length;
  const out = new Array(n);
  let l = 0;
  let r = n - 1;
  for (let i = n - 1; i >= 0; i--) {
    if (Math.abs(nums[l]) > Math.abs(nums[r])) {
      out[i] = nums[l] * nums[l];
      l += 1;
    } else {
      out[i] = nums[r] * nums[r];
      r -= 1;
    }
  }
  return out;
}
```

You compare the absolute values before you write. The larger end is squared into `out[i]`, and only that end moves. `i` walks from n − 1 down to 0, so the largest square lands at the end and the smallest square lands at index 0. The off-by-one is the write direction. If you wrote from the front, you would not yet know the smallest square,

and the first cell would get a value that belongs at the end. When the absolute values tie, the `else` takes the right end. Either end is a correct square in a tie. An empty input builds an empty output and the loop does not run.

```
[−4, −1, 0, 3, 10]
write 100 from the right, then 16 from the left, then 9, then 1, then 0
[0, 1, 9, 16, 100]
```

A palindrome is the same two indexes

Take the string `"a,c@a"`. The indexes are 0 `a`, 1 `,`, 2 `c`, 3 `@`, 4 `a`. `l` starts at 0 and `r` starts at 4. You lowercase both ends. Both are the letter a, so they match, and both ends step inward. Now `l` is 1 and `r` is 3. The left character is a comma. It is not a letter or a digit, so you move only `l`, to 2, and you do not touch `r` on that turn. Now the left character is `c` and the right character is `@`. `@` is not a letter or a digit, so you move only `r`, to 2. `l` and `r` now sit on the same index, and the loop stops. One middle character does not need a partner. The string is a palindrome after the skips.

A mismatch returns immediately. On `"ab@ca"` the outer `a` characters match, then `b` and `c` are both real characters and they differ, so you return false before any further skip. You do not repair a mismatch by moving only one end in this problem.

SKIP NON-LETTERS WITH A NAMED HELPER.

```
function isAlnum(ch) {
  return (ch >= "a" && ch <= "z") || (ch >= "0" && ch <= "9");
}

function isPalindrome(s) {
  let l = 0;
  let r = s.length − 1;
  while (l < r) {
    const L = s[l].toLowerCase();
    const R = s[r].toLowerCase();
    if (!isAlnum(L)) { l += 1; continue; }
    if (!isAlnum(R)) { r −= 1; continue; }
    if (L !== R) return false;
    l += 1;
    r −= 1;
  }
  return true;
}
```

You lowercase both characters before any test, so `A` and `a` can match. The helper answers "is this a letter or a digit?" If the left character fails, you increment `l` and `continue`, which skips the right-hand test on that turn. If the right character fails, you decrement `r` and continue. Only two real characters reach the inequality test. A difference returns false at once. A match moves both ends. The loop condition `l < r` is the off-by-one. Letting them cross, or comparing when they are equal, treats one index as a pair. An empty string never enters the loop and returns true.

Backspace, still from the ends

You can compare two strings with backspaces without building the edited text. A pointer here is still an index. You walk backward. Take `s = "ab#c"` and `t = "ad#c"`. Both end at index 3, the character `c`. Neither end is a `#`, so both real characters are `c`, they match, and both indexes step to 2. From index 2, `s` sees `#`. That raises a skip count to 1 and moves to index 1. Index 1 is `b`, and a skip is owed, so the skip is spent and the index moves to 0. Index 0 is `a` with no skip owed, so `a` is the next real character. The same walk on `t` spends the `#` on `d` and also lands on `a`. The characters match. Both indexes then step to −1. The next search from −1 finds nothing on either string, so both sides are exhausted together and the strings match.

EACH CALL RETURNS THE NEXT REAL CHARACTER AT OR BEFORE I.

```
function nextIndex(s, i) {
  let skip = 0;
  while (i >= 0) {
    if (s[i] === "#") {
      skip += 1;
      i -= 1;
    } else if (skip > 0) {
      skip -= 1;
      i -= 1;
    } else {
      return i;
    }
  }
  return -1;
}

function backspaceCompare(s, t) {
  let i = s.length - 1;
  let j = t.length - 1;
  while (i >= 0 || j >= 0) {
    i = nextIndex(s, i);
    j = nextIndex(t, j);
    if (i < 0 && j < 0) return true;
    if (i < 0 || j < 0 || s[i] !== t[j]) return false;
    i -= 1;
    j -= 1;
  }
  return true;
}
```

`nextIndex` starts at the index you hand it and walks left. A `#` adds 1 to `skip` and moves left. A real character with `skip > 0` spends one skip and moves left. It is not returned. A real character with skip 0 is returned, and it is not consumed inside the helper. The caller compares those two indexes, and only after a match does the caller step one past each of them. If you stepped inside the helper before returning, you would skip the character you meant to compare. If one side returns −1 and the other still has a character, the strings differ. `"a#c"` against `"c"` matches. `"ab#c"` against `"c"` does not, because the `a` is still there after the `#` deletes `b`.

Time and extra memory

Each end moves inward at most n times, so these templates are O(n) time and O(1) extra memory after the array is sorted. Squares allocates the output array of length n, which the problem requires.

Mistakes

- Moving both ends because the area felt small fails on `[1, 8, 6, 2, 5, 4, 8, 3, 7]`. The first area is only 8, and the left wall is the short one. If you move the taller right wall, you skip the later pair that holds area 49, and that pair does not come back.
- Returning the 0-based pair for two sum II fails on `[1, 2, 3, 5, 8, 11]` with target 10. The indexes are 1 and 4. The required answer is `[2, 5]`. The code has to add 1 to each index.
- Writing squares from the front fails on `[-4, -1, 0, 3, 10]`. The largest square is 100, at an end, so the first output cell is not known until the small middle squares are left. The write index has to start at the last slot.
- Letting `l` and `r` cross, or using `l <= r` as a pair loop, treats one index as a pair. On a single-character string the two indexes start equal. There is no pair to compare, and the palindrome is true.
- Replacing the palindrome filter with a chain of replaces you cannot read out loud hides which character moved. The named helper is the version you can walk on `"a,c@a"`.

Practice

Stay before chapter 6. Then before you leave the chapter.

167. Two Sum II - Input Array Is Sorted MEDIUM STAY
Sorted, exactly one solution, answer is 1-based. Too small moves l.

11. Container With Most Water MEDIUM STAY
Move the shorter side only. Width is r − l, height is the min of the two walls.

977. Squares of a Sorted Array EASY STAY
Write from index n − 1. Compare absolute values at the ends.

125. Valid Palindrome EASY STAY
Skip non-alphanumeric characters. Compare lowercase letters and digits.

344. Reverse String EASY THEN
Swap s[l] and s[r], then move both ends in. The input is a character array.

345. Reverse Vowels of a String EASY THEN
Same swap, but an end moves alone until it sits on a vowel.

680. Valid Palindrome II EASY THEN
On the first mismatch, test the slice that skips l, and the slice that skips r.

844. Backspace String Compare EASY THEN
The skip count from the right. A stack of kept characters is the version you can write if you blank on the skip.

REWRITE FROM MEMORY
Rewrite two sum II and the container loop from a blank file, and say why a shorter left wall cannot win with any index inside it.

Carry forward. Chapter 6 keeps two indexes, and both of them move to the right. One reads. One writes.

CHAPTER 06 · BUILDS ON CHAPTER 5

Same-direction two pointers

One index reads every cell. The other index writes, and it moves only when the cell is kept.

What this pattern is

A pointer is an index stored in an integer. It is not a memory address. This chapter uses two pointers that move in the same direction, toward the right. `r` reads every cell. `w` writes, and `w` advances only when the cell you just read is a cell you are keeping. In place means you overwrite the input with a handful of extra variables, not a second array of length n. The kept prefix is `nums[0..w)`. A prefix is the block from the start up to, but not including, index `w`. After the loop, `w` is the length the caller should read.

An invariant is a sentence that stays true when you finish a read. Here it is: `r` has consumed every index before the current read, and `nums[0..w)` is exactly the kept version of that consumed prefix. Picture a clerk copying a form onto the same page. The reading finger visits every line. The writing finger moves only when the line is one you keep. The gap between the fingers is the lines you have decided to drop.

What you already know

Chapter 5 moved two indexes toward each other and threw one end away. Both indexes in this chapter move right, and one of them writes. You already know an in-place answer returns a length and leaves the tail as garbage. This chapter adds the read index, the write index, and the rule for when the write index steps.

A worked example

Take `[3, 2, 2, 3, 4, 3]` and remove every 3. There are six values. `w` starts at 0, so the kept prefix is empty. `r` starts at 0. Index 0 holds 3, which is the value you are removing, so you do not write and `w` stays 0. Index 1 holds 2. You write 2 at index 0, then `w` becomes 1. The kept prefix is `[2]`. Index 2 holds 2. You write 2 at index 1, then `w` becomes 2. The prefix is `[2, 2]`. Index 3 holds 3, so you skip it. Index 4 holds 4. You write 4 at index 2, then `w` becomes 3. The prefix is `[2, 2, 4]`. Index 5 holds 3, so you skip it. You return 3. The caller reads the first 3 slots. The tail may still show old numbers, and the caller ignores it.

The write happens before `w` grows. If you grew `w` first, you would skip a slot and the kept values would shift right. If you grew `w` on a 3, the removed value would sit inside the prefix. On this input that bug returns a length that still counts a 3.

```
[3, 2, 2, 3, 4, 3], remove 3
r=0 skip
r=1 write 2 at 0, w = 1
r=2 write 2 at 1, w = 2
r=3 skip
r=4 write 4 at 2, w = 3
r=5 skip
return 3, prefix [2, 2, 4]
```

How you recognise it

- The array is sorted and you overwrite duplicates in place, returning the new length.
- You filter values out, or you slide zeros to the end, and the relative order of the kept values has to stay.
- You merge two sorted arrays into one buffer that already has empty room at the end.
- The read index visits every cell, and the write index moves only for a cell that stays.
- You check whether one sequence appears inside another in order. That is one index per sequence, and both move forward.

- If the two indexes move toward each other, or if you must not overwrite the array, this is a different chapter.

Use chapter 5 when the two indexes move toward each other on a sorted array or a palindrome.

Use chapter 7 when both indexes move right to mark a window, and the array itself stays unchanged.

Use chapter 13 when each value should be moved onto the index that matches the value.

What to say out loud

r reads every cell from left to right. w writes, and w moves only when I keep the cell I just read. When I finish a read, nums from 0 up to but not including w is the kept copy of what r has consumed. The length I return is w, not the last kept index.

The template

W ADVANCES ONLY WHEN THE VALUE STAYS.

```
function removeElement(nums, val) {
  let w = 0;
  for (let r = 0; r < nums.length; r++) {
    if (nums[r] !== val) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}
```

The invariant is the kept prefix `nums[0..w)` for every index `r` has already passed. `w` and `r` both start at 0. The loop runs `r` through every index, including the last one, and stops before `nums.length`. Inside the true branch you assign `nums[w] = nums[r]` first, then you add 1 to `w`. That order fills the current hole and then moves the hole forward. Adding first would leave a stale slot at the old `w` and write past the kept prefix. You do not add to `w` when the value equals `val`. The returned number is `w`, which is a length. On an empty array the loop never runs and the length is 0. On an array with no copies of `val`, `w` ends equal to n, and every write lands on the same index it read.

Move zeroes is this write cursor on the non-zeros, followed by a fill. Take `[0, 1, 0, 3, 12]`. Skip the 0 at index 0. Write 1 at index 0 and advance. Skip the next 0. Write 3, then write 12. The write index is now 3, and the array looks like `[1, 3, 12, 3, 12]` before the fill. You then set every slot from 3 to the end to 0. The result is `[1, 3, 12, 0, 0]`. The non-zeros kept their order. A blind swap that pulls a later zero backward can reorder those non-zeros. Write the keepers forward, then set the tail.

Duplicates in a sorted array

Sorted duplicates sit next to each other. Take `[1, 1, 2, 2, 3]` and keep one copy of each value. The array is not empty, so index 0 is kept without a comparison, and `w` starts at 1. `r` starts at 1. Index 1 holds 1, and the last kept value is `nums[0]`, which is also 1, so you skip. Index 2 holds 2, which differs from 1. You write 2 at index 1, then `w` becomes 2. The array begins `[1, 2, ...]`. Index 3 holds 2, and the last kept value is now `nums[1]`, which is 2, so you skip. Index 4 holds 3, which differs from 2. You write 3 at index 2, then `w` becomes 3. You return 3. The caller reads `[1, 2, 3]`.

A one-element array never enters the loop. `w` stays 1, and that length is correct. An empty array returns 0 before `w` is set, so you never read index −1.

Allowing two copies uses a different comparison. Take `[1, 1, 1, 2, 2, 3]`. `w` starts at 0, not at 1. The test `w < 2` lets the first two writes happen with no look backward. Index 0 writes 1, and `w` becomes 1. Index 1 writes 1, and `w` becomes 2. Index 2 holds 1, and `nums[w − 2]` is `nums[0]`, which is 1, so this third 1 is skipped. Index 3

holds 2, and `nums[0]` is 1, so you write 2 at index 2 and `w` becomes 3. Index 4 holds 2. `nums[w - 2]` is now `nums[1]`, which is 1, not 2, so you write another 2 and `w` becomes 4. Index 5 holds 3, which differs from `nums[2]`, so you write 3 and `w` becomes 5. The kept prefix is `[1, 1, 2, 2, 3]`.

ONE COPY COMPARES WITH W − 1. TWO COPIES COMPARE WITH W − 2.

```
function removeDuplicates(nums) {
  if (nums.length === 0) return 0;
  let w = 1;
  for (let r = 1; r < nums.length; r++) {
    if (nums[r] !== nums[w - 1]) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}

function removeDuplicatesII(nums) {
  let w = 0;
  for (let r = 0; r < nums.length; r++) {
    if (w < 2 || nums[r] !== nums[w - 2]) {
      nums[w] = nums[r];
      w += 1;
    }
  }
  return w;
}
```

For one copy, the empty check comes first. Then `w` starts at 1 so the comparison `nums[w - 1]` is index 0, never −1. You compare, then write at `w`, then increment. Comparing with `nums[r - 1]` also works for one copy on a sorted array, because duplicates are adjacent and you are keeping the first of each run. For two copies that neighbor test is the wrong slot. On `[1, 1, 1]` a test against `nums[r - 1]` sees equal neighbors and keeps only the first 1. The allowed prefix is `[1, 1]`, length 2. You look at the kept prefix, at `w - 2`, and the `w < 2` guard lets the first two keepers through before that index exists. The third 1 equals `nums[0]` and is dropped.

```
[1, 1, 1, 2, 2, 3], at most two copies
r=0 write, w=1
r=1 write, w=2
r=2 value 1 equals nums[0], skip
r=3 value 2, write, w=3
r=4 value 2, nums[w-2] is 1, write, w=4
r=5 value 3, write, w=5
prefix [1, 1, 2, 2, 3]
```

Merge from the end

Take `nums1 = [1, 2, 3, 0, 0, 0]` with m = 3, and `nums2 = [2, 5, 6]` with n = 3. The empty slots are the tail. The write index starts at m + n − 1, which is 5. `i` starts at 2, on the 3 in `nums1`. `j` starts at 2, on the 6 in `nums2`. 3 is not greater than 6, so you write 6 at index 5 and move `j` to 1. Next, 3 is not greater than 5, so you write 5 at index 4 and move `j` to 0. Next, 3 is greater than 2, so you write 3 at index 3 and move `i` to 1. Next, `nums1[1]` is 2 and `nums2[0]` is 2. 2 is not greater than 2, so the `else` writes the 2 from `nums2` at index 2 and `j` becomes −1. The loop stops because `nums2` is exhausted. Index 0 still holds 1 and index 1 still holds 2. They were already in the right place. The merged array is `[1, 2, 2, 3, 5, 6]`.

THE EMPTY TAIL IS THE SPACE YOU WRITE INTO.

```
function merge(nums1, m, nums2, n) {
  let i = m - 1;
  let j = n - 1;
```

```
  let w = m + n - 1;
  while (j >= 0) {
    if (i >= 0 && nums1[i] > nums2[j]) {
      nums1[w] = nums1[i];
      i -= 1;
    } else {
      nums1[w] = nums2[j];
      j -= 1;
    }
    w -= 1;
  }
}
```

You choose the value before you move `w`. The test reads `nums1[i]` only when `i` is still at least 0. If `i` has already fallen off, every remaining write copies `nums2`. When the two tails are equal, the test fails and you take `nums2`. That choice is safe, because the equal value in `nums1` is still sitting in the unprocessed prefix. The loop condition is `j >= 0`, not `w >= 0`. When `nums2` runs out, the leftover `nums1` prefix is already home, so you stop. Writing from index 0 would overwrite a `nums1` value you have not merged. On this input the first front write would destroy the 1.

```
nums1 = [1, 2, 3, 0, 0, 0], m = 3
nums2 = [2, 5, 6], n = 3
write 6, then 5, then 3, then 2 from nums2
j is exhausted, so the leftover [1, 2] stays where it is
[1, 2, 2, 3, 5, 6]
```

Sort by parity uses the same write slot as a swap destination. Each even number is swapped into the next write index, so the evens occupy a prefix and the odds fall behind. Sort by parity II keeps two cursors, one on even indexes and one on odd indexes. An even index wants an even value. When both cursors are sitting on the wrong kind of value, you swap them. On `[4, 2, 5, 7]` index 1 holds 2 and index 2 holds 5. Both are wrong. One swap produces `[4, 5, 2, 7]`.

Is subsequence is the forward pair on two strings, and it does not overwrite anything. `i` walks the pattern. `j` walks the text. On a match you advance `i`. You always advance `j`. The pattern is a subsequence when `i` reaches the pattern length. A subsequence keeps order and allows gaps. Take pattern `"abc"` and text `"ahbgdc"`. Index 0 of the text matches `a`, so the pattern index moves to 1. `h` does not match `b`. `b` does, so the pattern index moves to 2. `g` and `d` do not match `c`. `c` does, and the pattern index reaches 3. The answer is true. If the text ended before `i` reached the end, the answer would be false.

Time and extra memory

Remove and the duplicate walks are O(n) time, and the merge is O(m + n) time. Extra memory is O(1) besides the buffer the problem already gave you.

Mistakes

- Advancing `w` on a value you meant to delete fails on `[3, 2, 2, 3]` when you remove 3. The first 3 is copied into the kept prefix, and the returned length counts a value that should have been dropped.
- Comparing two-copy duplicates with `nums[r - 1]` only fails on `[1, 1, 1]`. Equal neighbors make you keep a single 1. The allowed prefix is two 1s. The comparison has to look at `nums[w - 2]` in the kept prefix, after the first two writes have been allowed through.
- Merging from index 0 fails on `nums1 = [1, 2, 3, 0, 0, 0]` and `nums2 = [2, 5, 6]`. The first write lands on the 1, and that 1 is gone before it can be placed. The empty slots are at the end, so the write cursor starts at the end.

- Filling moved zeros by swapping blindly can pull a zero between non-zeros and change their order. On `[0, 1, 0, 3, 12]` the keepers must stay `[1, 3, 12]` before the tail becomes zeros.

Practice

Stay before chapter 7. Then before you leave the chapter.

27. Remove Element EASY STAY
Write every value that is not val. Return the new length.

26. Remove Duplicates from Sorted Array EASY STAY
Sorted. Keep a value when it differs from the last kept value.

283. Move Zeroes EASY STAY
Pack the non-zeros, then fill zeros. Relative order stays.

88. Merge Sorted Array EASY STAY
Write from m + n − 1. Stop when nums2 is exhausted.

80. Remove Duplicates from Sorted Array II MEDIUM THEN
Keep the value when w < 2 or it differs from nums[w − 2].

905. Sort Array By Parity EASY THEN
Swap evens into the next write slot. Odds fall behind.

922. Sort Array By Parity II EASY THEN
Even indices want evens. Walk an even cursor and an odd cursor, and swap when both are wrong.

392. Is Subsequence EASY THEN
Advance the pattern index only on a match. The text index always advances.

REWRITE FROM MEMORY
Rewrite remove-duplicates and the merge-from-the-end loop from a blank file, including why the write cursor starts at the last slot.

Carry forward. Chapter 7 uses two indexes that both move right, and now they bound a window instead of overwriting the array.

CHAPTER 07 · BUILDS ON CHAPTER 6

Sliding window

A window is the band from index l through index r. You grow it from the right, and you shrink it from the left with a while loop, when a bigger band only gets worse.

What this pattern is

A window is the band of cells from index `l` through index `r`. Both ends are included. A subarray is that kind of band: a stretch of cells with no gaps.

Picture a short ruler laid on a row of numbers. You step the right end forward one cell. When the cells under the ruler break the rule, you pull the left end forward until the rule holds again. You do not move either end backward.

This is worth doing when the test is monotone. Monotone means a bigger band only moves you toward illegal, and dropping cells from the left moves you back toward legal. Sums of values that are never negative work that way. Adding a value cannot make the sum smaller. Dropping the left value cannot make the sum larger.

You shrink with a `while`, not with a single `if`. One step of `l` can leave the window illegal, so you keep moving `l` until the rule holds.

What you already know

Chapter 6 already walks two indices that both move right. There `r` reads every cell, and the write index moves only when you keep a cell. A window keeps those two indices and gives them a new job: they are the two ends of one subarray. Chapter 5 started at the two ends of the whole array and moved inward. Here both ends begin on the left, and the right end leads.

A worked example

Use the array `[2, 3, 1, 2, 4, 3]` and the target 7. `l` starts at 0 and the sum starts at 0. `r` is the next index you include.

At `r = 0` you add 2. The sum is 2, which is below 7, so `l` stays at 0. Dropping the only cell would make the sum smaller, and you still need to reach 7.

At `r = 1` you add 3. The sum is 5. It is still below 7, so you leave `l` at 0 and let the band grow. At `r = 2` you add 1. The sum is 6. A shrink cannot help here, because every value is at least 0, so a shorter band has a smaller sum.

At `r = 3` you add 2. The window is indexes 0 through 3, the cells 2, 3, 1, 2, and the sum is 8. The length is `r - l + 1`, which is 4. The sum is at least 7, so you record 4. You then shrink, because a shorter legal window would be a better answer. You subtract `nums[0]`, the sum falls to 6, and `l` moves to 1. The sum is below 7, so the `while` stops.

At `r = 4` you add 4. The window is indexes 1 through 4, the sum is 10, and the length is 4. You subtract `nums[1]`, which is 3. The sum is 7 and `l` moves to 2. The sum is still at least 7, so the `while` does not stop. The length is `4 - 2 + 1`, which is 3, and you record 3. You subtract `nums[2]`, which is 1. The sum is 6 and `l` moves to 3. Now the `while` stops. One `if` would have stopped when the sum was still 7, and you would never have recorded 3.

At `r = 5` you add 3. The window is indexes 3 through 5, the cells 2, 4, 3, and the sum is 9. You subtract `nums[3]`, the sum is 7, and `l` moves to 4. The length is `5 - 4 + 1`, which is 2, so you record 2. You subtract `nums[4]`, which is 4. The sum is 3 and `l` moves to 5. The `while` stops. The best length is 2. That window is the subarray `[4, 3]`.

The same steps, packed into a trace, look like this.

```
[2, 3, 1, 2, 4, 3], target 7
r=3, window [2, 3, 1, 2], sum 8, length 4
  drop 2, sum 6, stop. best = 4
r=4, add 4, window [3, 1, 2, 4], sum 10, length 4
  drop 3, sum 7, length 3
  drop 1, sum 6, stop. best = 3
r=5, add 3, window [2, 4, 3], sum 9, length 3
  drop 2, sum 7, length 2
  drop 4, sum 3, stop. best = 2
answer 2
```

How you recognise it

- The cells you care about form a subarray, so there are no gaps in the band.
- The test uses a sum, a product, a count of zeros, or a count of distinct values.
- Making the band longer only pushes the test toward illegal, and shortening it from the left pushes the test back.
- The values that the test uses are non-negative, or they are counts you can undo when a cell leaves the left edge.
- You want a shortest band, a longest band, or a count of the bands that end at the `r` you just added.

Some prompts look close and belong in another chapter.

- Use chapter 8 when a value can be negative and you need every subarray whose sum equals k, because a negative can make a later sum smaller.
- Use chapter 5 when the array is sorted and each step can drop one end of the whole array.
- Use chapter 15 when you need the maximum inside the window after that maximum slides out the left edge. One variable does not remember the second maximum.

What to say out loud

An invariant is a fact that is still true after every move. Yours is this: `r` is the end you just included, and `l` is the leftmost start that keeps `[l, r]` valid.

You move `l` with a `while`, because one step may not restore the invariant. The length you record is `r - l + 1`, and you record it before you drop `nums[l]`.

The template

NON-NEGATIVE VALUES. RECORD THE LENGTH BEFORE YOU SHRINK.

```
function minSubArrayLen(target, nums) {
  let l = 0;
  let sum = 0;
  let best = Infinity;
  for (let r = 0; r < nums.length; r++) {
    sum += nums[r];
    while (sum >= target) {
      const len = r - l + 1;
      if (len < best) best = len;
      sum -= nums[l];
      l += 1;
    }
  }
  return best === Infinity ? 0 : best;
}
```

After the `while`, the sum of `nums[l..r]` is below the target, or `l` has moved past `r`. You add `nums[r]` before you test the loop. Inside the loop you read the length, then you subtract `nums[l]`, then you add 1 to `l`.

The length is `r - l + 1`. You must read it before the subtract. If you subtract first, you record a window that no longer meets the target. The test is a `while` because one step of `l` can leave the sum still at least the target. On the step where `r` is 4, the sum goes from 10 to 7, and 7 still qualifies, so a second shrink has to run at that same `r`.

Fixed length

A window of length `k` does not hunt for `l`. Use `[1, 12, -5, -6, 50, 3]` and `k = 4`. The first window is indexes 0 through 3. Its sum is `1 + 12 + (-5) + (-6)`, which is 2. You store 2 as the best sum.

Then `r` moves to 4. The new cell is 50. The cell that falls out is at index `r - k`, which is 0, and that cell is 1. The sum becomes `2 + 50 - 1`, which is 51. You store 51. Then `r` moves to 5. You add 3 and you subtract the cell at index 1, which is 12. The sum becomes 42. The best sum stays 51, and the average is `51 / 4`, which is 12.75.

SUBTRACT THE CELL THAT JUST LEFT THE WINDOW.

```
function findMaxAverage(nums, k) {
  let sum = 0;
  for (let i = 0; i < k; i++) sum += nums[i];
  let best = sum;
  for (let r = k; r < nums.length; r++) {
    sum += nums[r];
    sum -= nums[r - k];
    if (sum > best) best = sum;
  }
  return best / k;
}
```

```
[1, 12, -5, -6, 50, 3], k = 4
first window sum = 1 + 12 + (-5) + (-6) = 2
next: drop 1, add 50, sum = 51
next: drop 12, add 3, sum = 42
best sum 51, average 12.75
```

`l` is not a free index here. The window always holds `k` cells, so the cell that leaves is at `r - k`. You add `nums[r]` and then you subtract `nums[r - k]`. If you subtract first, you remove a cell that is still inside the old window. The opening loop fills the first `k` cells before any cell leaves, and `best` starts as that first sum, so an array whose length is `k` still has an answer.

One deletion is required

Max consecutive ones III counts zeros in the window and shrinks while that count is above `k`. A flip keeps the cell, so the answer is the ordinary length `r - l + 1`. On `[1, 1, 0, 1, 1, 0, 1]` with `k = 1`, the longest legal window is indexes 0 through 4, the cells `[1, 1, 0, 1, 1]`. It holds one zero. The length is 5, and the flipped zero stays in the answer.

Longest subarray of 1s after deleting one element uses the same shrink, with at most one zero, and then it changes the length. The stored length is `r - l`. That is `(r - l + 1) - 1`. One deletion is required, even when the window holds no zero. An array of `n` ones returns `n - 1`.

Walk `[1, 1, 0, 1, 1, 0, 1]` with that rule. From `r = 0` through `r = 4` the window starts at 0 and the zero count is 0, then 1. At `r = 4` the stored length is `4 - 0`, which is 4. That is the five cells minus the one deletion. At `r = 5` you add a second zero. The `while` runs three times: index 0 is a 1, index 1 is a 1, and index 2 is the zero that you drop. `l` lands on 3 and the zero count is 1 again. The stored length `5 - 3` is 2, which does not

beat 4. At `r = 6` the length is 3. The answer stays 4. You deleted the zero in `[1, 1, 0, 1, 1]` and kept four 1s.

On `[1, 1, 1]` the zero count stays 0, so `l` stays 0. The stored lengths are 0, then 1, then 2. The answer is 2, which is one less than the length of the array.

R − L IS (R − L + 1) − 1. ONE DELETION IS REQUIRED.

```
function longestSubarray(nums) {
  let l = 0;
  let zeros = 0;
  let best = 0;
  for (let r = 0; r < nums.length; r++) {
    if (nums[r] === 0) zeros += 1;
    while (zeros > 1) {
      if (nums[l] === 0) zeros -= 1;
      l += 1;
    }
    const len = r - l;
    if (len > best) best = len;
  }
  return best;
}
```

You add to the zero count when `nums[r]` is 0, and you do that before the `while`. The `while` runs while the zero count is greater than 1. It drops `nums[l]` before `l` moves. The length you store is `r - l`, because one deletion is required. If you store `r - l + 1` on `[1, 1, 1]`, you return 3, and you have deleted nothing.

A repeat outside the window

Walk the string `abba`. At `r = 0` the letter is `a`. The map is empty, `l` stays 0, and you store `a` at index 0. The length is 1. At `r = 1` the letter is `b`. It is new, `l` stays 0, and the length is 2.

At `r = 2` the letter is `b` again. The previous `b` is at index 1, and 1 is still at least `l`, so that copy is inside the window. You set `l` to `1 + 1`, which is 2. The window is the single `b` at index 2. The length is 1. You then store the new index of `b`.

At `r = 3` the letter is `a`. The previous `a` is at index 0. That index is below `l`, so the old `a` is already outside the window. You leave `l` at 2. The window is indexes 2 and 3, the letters `ba`, and the length is 2. If you moved `l` back to 1, the window would be `bba`, and that repeats `b`.

JUMP L ONLY WHEN THE PREVIOUS INDEX IS STILL INSIDE THE WINDOW.

```
function lengthOfLongestSubstring(s) {
  const last = new Map();
  let l = 0;
  let best = 0;
  for (let r = 0; r < s.length; r++) {
    const ch = s[r];
    if (last.has(ch) && last.get(ch) >= l) {
      l = last.get(ch) + 1;
    }
    last.set(ch, r);
    const len = r - l + 1;
    if (len > best) best = len;
  }
  return best;
}
```

```
"abba"
r=0 a, l=0, len 1
r=1 b, l=0, len 2
r=2 b, previous b is 1, which is >= l, so l = 2, len 1
r=3 a, previous a is 0, which is < l, leave l alone, len 2
best 2
if you moved l back to 1 you would claim "bba", which repeats b
```

You test the stored index against `l` before you overwrite it with `r`. The jump is `l = last + 1`, and only when that stored index is still at least `l`. If the stored index is below `l`, that copy is already outside, and moving `l` there would pull the left end backward. You write the map after the test, so the map holds the newest index. The length, after the jump, is `r - l + 1`.

Two kinds of fruit

Fruit into baskets is the same shape with a frequency map. You may hold two distinct values. Walk `[1, 2, 1, 2, 3]`. Through `r = 3` the map holds only 1 and 2, and the window length is 4.

At `r = 4` you add 3. The map has three keys, so you shrink. You drop index 0, a 1, and the count of 1 is still above 0. You drop index 1, a 2. You drop index 2, a 1, and the count of 1 hits 0, so you delete that key. `l` is now 3. The map holds 2 and 3. The length is 2. The best answer stays 4, the band `[1, 2, 1, 2]`.

You add `nums[r]` first. The `while` runs while the map has more than two keys. You decrease the count at `l`, and you delete the key when the count hits 0, before you move `l`. A key left at count 0 still looks like a third fruit.

Count every valid subarray

Once `[l, r]` is valid, and every subarray of a valid window is valid, each start from `l` through `r` makes a valid subarray that ends at `r`. There are `r - l + 1` of them. You add that number. You do not list them.

Walk `[10, 5, 2, 6, 3]` with product limit 100. The values are positive, so dividing by `nums[l]` undoes the multiply. At `r = 0` the product is 10. You add 1. At `r = 1` the product is 50. You add 2. The count is 3.

At `r = 2` the product is 100, which is not below 100. You divide by 10 and move `l` to 1. The product is 10, so the `while` stops. You add `2 - 1 + 1`, which is 2. The count is 5. At `r = 3` the product is 60. You add 3. The count is 8. At `r = 4` the product is 180. You divide by 5 and move `l` to 2. The product is 36. You add 3. The count is 11.

If `k` is 1 or less, every product of positive integers is at least 1, so the function returns 0 before the loop.

POSITIVE INTEGERS. DIVIDING BY NUMS[L] UNDOES THE MULTIPLY.

```
function numSubarrayProductLessThanK(nums, k) {
  if (k <= 1) return 0;
  let l = 0;
  let prod = 1;
  let count = 0;
  for (let r = 0; r < nums.length; r++) {
    prod *= nums[r];
    while (prod >= k) {
      prod /= nums[l];
      l += 1;
    }
    count += r - l + 1;
  }
  return count;
}
```

You multiply by `nums[r]` before the `while`. The `while` divides by `nums[l]` and then moves `l`, and it keeps going while the product is at least `k`. You add `r - l + 1` after the window is legal. If you add that count before

the shrink, you count subarrays whose product is too big.

Exactly K

Exactly `K` is two at-most counts. `atMost(K)` counts subarrays with sum at most `K`, or with at most `K` distinct values, or with at most `K` odd numbers. Exactly `K` is `atMost(K)` minus `atMost(K - 1)`. This subtraction is legal only when the values are non-negative. Treat an odd number as a 1 when you are counting odds. Binary subarrays with sum, and nice subarrays, are this subtraction.

Walk `[1, 0, 1, 0, 1]` with goal 2. `atMost(2)` never has to drop a cell until `r = 4`, when the sum becomes 3. You drop the leading 1, the sum is 2, and `l` is 1. That pass adds 14. `atMost(1)` is stricter. At `r = 4` the sum is 2, so the `while` drops the 0 at index 1, the sum is still 2, and then it drops the 1 at index 2. The sum is 1 and `l` is 3. That pass adds 10. The difference is 4.

Those four subarrays are `[1, 0, 1]` at indexes 0 through 2, `[1, 0, 1, 0]` at indexes 0 through 3, `[0, 1, 0, 1]` at indexes 1 through 4, and `[1, 0, 1]` at indexes 2 through 4. Each sums to 2.

EXACTLY K, FOR 0/1 ARRAYS. THE VALUES ARE NON-NEGATIVE, SO THE WINDOW IS LEGAL.

```
function numSubarraysWithSum(nums, goal) {
  const atMost = (limit) => {
    if (limit < 0) return 0;
    let l = 0;
    let sum = 0;
    let count = 0;
    for (let r = 0; r < nums.length; r++) {
      sum += nums[r];
      while (sum > limit) {
        sum -= nums[l];
        l += 1;
      }
      count += r - l + 1;
    }
    return count;
  };
  return atMost(goal) - atMost(goal - 1);
}
```

Inside `atMost` you add `nums[r]`, then you shrink while the sum is greater than the limit, then you add `r - l + 1`. The shrink is a `while`. One drop may leave the sum too high, as the 0 does in `atMost(1)` at `r = 4`. If the limit is below 0, the helper returns 0, so `atMost(goal - 1)` is safe when `goal` is 0. A negative value breaks the subtraction. On `[1, 2, -1, 2]` with goal 2, the same helpers return 2. The real count of subarrays with sum 2 is 3. `l` moved past index 0 when the sum hit 3, and it could not come back for `[1, 2, -1]`.

Cards from the two ends

You take `k` cards, and each card comes from one of the two ends. That is the same as leaving a middle band of length `n - k` untouched. The best score is the total minus the minimum sum of a middle window of that length. If `k` equals `n`, the middle is empty and the answer is the total.

Take `[1, 2, 3, 4, 5, 6, 1]` and `k = 3`. The length is 7, so the middle window has length 4. The total is 22. The middle sums are `1+2+3+4 = 10`, `2+3+4+5 = 14`, `3+4+5+6 = 18`, and `4+5+6+1 = 16`. The minimum is 10. The score is `22 - 10`, which is 12. Those three cards are the right-hand 5, 6, and 1.

You build the total in one pass. You then slide a fixed window of length `n - k`, subtracting the cell that leaves and adding the cell that enters. You keep the smallest middle sum. You subtract that from the total after the slide finishes. An empty middle, when `k` equals `n`, has sum 0, so you must not look for a window of length 0.

Minimum operations to reduce `x` to zero, when every value is positive, is the same idea from the other side. You remove a prefix and a suffix, and those removed cells together sum to `x`. A prefix is the stretch from index 0 up to a cut. The leftover middle is the longest subarray whose sum is `total - x`. The answer is `n` minus that length, or −1 if that sum never appears. A negative `total - x` is impossible, and the answer is −1.

Take `[1, 1, 4, 2, 3]` and `x = 5`. The total is 11, so the middle must sum to 6. The longest such subarray is `[1, 1, 4]`, length 3. The answer is `5 - 3`, which is 2. The removed piece is the suffix `[2, 3]`. You find that longest middle with the same shrink you used for a target sum, then you subtract its length from `n`.

A window that is a multiset

Permutation in string and find-all-anagrams share one window whose length is the length of the pattern. You count what the pattern needs. A character becomes matched when its window count equals its needed count. It becomes unmatched when you are about to drop it and the two counts are still equal. When the number of matched characters equals the number of distinct characters in the pattern, the window is an anagram.

Walk `s = cbaebabacd` and `p = abc`. The window length is 3. The pattern needs one `a`, one `b`, and one `c`. Indexes 0 through 2 are `cba`. All three counts match, so you record the start 0. You check that equality before you decrement the leaving `c`. If you decrement first, `cba` no longer matches, and you miss index 0.

The window then slides. The next match is indexes 6 through 8, the letters `bac`. You record 6. Each slide adds the new right character and drops the old left character. You update the matched count when a count hits the needed count, and when a drop makes a matched count fall below the needed count. The answer list is `[0, 6]`.

The shortest covering window

Minimum window substring is the variable window on character counts. `missing` counts required copies, not distinct letters. You shrink while the window still covers every required copy, and you remember the shortest span.

Walk `s = aabbc` and `t = abc`. The need map starts as one `a`, one `b`, and one `c`. `missing` starts at 3. At `r = 0` you take an `a`. Its need falls to 0, and `missing` falls to 2. At `r = 1` you take another `a`. Its need falls to −1. That value is below 0, so `missing` stays 2. The extra `a` is not a new required copy.

At `r = 2` you take a `b` and `missing` falls to 1. At `r = 3` the extra `b` drives the need below 0, so `missing` stays 1. At `r = 4` you take the `c`. `missing` hits 0. The window `aabbc` covers the pattern, and its length is 5. You store start 0 and length 5.

The `while` now drops from the left. You put the first `a` back. Its need goes from −1 to 0, which is not above 0, so `missing` stays 0. `l` moves to 1. The window `abbc` has length 4, and you store that shorter span. You then put the second `a` back. Its need climbs from 0 to 1, so `missing` becomes 1, and the `while` stops. The answer is `abbc`. `bbc` has lost the required `a`.

SECOND PASS. MISSING COUNTS REQUIRED COPIES, NOT DISTINCT LETTERS.

```
function minWindow(s, t) {
  const need = new Map();
  for (const ch of t) need.set(ch, (need.get(ch) ?? 0) + 1);
  let missing = t.length;
  let bestL = 0;
  let bestLen = Infinity;
  let l = 0;
  for (let r = 0; r < s.length; r++) {
    if (need.has(s[r])) {
      need.set(s[r], need.get(s[r]) - 1);
      if (need.get(s[r]) >= 0) missing -= 1;
    }
    while (missing === 0) {
```

```
      const len = r - l + 1;
      if (len < bestLen) {
        bestLen = len;
        bestL = l;
      }
      if (need.has(s[l])) {
        need.set(s[l], need.get(s[l]) + 1);
        if (need.get(s[l]) > 0) missing += 1;
      }
      l += 1;
    }
  }
  return bestLen === Infinity ? "" : s.slice(bestL, bestL + bestLen);
}
```

You decrease the need when `s[r]` is required, and you decrease `missing` only when the need is still at least 0 after that decrease. The `while` runs while `missing` is 0. You record `r - l + 1` before you restore `s[l]`. You increase `missing` only when the restored count climbs above 0. Then you move `l`. If you move `l` before you record, you save a window that has already lost a required copy.

The maximum inside a sliding window does not fit in one variable. When that maximum slides out of the left edge, the variable does not know the second maximum. Chapter 15 keeps a deque of candidate indexes. On `[1, 3, 2, 1]` with `k = 2`, the window slides from `[3, 2]` to `[2, 1]`. A variable that only stored 3 still says 3. The maximum is now 2.

Time and extra memory

Each index enters the window once and leaves at most once, so the time is O(n), a sum uses O(1) extra memory, and a frequency map uses extra memory proportional to the distinct values you touch.

Mistakes

- On `[2, 3, 1, 2, 4, 3]` with target 7, shrinking with `if` returns 4. At `r = 4` one drop leaves the sum at 7, the `if` stops, and the later window never reaches `[4, 3]`. The real answer is 2.
- On that same input, storing `r - l` returns 1. The best window is indexes 4 and 5, the cells `[4, 3]`, and `r - l` is 1. The ordinary length is `r - l + 1`, which is 2.
- On `[1, 1, 1]`, `longestSubarray` must return 2. Storing `r - l + 1` returns 3, because that formula never performs the required deletion.
- On the string `abba`, moving `l` to 1 when the old `a` sits at 0 pulls the left end backward. The window becomes `bba`, which repeats `b`. The old `a` was already outside.
- On `[1, -2, 3]` with target 2, the shrink-while-the-sum-is-large loop returns 3. After it drops the leading 1, the sum is 1, so it stops, and it never tests the single cell 3. The real answer is 1. The negative broke the monotone rule.
- On `[1, 2, -1, 2]` with goal 2, `atMost(2)` minus `atMost(1)` returns 2. Three subarrays actually sum to 2. The window cannot move `l` back to recover `[1, 2, -1]`.
- On `[1, 3, 2, 1]` with window length 2, a single variable still holds 3 after that 3 has left. The window `[2, 1]` has maximum 2. Chapter 15 is the tool for that.

Practice

Stay before chapter 8. Then before you leave the chapter. The two hard problems are a second pass.

[643. Maximum Average Subarray I](#) EASY STAY

Fixed k. Add the new cell, subtract the cell that fell out.

209. Minimum Size Subarray Sum MEDIUM STAY
Positive numbers. Shrink while the sum is at least the target. Return 0 if none.

1004. Max Consecutive Ones III MEDIUM STAY
At most k zeros in the window. The answer is the window length, flips included.

3. Longest Substring Without Repeating Characters MEDIUM STAY
Jump l to lastIndex + 1 only when lastIndex is still >= l.

713. Subarray Product Less Than K MEDIUM STAY
Positive integers. Each valid window ending at r contributes r − l + 1.

1456. Maximum Number of Vowels in a Substring of Given Length MEDIUM THEN
Fixed k, but the sum is a vowel count: add 1 or 0, subtract the leaving 1 or 0.

1493. Longest Subarray of 1's After Deleting One Element MEDIUM THEN
At most one zero. Answer is window length minus 1.

904. Fruit Into Baskets MEDIUM THEN
At most two distinct values. Delete a key when its count hits 0.

930. Binary Subarrays With Sum MEDIUM THEN
Exactly goal, on 0/1 values: atMost(goal) − atMost(goal − 1).

1248. Count Number of Nice Subarrays MEDIUM THEN
Exactly k odd numbers. Treat odd as 1 and reuse the subtraction.

567. Permutation in String MEDIUM THEN
Fixed window of length s1. Compare counts, or track how many characters are matched.

438. Find All Anagrams in a String MEDIUM THEN
The same window. Record the start index every time the window matches.

1423. Maximum Points You Can Obtain from Cards MEDIUM THEN
Total minus the minimum middle window of length n − k.

1658. Minimum Operations to Reduce X to Zero MEDIUM THEN
Positive nums. Longest subarray with sum total − x, then n minus that length.

76. Minimum Window Substring HARD SECOND PASS
Shrink while every required character is covered. Keep the shortest span.

992. Subarrays with K Different Integers HARD SECOND PASS
atMost(K) − atMost(K − 1) with a frequency map of values.

REWRITE FROM MEMORY
Rewrite the minimum-length window and the exactly-K subtraction. Say the length out loud as r minus l plus 1.

Carry forward. Chapter 8 answers the sum questions a window must refuse: negatives, and counting every subarray whose sum is K.

CHAPTER 08 · BUILDS ON CHAPTERS 2 AND 7

Prefix sums

A prefix stores the sum of a starting piece. You ask how many earlier prefixes equal sum minus k, and only then do you record the sum you have now.

What this pattern is

A prefix is the sum of a starting piece of the array. `prefix[0]` is 0. That 0 is the empty prefix, the sum before you have read any cell. `prefix[i]` is the sum of the first `i` cells, which is `nums[0] + … + nums[i − 1]`.

Picture a taxi meter. The cost of the ride from street `i` through street `j`, both ends included, is the meter just after `j` minus the meter at `i`. In the array, that sum is `prefix[j + 1] − prefix[i]`.

A remainder is what is left after you divide a prefix by `k`. A subarray is a stretch of cells with no gaps. Two prefixes with the same remainder differ by a multiple of `k`, so the subarray between them is divisible by `k`.

You keep the prefixes you have already finished in a map. At the current sum you ask how many earlier prefixes equal `sum − k`. You record the current sum only after that question.

What you already know

Chapter 2 kept one running total while you walked. Chapter 7 used a running total inside a window, the band from index `l` through index `r`, and it had to refuse the job when a negative could make the sum fall. A prefix keeps every running total, not only the total inside the current band. From those totals you can read any subarray sum, and you can count the subarrays that add up to `k`.

A worked example

Use `[2, 1, −1, 3, 2]` and `k = 3`. The map starts with the empty prefix: key 0 has count 1. The sum starts at 0. You have not stored any cell yet.

At index 0 you add 2. The sum is 2. You ask for `2 − 3`, which is −1. The map has no −1, so the count stays 0. You then record 2. The map is `{0: 1, 2: 1}`.

At index 1 you add 1. The sum is 3. You ask for 0. The seed answers once. That hit is the subarray from the start, the cells `[2, 1]`. The count is 1. You then record 3.

At index 2 you add −1. The sum is 2. You ask for −1 and miss. You then record 2. The key 2 now has count 2, because the sum was 2 after index 0 and it is 2 again after index 2. You record that second 2 only after the question, so later indexes can see both copies and this index cannot see the copy it just made.

At index 3 you add 3. The sum is 5. You ask for 2. The map answers 2. One earlier prefix is the sum after index 0, so the subarray is `[1, −1, 3]`. The other is the sum after index 2, so the subarray is `[3]`. The count is 3. You then record 5.

At index 4 you add 2. The sum is 7. You ask for 4 and miss. The count stays 3. Those three subarrays are the only ones that add to 3. If `k` were 0, recording the sum before the question would count an empty piece, because `sum − k` would be the sum you just stored.

The same query-then-record order on a smaller array is this trace.

```
[1, 2, 3], k = 3
start {0: 1}
sum 1, ask for −2, miss, record 1
sum 3, ask for 0, hit 1, record 3     subarray [1, 2]
sum 6, ask for 3, hit 1, record 6     subarray [3]
count = 2

if you incremented first, a target of 0 would count the empty prefix as a new subarray
```

How you recognise it

- You need the sum of many ranges, and one pass can build the prefix before you answer them.
- You need the number of subarrays whose sum is k, and the values may be negative or zero.
- You need the longest subarray with sum k, and a negative means you keep the first index of each prefix.
- The test is about a remainder, so two prefixes match when their repaired remainders are equal.
- A window is the wrong tool because adding a cell can make the running sum smaller.

Some prompts look close and belong in another chapter.

- Use chapter 7 when every value is non-negative and you want the shortest subarray whose sum is at least k.
- Use chapter 11 when you want the one subarray with the largest sum, and an empty subarray is not allowed.
- Use chapter 4 when you need two cells that add to a target, and those cells do not have to form a subarray.

What to say out loud

An invariant is a fact you keep true on every step. The map holds prefixes you have already finished, and the empty prefix is stored as 0 with count 1.

You add the next cell into the sum. You ask for `sum − k`. Only then do you add one to the count of the sum you are standing on.

The template

SEED 0 → 1. QUERY, THEN INCREMENT.

```
function buildPrefix(nums) {
  const prefix = [0];
  for (const x of nums) prefix.push(prefix[prefix.length − 1] + x);
  return prefix;
}

function subarraySum(nums, k) {
  const freq = new Map();
  freq.set(0, 1);
  let sum = 0;
  let count = 0;
  for (const x of nums) {
    sum += x;
    count += freq.get(sum − k) ?? 0;
    freq.set(sum, (freq.get(sum) ?? 0) + 1);
  }
  return count;
}
```

Build the prefix by starting with `[0]`. For each value, push the last prefix plus that value. `prefix[0]` stays 0, the empty prefix. On `[2, 1, −1, 3, 2]` the prefix is `[0, 2, 3, 2, 5, 7]`. The inclusive sum of indexes 1 through 3 is the cells 1, −1, and 3. That is `prefix[4] − prefix[1]`, because `j` is 3 and `i` is 1, so you read `prefix[j + 1] − prefix[i]`. The difference is `5 − 2`, which is 3.

In the counting loop the seed is `freq.set(0, 1)` before you read a cell. Without that seed, a subarray that starts at index 0 is invisible. On `[1, 2]` with `k = 3`, the sum becomes 3 and the question is 0. If 0 is missing, the count stays 0, and you miss `[1, 2]`.

The order inside the loop is fixed. Add the cell, ask for `sum - k`, then increment. If you increment first, a target of 0 counts an empty piece. On `[1]` with `k = 0`, no non-empty subarray sums to 0. The wrong order stores 1 and then asks for 1, so it returns 1. That 1 is the empty piece from the prefix to itself.

The longest subarray with sum `k` stores the first index of each prefix, seeded as 0 at index −1. When you see `sum - k`, the length is the current index minus that first index. You do not overwrite an earlier index, because an earlier start makes a longer subarray. On `[1, -1, 1, -1]` with `k = 0`, the prefixes are 1, 0, 1, 0. The first 0 is the seed at −1, so the length at the end is 4. If you store the later 0 instead, the length you report is 2.

The same remainder

JavaScript keeps the sign of the dividend. `(-7) % 5` is −2, not 3. Two prefixes are the same remainder only after you repair the value. The repair is `((sum % k) + k) % k`. For −7 and `k = 5`, that expression is 3. The bare −2 and the repaired 3 differ by a multiple of 5, and they miss each other if you store the raw result.

Walk `[2, -3, 1, 3, 2]` with `k = 5`. The map again starts as `{0: 1}`. Index 0 makes the sum 2. The repaired remainder is 2. You miss, then you store 2.

Index 1 makes the sum −1. In JavaScript `(-1) % 5` is −1. The repair turns that into 4. You store 4, not −1. Index 2 makes the sum 0. The remainder is 0, and the seed answers once. The subarray `[2, -3, 1]` sums to 0, which is divisible by 5. The count is 1. You then increment the count of 0, so the key 0 now holds 2.

Index 3 makes the sum 3. The remainder is 3. You miss. Index 4 makes the sum 5. The remainder is 0. The map answers 2. One hit is the whole array, which sums to 5. The other is `[3, 2]`, which also sums to 5. The count is 3. You asked for the repaired remainder before you incremented it, the same order as the sum map.

THE ADDED K FOLDS A NEGATIVE REMAINDER BACK INTO 0..K−1.

```
function subarraysDivByK(nums, k) {
  const freq = new Map();
  freq.set(0, 1);
  let sum = 0;
  let count = 0;
  for (const x of nums) {
    sum += x;
    const mod = ((sum % k) + k) % k;
    count += freq.get(mod) ?? 0;
    freq.set(mod, (freq.get(mod) ?? 0) + 1);
  }
  return count;
}
```

```
(-7) % 5 in JavaScript is -2
((-7 % 5) + 5) % 5 = 3
that matches 3, because -7 and 3 differ by -10, a multiple of 5
```

You add the cell, you repair the remainder, you query that repaired value, and then you increment it. The added `k` folds a negative remainder back into the range 0 through `k - 1`. If you store the raw `%`, the −2 bucket and the 3 bucket never meet. The seed 0 with count 1 is still required, so a prefix that is itself divisible by `k` is counted.

Length at least 2

Continuous subarray sum asks whether some subarray of length at least 2 has a sum divisible by `k`. Current constraints have `k` at least 1, so you will not divide by zero. You seed remainder 0 at index −1. You store only the

first index of each remainder. A later copy at index `i` is a hit when `i - first >= 2`.

Walk `[23, 2, 4, 6, 7]` with `k = 6`. At index 0 the sum is 23 and the remainder is 5. The map does not have 5, so you store 5 at index 0. At index 1 the sum is 25 and the remainder is 1. You store 1 at index 1.

At index 2 the sum is 29 and the remainder is 5. The map already has 5, stored at index 0. `2 - 0` is 2, which is at least 2, so you return true. The subarray is indexes 1 through 2, the cells `[2, 4]`, and the sum is 6. You do not replace the stored 0 with 2. The first index is the one that makes the span long enough.

DO NOT UPDATE AN INDEX YOU HAVE ALREADY STORED.

```
function checkSubarraySum(nums, k) {
  const first = new Map();
  first.set(0, -1);
  let sum = 0;
  for (let i = 0; i < nums.length; i++) {
    sum += nums[i];
    const mod = ((sum % k) + k) % k;
    if (first.has(mod)) {
      if (i - first.get(mod) >= 2) return true;
    } else {
      first.set(mod, i);
    }
  }
  return false;
}
```

```
[23, 2, 4, 6, 7], k = 6
i=0 sum 23, mod 5, store 5 at 0
i=1 sum 25, mod 1, store 1 at 1
i=2 sum 29, mod 5, 2 - 0 = 2, return true
the subarray [2, 4] sums to 6
```

You repair the remainder before you look in the map. If the key is already there, you test the distance and you do not write. You write the index only when the remainder is new. The seed puts 0 at −1 so a subarray that begins at index 0 can be long enough. On `[5, 5]` with `k = 5`, the second remainder 0 is two steps from −1, and `[5, 5]` is a hit. If you overwrite the first index, a long enough span shrinks to a short one. On `[5, 0, 0]` with `k = 5`, replacing −1 with 0 makes every later step look one cell too short, and the function returns false. `[5, 0]` and `[0, 0]` are both valid.

The same count of zeros and ones

An equal number of 0s and 1s becomes a prefix sum if you map 0 to −1 and 1 to +1. A subarray with sum 0 is a balance. You seed index −1 at prefix 0, and you store the first index only.

Walk `[0, 1, 0, 1]` after the map, so the values are `[-1, 1, -1, 1]`. The running sums are −1, 0, −1, and 0. At index 1 the sum is 0. The seed sits at −1, so the length is 2. That subarray is `[0, 1]`. You do not overwrite the seed. At index 3 the sum is 0 again. The length from −1 is 4, the whole array, which has two 0s and two 1s. If you had stored the later index of 0, this last step would report length 2 and you would miss the longer balance.

You write an index only the first time you see that prefix. Each later copy uses the stored index to measure the length. The answer is the largest of those lengths.

Time and extra memory

You walk the array once and each map operation is expected constant time, so the time is O(n) and the map uses O(n) extra memory; a prefix array for range queries is O(n) to build and then O(1) per query.

Mistakes

- On `[1, 2]` with `k = 3`, forgetting the seed `0 → 1` returns 0. The subarray `[1, 2]` starts at index 0, and it needs the empty prefix.
- On `[1]` with `k = 0`, incrementing the frequency before the query returns 1. No non-empty subarray sums to 0. The extra 1 is the empty piece.
- On a running sum of −7 with `k = 5`, the raw JavaScript remainder is −2. The repaired remainder is 3. Storing −2 splits the buckets, so a later prefix with remainder 3 misses it.
- On `[1, -1, 1, -1]` with `k = 0`, storing the latest index of prefix 0 returns 2. The longest subarray with sum 0 is the whole array, length 4.
- On `[5, 5]` with `k = 5`, a continuous-subarray check with no seed at −1 returns false. `[5, 5]` sums to 10, and the distance from the missing seed is what makes the length 2.
- On `[5, 0, 0]` with `k = 5`, overwriting the first index of remainder 0 returns false. The spans `[5, 0]` and `[0, 0]` both qualify, and each later write makes the distance look shorter than 2.

Practice

Stay before chapter 9. Then before you leave the chapter.

303. Range Sum Query - Immutable EASY STAY
Build prefix once in the constructor. sumRange(i, j) is prefix[j + 1] − prefix[i].

560. Subarray Sum Equals K MEDIUM STAY
Seed 0 → 1. Add the count of sum − k, then increment sum.

525. Contiguous Array MEDIUM STAY
Map 0 to −1. Longest prefix balance of 0. Seed index −1, store the first index.

974. Subarray Sums Divisible by K MEDIUM THEN
Frequency of the repaired remainder. The repair is ((sum % k) + k) % k.

523. Continuous Subarray Sum MEDIUM THEN
First index only, seed −1, length at least 2. k ≥ 1 under the current constraints.

REWRITE FROM MEMORY
Rewrite the frequency map, including the seed 0 to 1 and the query before the update, and the repair for a negative remainder.

Carry forward. Chapter 9 runs this idea backward. Instead of summing a range, you add a value onto a range with two writes.

CHAPTER 09 · BUILDS ON CHAPTER 8

Difference array

A range add is two marks. One later scan turns those marks back into the array.

What this pattern is

A difference array queues range additions. You want to add a value onto many ranges, and you will read the array only after every update is queued. Each range becomes two writes. One later scan rebuilds the array.

Picture a row of fence planks. To paint planks `L` through `R` with 10 more coats, you mark +10 on plank `L` and −10 on the plank just after `R`. You do not touch the planks in between. One walk along the fence applies the marks: the running total goes up when a mark starts, and it goes down when a mark stops.

For an inclusive range `[L, R]`, the stop mark sits at `R + 1`. Index `R` still receives the value. The difference array has one extra cell so a mark that lands just past the end has a place to sit.

What you already know

Chapter 8 builds a prefix, the sum of a starting piece, so you can read a range sum in constant time after a linear build. A difference array is that idea run backward. You already know a scan can turn a list of changes into the values you stand on. Here you queue the changes first, two writes per range, and you scan once at the end.

A worked example

Take `n = 5`, so the real indexes are 0 through 4, and give the difference array a sixth cell at index 5. It starts as six zeros. You will add 10 on the inclusive range `[1, 3]`, then add 5 on the inclusive range `[2, 4]`.

The first update writes +10 at index 1 and −10 at index 4, because `R + 1` is 4. The array is `[0, 10, 0, 0, −10, 0]`. Index 3 has no mark of its own. It will still receive the 10, because the +10 is still in the running total when you arrive there.

The second update writes +5 at index 2 and −5 at index 5. The −5 lands in the extra cell, past the last real index. The array is `[0, 10, 5, 0, −10, −5]`.

Now you scan. The running value starts at 0. At index 0 you add 0, so the cell is 0. Nothing was added at the start. At index 1 you add 10, so the cell is 10. The first range has turned on.

At index 2 you add 5, so the cell is 15. Both ranges cover index 2. At index 3 you add 0, so the cell stays 15. Index 3 is inside `[1, 3]` and inside `[2, 4]`. The running value carries the earlier marks. You did not need a mark on the plank itself.

At index 4 you add −10, so the cell is 5. The first range ended at 3, so index 4 must lose the 10. The second range still covers 4, so 5 remains. You never copy cell 5 into the answer. It holds the −5 that would cancel the second range if the array continued. The final array is `[0, 10, 15, 15, 5]`.

If the first update had subtracted at `R` instead of `R + 1`, the −10 would land on index 3. Index 3 would become 5, and it would miss the 10 that the inclusive range promised it.

The marks and the scan, in a compact trace, look like this.

```
n = 5, add 10 on [1, 3], add 5 on [2, 4]
diff starts [0, 0, 0, 0, 0, 0]
after [1, 3]: [0, 10, 0, 0, -10, 0]
after [2, 4]: [0, 10, 5, 0, -10, -5]
scan: 0, 10, 15, 15, 5
```

How you recognise it

- You add a value onto many ranges, and you read the array only after every update is queued.
- Each event has a start and an end, such as a booking, a ride, or a birth and a death.
- The timeline is a line of slots, such as flight numbers, stops on a road, or years from 1950 to 2050.
- Each update can be a mark where the change starts and a mark where the change stops.
- You do not need the array in the middle of the updates. One scan at the end is enough.

Some prompts look close and belong in another chapter.

- Use chapter 8 when you will read many range sums, and you are not adding a value onto many ranges.
- Use chapter 7 when one fixed-length band slides forward and you only add the new cell and drop the old cell.
- Use chapter 21 when the ranges are rectangles on a grid and you need a two-dimensional prefix.

What to say out loud

An invariant is a fact you keep true after every mark. `diff[i]` is the change in the running value as you step onto index `i`.

A range from `L` through `R`, both ends included, writes `+v` at `L` and `−v` at `R + 1`. The scan adds those marks into a running value and writes that value into the answer.

The template

THE EXTRA CELL HOLDS THE −V THAT WOULD HAVE LANDED JUST PAST THE END.

```
function applyRanges(n, updates) {
  const diff = new Array(n + 1).fill(0);
  for (const [L, R, v] of updates) {
    diff[L] += v;
    diff[R + 1] -= v;
  }
  const arr = new Array(n).fill(0);
  let run = 0;
  for (let i = 0; i < n; i++) {
    run += diff[i];
    arr[i] = run;
  }
  return arr;
}
```

You apply every update as two writes before you read any answer cell. Inclusive `[L, R]` is `+v` at `L` and `−v` at `R + 1`. `diff` has length `n + 1` so the minus can land just past the end.

The scan adds `diff[i]` into the running value and then writes the answer cell. You do not read `diff[n]` into the answer. If you subtract at `R`, index `R` loses the value on the same step it should receive it. On the first update above, that mistake turns index 3 from 15 into 5 once the second update is included, because the 10 is cancelled one plank too soon.

Flights numbered from 1

Corporate flight bookings are 1-based. Flight numbers run from 1 to `n`. A booking of `seats` on flights `first` through `last`, both ends included, is not a write at index `first`. You write `diff[first − 1] += seats` and `diff[last] −= seats`.

Take `n = 5` and one booking of 10 seats on flights 2 through 4. Flight 2 is index 1, so you add 10 at index 1. The subtraction lands at index 4, which is flight 5, the first flight that should not receive the seats. The scan produces `[0, 10, 10, 10, 0]`. Flights 2, 3, and 4 gained the seats.

When `last` equals `n`, the subtraction hits the extra cell and never enters the answer. A booking of 10 seats on flight 1 only, with `n = 1`, writes +10 at index 0 and −10 at index 1. The scan returns `[10]`. If you treat the flight number as an index and write both marks on index 1, they cancel, and flight 1 stays 0.

Passengers leave at the destination

Car pooling uses a half-open ride. Passengers board at `from` and leave at `to`, so they do not occupy the destination. You write `diff[from] += passengers` and `diff[to] -= passengers`. You scan the road. If the running load ever exceeds capacity, you return false. The timeline only needs to be as long as the farthest stop.

Take capacity 2, a trip of 2 passengers from stop 1 to stop 3, and a trip of 2 passengers from stop 3 to stop 4. The first trip adds 2 at stop 1 and subtracts 2 at stop 3. The second trip adds 2 at stop 3 and subtracts 2 at stop 4. At stop 1 the load is 2. At stop 2 the load is 2. At stop 3 the first group leaves and the second group boards, so the load stays 2. The load never exceeds 2, so the answer is true.

You add the board mark and the leave mark for every trip before you scan. During the scan you add `diff[i]` and then you compare the load with capacity. If you keep the first group through the destination, stop 3 holds both groups and the load is 4. That wrong code returns false on this input. The passengers from the first trip are already gone at stop 3.

A tie keeps the earlier year

Maximum population year is the same two writes on a tiny timeline. A person is alive from `birth` inclusive to `death` exclusive. You shift the years by 1950 so the indexes start at 0. When two years tie, you keep the earlier one. You update the best year only when the running population is strictly greater.

Take one person born in 1950 who dies in 1960. That person is alive in 1950 through 1959. You add 1 at index `1950 - 1950`, which is 0, and you subtract 1 at index `1960 - 1950`, which is 10. The scan sees population 1 for ten years, then 0.

The first year with population 1 is 1950. Every later year through 1959 ties that population. A strict greater test leaves the stored year at 1950. If you update whenever the population is greater than or equal to the best, each tied year replaces the stored year, and you return 1959. Death is exclusive, so 1960 itself is not a living year and must not receive the +1.

Coverage and a dot product

Maximum sum of any permutation scores each index by how many requests cover it. The difference array builds that coverage. You sort the values and you sort the coverages. You pair the largest value with the most requested index. You take the dot product modulo `1e9+7`. You reduce the sum modulo at every step so the product stays inside a safe JavaScript integer.

Take three indexes and two requests, `[0, 1]` and `[1, 2]`. The first request writes +1 at 0 and −1 at 2. The second writes +1 at 1 and −1 at 3. The scan gives coverage `[1, 2, 1]`. Index 1 is covered twice, and the ends are covered once.

If the values are `[1, 2, 3]`, you sort them to `[3, 2, 1]` and you sort the coverage to `[2, 1, 1]`. The dot product is `3 * 2 + 2 * 1 + 1 * 1`, which is 9. You place the biggest value on the busiest index, then the next biggest on the next busiest index. The order of those two sorts is the whole second step.

Flips as a running parity

Minimum number of K consecutive bit flips stores each flip as parity. A window here is the band of `k` neighboring cells that one flip covers. The running value is the number of active flips modulo 2. If the current bit, after the flips already started, is 0, you must start a flip at `i`. That flip adds 1 now and expires `k` steps later. If that end would fall past `n`, you return −1.

Walk `[0, 1, 0]` with `k = 1`. At index 0 the running parity is 0, and `(0 + 0)` is even, so the bit you see is 0. A flip of length 1 ends at index 1, which is inside the array. You count one flip, the parity becomes 1, and you write −1 at `diff[1]`.

At index 1 you add that −1, so the parity returns to 0. The stored bit is 1, and `(1 + 0)` is odd, so you do not flip. At index 2 the parity is 0 and the stored bit is 0, so you see a 0. A flip of length 1 ends at index 3, which is allowed. You count a second flip. The answer is 2. The first flip turns the array into `[1, 1, 0]`, and the second turns it into `[1, 1, 1]`.

The same array with `k = 3` fails. Index 0 starts a flip, and the parity becomes 1. At index 1 the bit you see is `(1 + 1)` modulo 2, which is 0, so you need another flip. `i + k` is 4, which is past `n`. You return −1. The only flip that fits, the one that starts at 0, turns `[0, 1, 0]` into `[1, 0, 1]` and leaves index 1 at 0.

SECOND PASS. STARTING A FLIP IS A +1 THAT EXPIRES AT I + K.

```
function minKBitFlips(nums, k) {
  const n = nums.length;
  const diff = new Array(n + 1).fill(0);
  let cur = 0;
  let flips = 0;
  for (let i = 0; i < n; i++) {
    cur += diff[i];
    if ((nums[i] + cur) % 2 === 0) {
      if (i + k > n) return -1;
      flips += 1;
      cur += 1;
      diff[i + k] -= 1;
    }
  }
  return flips;
}
```

You add `diff[i]` into the running parity before you look at `nums[i]`. The bit you see is `(nums[i] + cur) % 2`. When that is 0, you need a flip at `i`. If `i + k` is greater than `n`, you return −1. Otherwise you add 1 to the flip count, add 1 to the running parity, and you write −1 at `diff[i + k]` so the flip expires when the window ends. You do not also write +1 at `diff[i]`, because you already added that 1 into the running parity for the current index.

Time and extra memory

Each update is two writes and the final scan visits each slot once, so the time is O(n + u) for u updates and the extra memory is O(n).

Mistakes

- On `n = 5` with +10 on inclusive `[1, 3]`, subtracting at 3 instead of 4 drops index 3. That index is inside the range, and the wrong scan never leaves the 10 there.
- On `n = 1` with 10 seats booked on flight 1, treating the flight number as an index writes +10 and −10 on the same cell. The answer comes back `[0]`. The shifted write, +10 at index 0 and −10 at index 1, returns `[10]`.
- On capacity 2 with passengers 2 from stop 1 to stop 3 and passengers 2 from stop 3 to stop 4, occupying the destination returns false. The load at stop 3 looks like 4. The first group has already left, and the real load is 2.
- On a person alive from 1950 inclusive to 1960 exclusive, updating the best year on a tie returns 1959. The population is 1 for every year from 1950 through 1959, and the earlier year 1950 is the one you keep.

Practice

Stay before chapter 10. The last two are a second pass.

1854. Maximum Population Year EASY STAY
Difference on years 1950..2050. Death is exclusive. Ties keep the earlier year.

1109. Corporate Flight Bookings MEDIUM STAY
1-based. diff[first − 1] += seats and diff[last] −= seats.

1094. Car Pooling MEDIUM STAY
Half-open trip [from, to). Fail as soon as the running load exceeds capacity.

1589. Maximum Sum Obtained of Any Permutation HARD SECOND PASS
Coverage by a difference array, sort both sides, dot product, modulo 1e9+7.

995. Minimum Number of K Consecutive Bit Flips HARD SECOND PASS
A flip is a parity update of length k. Return −1 when a needed flip runs off the end.

REWRITE FROM MEMORY
Rewrite the inclusive update, plus v at L and minus v at R plus 1, and the scan that rebuilds the array.

Carry forward. Chapter 10 builds a full left summary and a full right summary, then reads them together at each index.

CHAPTER 10 · BUILDS ON CHAPTERS 2 AND 5

A left pass and a right pass

You build one summary from the left and one from the right. At each index you read the two summaries together.

What this pattern is

Some answers need both directions. You build one array from left to right. You build another from right to left. Then you combine them at each index.

Picture yourself standing at one bar in a row of bars. You look left and remember the tallest bar on that side. You look right and remember the tallest bar on that side. The water at your feet is the lower of those two heights, minus the bar you are standing on. A product of every cell except this one is the same shape: the product of everything to the left, times the product of everything to the right.

The extra memory is often `O(n)` for the two summaries. The product routine can store the left products in the answer array, so the extra memory beyond the answer is constant. The passes are still linear.

What you already know

Chapter 2 was enough when the answer at `i` used only the cells you had already seen, or only a scan that ran backward. Chapter 5 moved inward from the two ends when each step could drop one end. Here you finish a summary of the whole left side and a summary of the whole right side, and you read the pair at `i`.

A worked example

Use `[1, 2, 3, 4, 5]`. You want the product of every cell except the one you are on. You will not use division. `left` starts at 1, which is the product of no cells.

At index 0 you write `left` into the answer before you multiply. The answer cell becomes 1. Then `left` becomes `1 * 1`, which is 1. At index 1 you write 1, then `left` becomes `1 * 2`, which is 2. At index 2 you write 2, then `left` becomes 6. At index 3 you write 6, then `left` becomes 24. At index 4 you write 24, then `left` becomes 120. The answer so far is `[1, 1, 2, 6, 24]`. Each cell holds the product of the cells strictly before it.

You now walk backward. `right` starts at 1, the product of no cells to the right of the end. At index 4 you multiply the answer cell by `right` before you fold `nums[4]` into `right`. `24 * 1` is 24, and `right` becomes 5.

At index 3 the answer cell is 6, and `right` is 5, so the cell becomes 30. Then `right` becomes `5 * 4`, which is 20. At index 2 the cell 2 becomes `2 * 20`, which is 40, and `right` becomes 60. At index 1 the cell 1 becomes 60, and `right` becomes 120. At index 0 the cell 1 becomes 120, and `right` becomes 120.

The answer is `[120, 60, 40, 30, 24]`. Check index 2: the product of everything except 3 is `1 * 2 * 4 * 5`, which is 40. The left product written at that index was 2, and the right product multiplied in was 20. The cell's own value, 3, was never written into its own answer.

The order is the whole trick. You write the left product before you multiply `nums[i]` into `left`. If you multiply first on this array, the left pass writes `[1, 2, 6, 24, 120]`, and every cell already contains itself.

How you recognise it

- The value at index i depends on a summary of every cell to its left and a summary of every cell to its right.
- You need a product of the other cells, the water above a bar, the length of a mountain, or a candy count forced by both neighbors.
- You want a summary of a whole side, rather than the nearest cell that beats this one.
- One running value from a single direction is not enough, because the other direction is part of the answer.

- The two passes are still one read each, and you can spend linear extra memory on the two summaries.

Some prompts look close and belong in another chapter.

- Use chapter 2 when the answer at i needs only a summary of the cells before i.
- Use chapter 5 when two indices start at the two ends of a sorted array and you are choosing which end to drop.
- Use chapter 15 when you need the nearest taller bar, rather than the tallest bar on the whole side.

What to say out loud

An invariant is a fact you keep true at the end of a pass. After the left pass, the value at `i` describes cells strictly before `i`.

After the right pass, you have folded in the cells strictly after `i`. The cell `i` itself stays out of its own product. You multiply it into the running value only after the answer cell has been written.

The template

THE OUTPUT ARRAY STORES THE LEFT PRODUCTS, SO THE EXTRA MEMORY BEYOND THE ANSWER IS O(1).

```
function productExceptSelf(nums) {
  const n = nums.length;
  const out = new Array(n).fill(1);
  let left = 1;
  for (let i = 0; i < n; i++) {
    out[i] = left;
    left *= nums[i];
  }
  let right = 1;
  for (let i = n - 1; i >= 0; i--) {
    out[i] *= right;
    right *= nums[i];
  }
  return out;
}
```

```
[1, 2, 3, 4]
left pass writes [1, 1, 2, 6]
right = 1
i=3: 6 * 1 = 6, right = 4
i=2: 2 * 4 = 8, right = 12
i=1: 1 * 12 = 12, right = 24
i=0: 1 * 24 = 24
[24, 12, 8, 6]
```

The trace is the same two passes on `[1, 2, 3, 4]`. The left pass writes `[1, 1, 2, 6]`. The right pass then produces `[24, 12, 8, 6]`. On both arrays the invariant is the same: `out[i] = left` happens before `left *= nums[i]`, and `out[i] *= right` happens before `right *= nums[i]`.

`left` and `right` both start at 1, the product of an empty side. If you multiply `nums[i]` into `left` first, the cell includes itself. On `[1, 2, 3, 4]` that wrong order returns `[24, 24, 24, 24]`, the full product in every cell. The real answer is `[24, 12, 8, 6]`.

Water from the two side maxima

Water at index `i` is `min(leftMax[i], rightMax[i]) - height[i]`. Walk `[4, 2, 0, 3, 2, 5]`. The left-max array includes the bar at `i`, because each step takes the max of the previous left-max and `height[i]`.

Index 0 sets `leftMax` to 4. Index 1 stays 4, because 2 is shorter. Indexes 2, 3, and 4 stay 4. Index 5 becomes 5, because the last bar is taller. `leftMax` is `[4, 4, 4, 4, 4, 5]`.

The right pass starts at the last bar, which is 5. Every bar to its left is shorter than 5, so `rightMax` is `[5, 5, 5, 5, 5, 5]`. At index 2 the lower side-max is `min(4, 5)`, which is 4, and the bar is 0, so the water is 4. A wall of 4 on the left and a wall of 5 on the right hold that water. The hole cannot see a taller line than the shorter wall.

The six water values are 0, 2, 4, 1, 2, and 0. The total is 9. Indexes 0 and 5 hold 0 because each of those bars is one of the side maxima, and `min(side, side) − height` is 0 on a bar that ties the lower wall. Including `height[i]` inside both max arrays is safe for that reason.

WATER AT I IS THE LOWER OF THE TWO SIDE-MAXES, MINUS THE BAR.

```
function trap(height) {
  const n = height.length;
  if (n === 0) return 0;
  const leftMax = new Array(n);
  const rightMax = new Array(n);
  leftMax[0] = height[0];
  for (let i = 1; i < n; i++) {
    leftMax[i] = Math.max(leftMax[i - 1], height[i]);
  }
  rightMax[n - 1] = height[n - 1];
  for (let i = n - 2; i >= 0; i--) {
    rightMax[i] = Math.max(rightMax[i + 1], height[i]);
  }
  let water = 0;
  for (let i = 0; i < n; i++) {
    const h = Math.min(leftMax[i], rightMax[i]) - height[i];
    if (h > 0) water += h;
  }
  return water;
}
```

```
[2, 0, 2]
leftMax  [2, 2, 2]
rightMax [2, 2, 2]
water    [0, 2, 0]
total 2
```

The trace is the same formula on `[2, 0, 2]`. Both max arrays are `[2, 2, 2]`, the water is `[0, 2, 0]`, and the total is 2. You finish `leftMax` from the left, including `height[i]` at each step. You finish `rightMax` from the right the same way. Only then do you subtract `height[i]` from the lower of the two maxima. If you add the lower maximum and forget to subtract the bar, `[2, 0, 2]` returns 6 instead of 2.

Water with two moving indices

The constant-memory version makes the same choice the container problem made. The side whose maximum is smaller is the bottleneck. Water on that side is already settled, so you add it and you move that index inward. You do not need a future bar on the taller side, because that side is already at least as tall.

Walk `[4, 2, 0, 3, 2, 5]` again. `l` starts at 0 and `r` starts at 5. Both side maxima start at 0. `height[0]` is 4 and `height[5]` is 5, so the left side is the smaller wall. 4 is at least the left maximum, so the left maximum becomes 4 and you move `l` to 1. A bar that sets the maximum holds no water.

Now `height[1]` is 2, which is still below `height[5]`. 2 is below the left maximum 4, so you add `4 − 2`. The water is 2, and `l` moves to 2. Index 2 is 0, still the smaller side. You add `4 − 0`. The water is 6, and `l` moves to 3. Index 3 is 3. You add `4 − 3`. The water is 7, and `l` moves to 4. Index 4 is 2. You add `4 − 2`. The water is 9, and `l` moves to 5.

`l` has met `r`, so you stop. The total is 9, the same total as the two arrays. Each time, you moved the left index because `height[l]` was below `height[r]`. The right wall of 5 was tall enough that no unseen bar on the right could lower the waterline on the left.

MOVE THE SMALLER WALL. THAT SIDE'S WATER IS SETTLED.

```
function trap(height) {
  let l = 0;
  let r = height.length - 1;
  let leftMax = 0;
  let rightMax = 0;
  let water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] >= leftMax) leftMax = height[l];
      else water += leftMax - height[l];
      l += 1;
    } else {
      if (height[r] >= rightMax) rightMax = height[r];
      else water += rightMax - height[r];
      r -= 1;
    }
  }
  return water;
}
```

You compare `height[l]` and `height[r]`, and you move the side that is smaller. On that side, if the bar is at least the side maximum, it becomes the new maximum and you add no water. If the bar is shorter, you add `sideMax - height`. Then you move that index inward. You stop when `l` meets `r`. That last bar is a wall, and the other side has already settled the water next to it. When the two heights are equal, the `else` branch moves `r`. Either side is a legal bottleneck in a tie, and moving one of them keeps the loop moving.

A mountain

A mountain is a strict rise followed by a strict fall. Walk left to right. `up[i]` is how many strict steps you climbed to reach `i`. Walk right to left. `down[i]` is how many strict steps you descend after `i`. A peak is an index where both are positive. Its length is `up[i] + down[i] + 1`. A plateau has a zero on one side and does not count.

Walk `[2, 1, 4, 7, 3, 2, 5]`. The up lengths are 0, 0, 1, 2, 0, 0, 1. Index 3 is 7, and you climbed 1 then 4 then 7, so `up[3]` is 2. Index 4 drops, so its up length is 0. The down lengths are 1, 0, 0, 2, 1, 0, 0. From index 3 you descend 7, 3, 2, so `down[3]` is 2.

Index 3 is the only place where both numbers are positive. The length is `2 + 2 + 1`, which is 5. The mountain is `[1, 4, 7, 3, 2]`. Index 6 rises out of the last valley, and its down length is 0, so it is not a peak.

You fill `up` from the left, and a step counts only when `height[i]` is strictly greater than `height[i - 1]`. You fill `down` from the right, and a step counts only when `height[i]` is strictly greater than `height[i + 1]`. You then scan for an index where both are positive, and you keep the largest `up + down + 1`. On a strictly increasing array such as `[1, 2, 3, 4]`, every `down[i]` is 0, so the best length stays 0.

Candy from both neighbors

Candy gives each child one candy, then extra candies when a neighbor has a lower rating. Both neighbors can force the count. `left[i]` is 1, or `left[i - 1] + 1` when the rating rises as you walk right. `right[i]` does that from the other side. The candies at `i` are the max of the two numbers, because the child must satisfy the higher neighbor on each side. You sum them.

Walk ratings `[1, 3, 2, 1]`. The left pass writes `[1, 2, 1, 1]`. The rise from 1 to 3 makes index 1 a 2. The later descent resets to 1, because a left-only walk has not seen the children further right.

The right pass writes `[1, 3, 2, 1]`. Index 3 is 1. Index 2 has a higher rating than index 3, so it becomes 2. Index 1 has a higher rating than index 2, so it becomes 3. Index 0 does not rise toward the right, so it stays 1. The answer at each index is the max of the two passes: `[1, 3, 2, 1]`. The sum is 7.

You finish the left array before you read it against the right array. A single left-to-right pass on this input sums to 5, from `[1, 2, 1, 1]`. The descent 3, 2, 1 needs 3, 2, 1 candies, and the left pass has not seen that slope yet. The max with the right pass is what repairs index 1 and index 2.

Time and extra memory

Each pass reads every index once, so the time is O(n). The product routine uses O(1) extra memory besides the returned array, and the two max arrays use O(n) extra memory.

Mistakes

- On `[1, 2, 3, 4]`, multiplying `nums[i]` into `left` before you store the answer returns `[24, 24, 24, 24]`. Every cell includes itself. The real answer is `[24, 12, 8, 6]`.
- On `[2, 0, 2]`, adding `min(leftMax, rightMax)` and forgetting to subtract `height[i]` returns 6. The water is `[0, 2, 0]`, and the total is 2.
- On `[1, 2, 3, 4]`, calling a strict rise a mountain returns 4. `down[i]` is 0 at every index, so the best length stays 0.
- On ratings `[1, 3, 2, 1]`, a single left-to-right candy pass returns 5, from `[1, 2, 1, 1]`. The right pass is what raises the descent, and the real sum is 7.

Practice

Stay before chapter 11. Candy is a second pass.

238. Product of Array Except Self MEDIUM STAY
Left products into the answer, right products on the way back. No division.

42. Trapping Rain Water HARD STAY
Write the two max arrays first. Then collapse them to the two-index version if you still have time.

845. Longest Mountain in Array MEDIUM STAY
up[i] and down[i] both positive. Length is up + down + 1. A flat peak is not a mountain.

135. Candy HARD SECOND PASS
Two passes. The answer at i is the max of the left constraint and the right constraint.

REWRITE FROM MEMORY

Rewrite product-except-self in both directions, and say the water formula as the lower side-max minus the bar.

Carry forward. Chapter 11 keeps only the best sum that ends at i. That sum is the prefix ending here minus the worst prefix before it.

CHAPTER 11 · BUILDS ON CHAPTERS 8 AND 10

Kadane

The best subarray that ends here is this cell, or the best ending before it plus this cell. You start from the first cell, and you do not restart from 0.

What this pattern is

You want the maximum sum of one contiguous subarray. A subarray here is a stretch of cells with no gaps, and it must contain at least one cell. The empty subarray has sum 0, and it is not allowed.

Picture a hike whose elevation changes at every step. The best climb that ends at the step you are on is either that step alone, or the best climb that ended on the previous step plus this step. If the trip so far is a loss, this step is a better start than dragging the loss along. You still score the step. You do not score a trip of zero steps.

`bestEnding` is that best climb ending at the current index. It is `max(nums[i], bestEnding + nums[i])`. You start both `bestEnding` and the global best at `nums[0]`. You do not reset a negative `bestEnding` to 0.

What you already know

Chapter 8 writes every prefix. A prefix is the sum of a starting piece. The best subarray that ends at index `j` is `prefix[j + 1]` minus the smallest prefix among `prefix[0..j]`. Kadane keeps that best ending without storing the prefixes. Chapter 10 built two full side arrays when the answer needed both directions. Here one running choice is enough, because the future is not part of the subarray that ends at `i`.

A worked example

Use `[−2, 1, −3, 4, −1, 2, 1]`. You start at index 0. `bestEnding` is −2 and `best` is −2. You do not start at 0. A start of 0 would already be the empty subarray.

At index 1 the value is 1. `bestEnding` becomes `max(1, −2 + 1)`, which is `max(1, −1)`, which is 1. The previous ending −2 hurts, so you take the cell 1 alone. `best` becomes 1.

At index 2 the value is −3. `bestEnding` becomes `max(−3, 1 + −3)`, which is −2. You keep the subarray `[1, −3]`, because −2 is better than −3 alone. `best` stays 1, because −2 is worse than the subarray `[1]`.

At index 3 the value is 4. `bestEnding` becomes `max(4, −2 + 4)`, which is 4. The previous ending is negative, so 4 alone beats extending it. `best` becomes 4. This is a fresh start at the current cell, and the fresh start is `nums[i]`, not 0.

At index 4 the value is −1. `bestEnding` becomes `max(−1, 4 + −1)`, which is 3. You keep `[4, −1]`. `best` stays 4, because 3 is smaller than the single cell 4.

At index 5 the value is 2. `bestEnding` becomes `max(2, 3 + 2)`, which is 5. You extend to `[4, −1, 2]`, and `best` becomes 5. At index 6 the value is 1. `bestEnding` becomes `max(1, 5 + 1)`, which is 6. `best` becomes 6. The subarray is `[4, −1, 2, 1]`.

The answer is 6. You never restarted from 0. On the smaller input `[−2, −1]`, the same rule starts at −2, then takes `max(−1, −2 + −1)`, which is −1. The answer is −1. A reset to 0 would return 0, and 0 is the empty subarray, which is illegal.

How you recognise it

- You want the maximum sum of one contiguous subarray, and the empty subarray is not allowed.
- The values may be negative, so at some indexes the best ending is the current cell alone.
- You are discarding a negative piece that ends just before i, and you are not holding a sum inside a limit.

- A maximum product is the same choice, plus the minimum product, because a negative factor flips the two.
- A circular array is one ordinary run and one total-minus-minimum run, and you reject the wrap when it would be empty.

Some prompts look close and belong in another chapter.

- Use chapter 7 when the values are non-negative and the sum is held under a limit.
- Use chapter 8 when you must count the subarrays whose sum is k, including arrays that contain negatives.
- Use chapter 15 when the sum is bounded by k and the values may be negative, because that needs a deque of prefixes.

What to say out loud

An invariant is a fact you keep true at every index. `bestEnding` is the best non-empty subarray that ends exactly at `i`.

`best` is the best of those endings over every index you have seen. The update is `bestEnding = max(nums[i], bestEnding + nums[i])`. It never restarts from 0.

The template

START AT NUMS[0]. NEVER RESTART FROM 0.

```
function maxSubArray(nums) {
  let bestEnding = nums[0];
  let best = nums[0];
  for (let i = 1; i < nums.length; i++) {
    bestEnding = Math.max(nums[i], bestEnding + nums[i]);
    if (bestEnding > best) best = bestEnding;
  }
  return best;
}
```

```
[-2, -1]
bestEnding = -2, best = -2
i=1: max(-1, -2 + -1) = -1
best = -1
a reset to 0 would return 0, and 0 is the empty subarray, which is illegal

[1, -2, 3]
i=1: max(-2, 1 + -2) = -1, best stays 1
i=2: max(3, -1 + 3) = 3, best = 3
```

`bestEnding` and `best` both start at `nums[0]`. The loop starts at index 1, so the first cell is not added twice. The new `bestEnding` uses the old `bestEnding` in `bestEnding + nums[i]`, and it uses `nums[i]` alone as the other choice. You update `best` after that assignment.

You do not reset a negative `bestEnding` to 0. On `[-2, -1]` the reset returns 0. The trace shows the legal answer, −1. That 0 would be the empty subarray, and the problem requires a non-empty one. Starting at `nums[0]` is what keeps an all-negative array honest.

The maximum product

A negative multiplier flips the largest product and the smallest product. You keep both. Walk `[2, 3, -2, 4]`. You start with `mx = 2`, `mn = 2`, and `best = 2`.

At 3, the candidates for the max are 3, `2 * 3`, and `2 * 3`. The max is 6 and the min is 3. `best` becomes 6. At −2, the candidates are −2, `6 * -2` which is −12, and `3 * -2` which is −6. You compute both temps from the old pair.

`nextMx` is −2 and `nextMn` is −12. Then you assign. `best` stays 6.

At 4, the candidates are 4, `-2 * 4` which is −8, and `-12 * 4` which is −48. `nextMx` is 4 and `nextMn` is −48. `best` stays 6. The flip at −2 used the old max and the old min together. The new minimum needed the old maximum, because `6 * -2` is the most negative candidate.

If you overwrite `mx` first, that flip is gone. On the longer input `[2, 3, -2, -1]` the correct answer is 12, the product of the whole array. After −2, the true minimum is −12. The wrong order stores −6, because it multiplies the new max, which is already −2, instead of the old max 6. The last −1 then produces 6, and the function returns 6. It never sees `-12 * -1`.

TEMPS, BECAUSE THE NEW MINIMUM NEEDS THE OLD MAXIMUM.

```
function maxProduct(nums) {
  let mx = nums[0];
  let mn = nums[0];
  let best = nums[0];
  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    const nextMx = Math.max(x, mx * x, mn * x);
    const nextMn = Math.min(x, mx * x, mn * x);
    mx = nextMx;
    mn = nextMn;
    if (mx > best) best = mx;
  }
  return best;
}
```

```
[2, 3, -2, 4]
start mx 2, mn 2, best 2
x=3: mx 6, mn 3, best 6
x=-2: candidates  -2, 6*-2=-12, 3*-2=-6
      mx = -2, mn = -12, best stays 6
x=4: candidates 4, -2*4=-8, -12*4=-48
      mx = 4, mn = -48, best stays 6
the flip at -2 used the old mn and the old mx together
```

You read `x`. You compute `nextMx` and `nextMn` from `x`, from the old `mx * x`, and from the old `mn * x`. You assign `mx` and `mn` only after both temps exist. If you write `mx = Math.max(x, mx * x, mn * x)` and then compute the min from that new `mx`, the old `mx * x` is no longer available. `best` is updated from the new `mx`, after the pair has been stored.

The circular case

A circular subarray is either ordinary, or it wraps around the ends. The wrap is the total minus a non-wrapping middle that you throw away. The middle you throw away is the minimum subarray. If every value is negative, the minimum subarray is the whole array, the wrap is empty, and `total - minBest` is 0. You return the ordinary maximum instead. The test is `maxBest < 0`.

Walk `[5, -3, 5]`. You start with every running value at 5, and the total at 5. At −3, the max ending is `max(-3, 5 + -3)`, which is 2. `maxBest` stays 5. The min ending is `min(-3, 2)`, which is −3. `minBest` becomes −3. The total is 2.

At the last 5, the max ending is `max(5, 2 + 5)`, which is 7. `maxBest` becomes 7. The min ending is `min(5, -3 + 5)`, which is 2. `minBest` stays −3. The total is 7. 7 is not below 0, so you return the larger of 7 and `7 - (-3)`. That is 10. The wrap is the two 5s, and the thrown-away middle is the −3.

Now walk `[-3, -2, -1]`. `maxBest` ends at −1, the largest single cell. `minBest` is −6, the whole array, and the total is −6. `total - minBest` is 0. `maxBest` is below 0, so you return −1. The 0 is the empty wrap. You must

not return it.

IF THE BEST ORDINARY SUM IS NEGATIVE, EVERY WRAP IS WORSE OR EMPTY.

```
function maxSubarraySumCircular(nums) {
  let maxEnding = nums[0];
  let maxBest = nums[0];
  let minEnding = nums[0];
  let minBest = nums[0];
  let total = nums[0];
  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    maxEnding = Math.max(x, maxEnding + x);
    if (maxEnding > maxBest) maxBest = maxEnding;
    minEnding = Math.min(x, minEnding + x);
    if (minEnding < minBest) minBest = minEnding;
    total += x;
  }
  if (maxBest < 0) return maxBest;
  return Math.max(maxBest, total - minBest);
}
```

```
[5, -3, 5]
ordinary best = 7? let's compute:
maxEnding: 5, then max(-3, 2) = 2, then max(5, 7) = 7. maxBest = 7
minEnding: 5, then min(-3, 2) = -3, then min(5, 2) = 2. minBest = -3
total = 7
7 is not negative, so max(7, 7 - (-3)) = 10
the wrap is the two 5s

[-3, -2, -1]
maxBest = -1
minBest = -6, total = -6, total - minBest = 0
maxBest < 0, return -1, not the empty wrap
```

One loop updates the max ending, the min ending, and the total. Each ending is the max or the min of `x` alone and the previous ending plus `x`, and both endings start at `nums[0]`. You update the best max and the best min after those assignments, and you add `x` into the total on the same iteration.

After the loop, if `maxBest` is below 0 you return `maxBest`. Every value is negative, the minimum subarray is the whole array, and `total - minBest` is 0, the empty wrap. Otherwise you return the larger of `maxBest` and `total - minBest`. On `[-3, -2, -1]` the second expression is 0, and the function must return −1.

Every rise is a sale

Best time to buy and sell stock II allows as many transactions as you want, and they must not overlap. The profit is the sum of every positive difference `prices[i] - prices[i - 1]`. Any climb from a valley to a peak equals the sum of the daily rises along it, so you do not need to find the valleys by hand.

Walk `[7, 1, 5, 3, 6, 4]`. The difference from 7 to 1 is −6, and you skip it. From 1 to 5 the difference is 4, and you add 4. From 5 to 3 the difference is −2, and you skip it. From 3 to 6 the difference is 3, and you add 3. From 6 to 4 the difference is −2, and you skip it. The profit is 7. That is the same as buying at 1 and selling at 5, then buying at 3 and selling at 6.

You walk from index 1. When today's price is above yesterday's, you add the difference. When it is not, you add nothing. You do not keep a running minimum the way the one-transaction problem does. Each rise is its own sale.

States that are not this loop

Stock III allows at most two transactions. Cooldown says that after a sale you skip a day. The transaction fee is paid when you sell. Those three are states, not Kadane. A state is holding or not, plus how many buys you have used, plus whether yesterday was a sale. Do not force them into `bestEnding`.

The problem numbers to link, and to leave for later, are 123, 309, and 714. You can name the states now. For cooldown they are hold, sold, and rest. For the fee they are hold versus cash. For two transactions they track how many buys are still available. The update is a transition between those states, not `max(nums[i], bestEnding + nums[i])`.

Time and extra memory

One pass updates a constant number of variables, so the time is O(n) and the extra memory is O(1).

Mistakes

- On `[-2, -1]`, resetting a negative `bestEnding` to 0 returns 0. The legal answer is −1. The 0 is the empty subarray, and the empty subarray is not allowed.
- On `[2, 3, -2, -1]`, writing `mx` before `mn` returns 6. The min after −2 is stored as −6 instead of −12, so the last −1 cannot flip −12 into 12. The real answer is 12.
- On `[-3, -2, -1]`, returning `total - minBest` with no test returns 0. That 0 is the empty wrap. `maxBest` is −1, and because it is below 0 the function must return −1.
- On a sum that must stay at most k, Kadane returns the largest sum it can find, with no regard for the limit. Non-negative values under a limit belong to chapter 7. Negatives under a limit belong to the deque of prefixes in chapter 15.

Practice

Stay before chapter 12. The circular problem is Then. The three state problems are later, on a second pass.

53. Maximum Subarray MEDIUM STAY
bestEnding is nums[i] or bestEnding + nums[i]. Start at nums[0].

152. Maximum Product Subarray MEDIUM STAY
Track min and max. Compute both temps from the old pair, then assign.

122. Best Time to Buy and Sell Stock II MEDIUM STAY
Add every positive daily rise. Unlimited non-overlapping transactions.

918. Maximum Sum Circular Subarray MEDIUM THEN
max(ordinary, total − min subarray). If ordinary is negative, return it.

123. Best Time to Buy and Sell Stock III HARD SECOND PASS
At most two transactions. This is a state machine, not Kadane.

309. Best Time to Buy and Sell Stock with Cooldown MEDIUM SECOND PASS
After a sale you must skip a day. States: hold, sold, rest.

714. Best Time to Buy and Sell Stock with Transaction Fee MEDIUM SECOND PASS
A fee on each sale. Hold versus cash, and the fee is paid when you sell.

REWRITE FROM MEMORY
Rewrite Kadane on [-2, -1], the product temps, and the circular case that rejects the empty wrap.

Carry forward. Chapter 12 pays n log n for an order, then runs the chapter 5 scan on pairs and on intervals.

CHAPTER 12 · BUILDS ON CHAPTERS 4 AND 5

Sort, then scan

You sort so the values sit in a known order. A scan from the ends can then throw away a whole side.

What this pattern is

An **interval** is a closed span from a start to an end. Both ends are inside the span. If the next start lands on the previous end, the two spans share that point.

A map answers one question: have you seen this value? A sort answers a different one. You line the values up. Then a scan from the two ends can throw an end away, because everything past that end is too big or too small.

You pay for the sort once. After that, the scan is a straight walk. For three values that sum to zero, the walk is slower than a single pass, and that slower cost is the accepted one.

Picture people lined up from lightest to heaviest. You point at the lightest and the heaviest and decide who shares a boat. You do not search the line from scratch for every person. The same picture covers spans on a number line: once they are ordered by a start or by an end, you only compare a span with the one you are already holding.

What you already know

Chapter 4 stores a value so you can ask whether you have seen it. Chapter 5 walks the two ends of a line that is already ordered. This chapter builds that order when you do not have it. You sort, you name what the order means, and then you run the end scan you already know. A sum that is too small moves the left end. A sum that is too large moves the right end.

A worked example

Take `[-1, 0, 1, 2, -1, -4]`. You want every triple of values that sums to 0, and you want each triple once.

Sort with the numeric compare `(a, b) => a - b`. The line becomes `[-4, -1, -1, 0, 1, 2]`. A default compare can order 10 before 2, so the numeric compare is part of the algorithm.

Fix index 0. The value is -4, so the other two values must sum to 4. Put `l` on index 1 and `r` on the last index. The pair is -1 and 2. Their sum is 1, which is less than 4, so you move `l` right. You need a larger pair, and the array is sorted, so the right side of `l` is where larger values live. You keep moving `l` until the two indexes meet. No pair sums to 4. You record nothing for -4.

Fix index 1. The value is -1, so the other two must sum to 1. `l` starts on index 2, still -1. `r` starts on the last index, value 2. The three values sum to 0, so you record `[-1, -1, 2]`.

You then move both ends. `l` goes to index 3 and `r` goes to index 4. If you moved only one end, the next check would see the same pair again. Those new values are 0 and 1. Together with -1 they also sum to 0, so you record `[-1, 0, 1]`. The indexes then cross, and this fixed position is finished.

After a hit you move both ends first, then you skip a repeated `l` and a repeated `r`. Here the new ends are 0 and 1, so those skips do not move. The order still matters. You compare an end with the neighbor you just left, which is a real duplicate, rather than with itself.

Index 2 holds -1, the same value as index 1. The test is `i > 0` and `nums[i] === nums[i - 1]`. It is true, so you skip index 2. If you did not, you would record the same two triples again.

Index 3 holds 0. Everything to its right is positive, so every sum is positive. You record nothing. Index 4 holds 1, which is greater than 0. You stop the outer loop. The array is sorted and the target is 0, so every later triple is positive too. The answer is `[[-1, -1, 2], [-1, 0, 1]]`.

```
[-1, 0, 1, 2, -1, -4]
sorted [-4, -1, -1, 0, 1, 2]
i = -4: no pair in the suffix sums to 4
i = -1 at index 1
  l = -1, r = 2, sum 0 → emit [-1, -1, 2]
  l and r move to 0 and 1, sum 0 → emit [-1, 0, 1]
i = -1 at index 2, same as previous i, skip
i = 0: the suffix is positive, sums stay positive
i > 0 would break
answer [[-1, -1, 2], [-1, 0, 1]]
```

How you recognise it

- The problem asks for pairs or triples that sum to a target, and you are allowed to reorder the array.
- The input is a list of intervals, and you must merge them, insert one, drop overlaps, shoot arrows, or intersect two lists.
- You pair the lightest person with the heaviest, or you fix the largest side of a triangle and count pairs that can form it.
- The array is already sorted, and you only walk runs of values that increase by one. Summary ranges is that walk, and you do not sort it yourself.
- After one sort, a fact about an end lets you discard a whole side of the remaining line.
- Duplicate values must not create duplicate answers, so the scan skips copies on purpose.
- Use chapter 4 when the only question is whether a complement exists, and you must keep the original indexes.
- Use chapter 5 when the array is already sorted and two ends are enough, with no extra fixed index and no interval rule.
- Use chapter 11 when you want the best contiguous subarray sum. Sorting would destroy that subarray.

What to say out loud

You sort, then you name the order you just created. You say which duplicate skip keeps each answer unique. For intervals, you say whether you sorted by the start or by the end, because those two orders answer different questions.

The template

SKIP DUPLICATE I, AND SKIP BOTH ENDS AFTER A HIT.

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const out = [];
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 0) break;
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    let l = i + 1;
    let r = nums.length - 1;
    while (l < r) {
      const sum = nums[i] + nums[l] + nums[r];
      if (sum === 0) {
        out.push([nums[i], nums[l], nums[r]]);
        l += 1;
        r -= 1;
        while (l < r && nums[l] === nums[l - 1]) l += 1;
        while (l < r && nums[r] === nums[r + 1]) r -= 1;
      } else if (sum < 0) {
        l += 1;
      } else {
        r -= 1;
      }
```

```
    }
  }
  return out;
}
```

An **invariant** is a fact that stays true every time the loop comes back to check it. Here the array is sorted, index `i` is the left value of the triple, and every index before `i` is already finished. `l` and `r` sit strictly to the right of `i`, and you only read them while `l` is still left of `r`.

The off-by-one risk is the duplicate skip. You skip a repeated `i` only when `i > 0` and `nums[i]` equals `nums[i - 1]`. At index 0 there is no previous value. After a hit, you move both ends, and only then skip a repeated `l` and a repeated `r`. The left skip looks at `nums[l - 1]`. The right skip looks at `nums[r + 1]`. Both stops also require `l < r`, so you never walk the ends past each other.

Update order on a hit: push the triple, add 1 to `l`, subtract 1 from `r`, then run the two skip loops. A sum that is too small adds 1 to `l` only. A sum that is too large subtracts 1 from `r` only. You break out of the `i` loop only when `nums[i] > 0`, and only because the target is 0. Leave that break off 4Sum. A 4Sum target can be negative, so a positive `nums[i]` can still belong in a valid four.

Merge, and when a touch counts

Take `[[1, 3], [2, 6], [8, 10], [15, 18]]`. Sort by start. This list is already in that order. You hold `[1, 3]`. The next start is 2. 2 is not greater than 3, so this is the same block. The new end 6 sticks out past 3, so the held end becomes 6. The next start is 8, and 8 is greater than 6, so you push a new piece `[8, 10]`. The next start is 15, and 15 is greater than 10, so you push `[15, 18]`. The result is `[[1, 6], [8, 10], [15, 18]]`.

A touch merges, because the intervals are closed. `[[1, 4], [4, 5]]` becomes `[1, 5]`. The new start equals the last end, so you do not open a new piece. A new piece starts only when the start is greater than the last end. A shorter interval that sits inside a longer one leaves the last end alone: end 4 against a held end of 6 does not shrink the 6. Meeting Rooms is this sort plus one question: after sorting, does any start fall before the previous end? You do not need that problem. The sentence is the whole variant.

OVERLAP TEST FOR CLOSED INTERVALS: START <= LAST END. THE CODE USES THE COMPLEMENTARY TEST, START > LAST END, TO OPEN A NEW INTERVAL.

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const out = [];
  for (const [start, end] of intervals) {
    const last = out[out.length - 1];
    if (out.length === 0 || start > last[1]) out.push([start, end]);
    else if (end > last[1]) last[1] = end;
  }
  return out;
}
```

```
[[1, 3], [2, 6], [8, 10], [15, 18]]
already sorted
[1, 3] starts the output
2 <= 3, extend end to 6
8 > 6, push [8, 10]
15 > 10, push [15, 18]
[[1, 6], [8, 10], [15, 18]]
```

Update order: sort by the start before you scan. If the output is empty, or the start is greater than the last end, push a new pair. Otherwise, if this end is greater than the last end, write that larger end into the last pair. You do not push on an overlap, and you do not shrink an end. The empty check has to come first, because there is no last pair to read on the first interval.

Insert: copy, fold, copy

You can say the short version first. Push the new interval into the list, sort, and reuse merge. The linear version does not sort. The input is already sorted, and the old intervals do not overlap each other.

Take `[[1, 3], [6, 9]]` and insert `[2, 5]`. Copy every interval that ends before the new start. The test is strict: `end < start`. `[1, 3]` ends at 3, which is not less than 2, so you do not copy it. A touch, where the old end equals the new start, must be folded.

Fold while the next start is less than or equal to the current end. `[1, 3]` starts at 1, and 1 is within the end 5. The folded start becomes the smaller of 2 and 1, which is 1. The folded end becomes the larger of 5 and 3, which is 5. The next interval `[6, 9]` starts at 6, which is past 5, so the fold stops. Push `[1, 5]`, then copy the tail. The result is `[[1, 5], [6, 9]]`.

A second walk shows the copy and the touch. Insert `[4, 6]` into `[[1, 3], [6, 9]]`. `[1, 3]` ends at 3, and 3 is less than 4, so you copy it. `[6, 9]` starts at 6, which equals the current end, so you fold. The start stays 4. The end becomes 9. The result is `[[1, 3], [4, 9]]`.

COPY, FOLD, COPY. TOUCHING MERGES, SO THE TESTS ARE STRICT ON THE LEFT AND CLOSED ON THE FOLD.

```
function insert(intervals, newInterval) {
  const out = [];
  let [start, end] = newInterval;
  let i = 0;
  while (i < intervals.length && intervals[i][1] < start) {
    out.push(intervals[i]);
    i += 1;
  }
  while (i < intervals.length && intervals[i][0] <= end) {
    start = Math.min(start, intervals[i][0]);
    end = Math.max(end, intervals[i][1]);
    i += 1;
  }
  out.push([start, end]);
  while (i < intervals.length) {
    out.push(intervals[i]);
    i += 1;
  }
  return out;
}
```

Update order: copy while the old end is strictly before the new start, and advance `i` as you copy. Then fold while the next start is less than or equal to the current end. On each fold, take the min start and the max end before you advance `i`. Push the folded pair once. Then copy whatever index remains. If you push before the fold finishes, you emit a piece that the next interval still needs to extend.

Closest triple, and 4Sum

3Sum closest uses the same loops and does not collect hits. You keep the sum whose distance from the target is smallest. Take `[−1, 2, 1, −4]` and target 1. Sorted, the line is `[−4, −1, 1, 2]`. Fix -4. `l` is -1 and `r` is 2. The sum is -3. The distance to 1 is 4, so -3 is the best so far. The sum is too small, so you move `l`. The next sum is `−4 + 1 + 2`, which is -1. The distance is 2, which is better. The pointers then meet. Fix -1. The pair 1 and 2 sums with -1 to 2. The distance to 1 is 1, which is better still. The sum is too large, so you move `r`, and the pointers meet. The closest sum is 2.

4Sum adds one more fixed index. The duplicate skips stay on every index you fix, and on both ends after a hit. You may skip an `i` when the smallest possible four from here is already above the target, and you may skip an `i` when the largest possible four is still below the target. Those prunes are optional. The duplicate skips are not. Do

not break because a value is positive. If the target is -10, a positive number can still sit in a four whose other three values are negative enough to hit -10. The 3Sum break would drop that four.

Boats

Sort the people. While `l` is still at or left of `r`, the heaviest remaining person takes a boat. The lightest joins when the two fit. One person, the case `l === r`, still takes one boat. The problem promises nobody is heavier than the limit.

Take weights `[3, 2, 2, 1]` and limit 3. Sorted, the line is `[1, 2, 2, 3]`. `l` is on 1 and `r` is on 3. The sum is 4, which does not fit, so the heavy person goes alone. Move `r` left. Now 1 and 2 sum to 3, so they share, and both ends move. The two ends meet on the last 2. That one person takes a boat. Three boats.

Triangles

A triangle needs the two smaller sides to sum to more than the largest side. Sort, fix the largest side at index `k`, and set `l` and `r` inside the prefix to the left of `k`.

Take `[2, 2, 3, 4]`. Fix 4 at the last index. `l` starts on the first 2 and `r` starts on 3. `2 + 3` is 5, which is greater than 4. Every index from `l` through `r - 1` also works with this `r`, because those values are at least `nums[l]`. There are `r - l` such indexes, which is 2. Add 2, then move `r` left. Now the pair is 2 and 2. `2 + 2` is 4, which is not greater than 4, so you move `l` right. The pointers meet. Fix the next side, 3. The prefix pair is 2 and 2, and `2 + 2` is greater than 3, so you add `r - l`, which is 1. The total is 3. The triples are the two ways to choose a 2 with 3 and 4, plus the two 2s with 3.

Non-overlapping intervals

Sort by the end, not by the start. Keep an interval when its start is greater than or equal to the end you last kept. Touching is allowed. `[1, 2]` and `[2, 3]` both stay, because this problem does not treat a shared endpoint as overlap. Merge did. The answer is how many you erase, which is `n` minus the number you kept.

Take `[[1, 2], [2, 3], [3, 4], [1, 3]]`. Sorted by end, the order is `[1, 2]`, `[2, 3]`, `[1, 3]`, `[3, 4]`. Keep `[1, 2]`. The next start is 2, which equals the kept end, so keep `[2, 3]`. The next start is 1, which is behind 3, so erase `[1, 3]`. The next start is 3, which equals the kept end, so keep `[3, 4]`. You kept 3 out of 4. The answer is 1.

The interval that ends first leaves the most room. On `[[1, 100], [11, 22], [1, 11], [2, 12]]`, sorting by end keeps `[1, 11]` and `[11, 22]` and erases the other two. Sorting by start and keeping the earliest start holds `[1, 100]` and erases the three short ones. That erases more than the minimum.

Arrows

Sort by the end again. You need a new arrow only when the next balloon starts strictly after the current arrow's end. Touching balloons share an arrow. Non-overlapping keeps a piece when the start is at least the last kept end. Arrows spend a new arrow only when the start is strictly after the current end. A touch stays as two pieces in the erase problem. A touch shares one arrow here.

Take `[[10, 16], [2, 8], [1, 6], [7, 12]]`. Sorted by end, the order is `[1, 6]`, `[2, 8]`, `[7, 12]`, `[10, 16]`. The first arrow ends at 6. The next start is 2, which is not after 6, so it shares that arrow. The next start is 7, which is after 6, so you need a new arrow, and it ends at 12. The next start is 10, which is not after 12, so it shares. Two arrows.

Two interval lists

You have two lists that are already sorted. Keep one index on each. If the two current intervals overlap, the overlap runs from the larger start to the smaller end. Then advance the list whose current interval ends first. The

other list may still overlap the next one.

Take `A = [[0, 2], [5, 10]]` and `B = [[1, 5], [8, 12]]`. The larger start is 1 and the smaller end is 2, so you record `[1, 2]`. A ends first, so you advance A. Next, `[5, 10]` against `[1, 5]` overlaps at `[5, 5]`. B ends first, so you advance B. Next, `[5, 10]` against `[8, 12]` overlaps at `[8, 10]`. A ends first. The result is `[[1, 2], [5, 5], [8, 10]]`.

Summary ranges

You do not sort. The input is already sorted, and every value is unique. Walk once. While the next value is exactly one more than the current value, the range stays open. When the next value is anything else, close the range you were building.

Take `[0, 1, 2, 4, 5, 7]`. 1 is 0 plus 1, and 2 is 1 plus 1, so the open range runs from 0 to 2. 4 is not 3, so you close `0->2`. 5 is 4 plus 1. 7 is not 6, so you close `4->5`. The last value closes as `7`. A range of one number is that number alone.

Time and extra memory

3Sum is O(n^2) time after the O(n log n) sort. Extra memory is O(1) besides the output, if the sort is in place. Merge, insert, boats, triangles, arrows, and the erase scan are a sort plus a linear pass, or a linear pass alone when the input is already ordered. Summary ranges and the two-list intersection do not sort.

Mistakes

- On interval starts [10, 2], a default compare can place 10 before 2. You then merge or pair the wrong neighbors. Sort numbers with a numeric compare, and sort intervals by a start or by an end.
- On [-1, 0, 1, 2, -1, -4], if you skip only a repeated i, a repeated end can still emit the same triple twice from one i. On [-2, 0, 0, 2, 2], the hit [-2, 0, 2] is printed again unless you skip the repeated l and the repeated r after both ends move.
- On a 4Sum whose target is negative, breaking when nums[i] is greater than 0 drops fours that use a positive value and still hit the target. That break belongs to 3Sum, where the target is 0.
- On [[1, 4], [4, 5]], merging only when the start is strictly before the last end leaves two pieces. Closed intervals share the endpoint 4, so the result is [1, 5].
- On [[1, 100], [11, 22], [1, 11], [2, 12]], sorting by start and keeping the earliest start holds [1, 100] and erases three short intervals. Sorting by end erases two. The earliest ending interval is the one that leaves the most room.
- On balloons [[1, 2], [2, 3]], treating a start that equals the current end as a new arrow shoots twice. Those balloons share the point 2, so one arrow is enough.

Practice

Stay before chapter 13. Then before you leave the chapter.

15. 3Sum MEDIUM STAY
Sort, skip duplicate i, skip both ends after a hit, break once nums[i] > 0.

56. Merge Intervals MEDIUM STAY
Sort by start. A new piece starts only when start > last end.

881. Boats to Save People MEDIUM STAY
Sort. Pair lightest with heaviest while l <= r. Always send the heavy person.

228. Summary Ranges EASY STAY
Already sorted. Close a range when the next value is not current + 1.

16. 3Sum Closest MEDIUM THEN
The same loops. Keep the sum closest to the target instead of the zeros.

18. 4Sum MEDIUM THEN
One more loop, the same duplicate skips. Do not break just because a value is positive.

611. Valid Triangle Number MEDIUM THEN
Fix the largest side. When l and r fit, count r − l and move r.

57. Insert Interval MEDIUM THEN
Say "push and reuse merge" first. Then write the copy, fold, copy pass.

435. Non-overlapping Intervals MEDIUM THEN
Sort by end. Keep a start that is >= the last kept end. Touching is allowed.

452. Minimum Number of Arrows to Burst Balloons MEDIUM THEN
Sort by end. A new arrow only when start > current end. Touching shares.

986. Interval List Intersections MEDIUM THEN
Two indices. Overlap is [max starts, min ends]. Advance the interval that ends first.

REWRITE FROM MEMORY
Rewrite 3Sum from memory, including both duplicate skips, and the merge test that opens a new interval only when the start is past the last end.

Carry forward. Chapter 13 does not sort. The value itself tells you the index where it belongs.

CHAPTER 13 · BUILDS ON CHAPTERS 6 AND 12

Place each value where it belongs

A value v in 1..n wants index v - 1. You swap it home, and you do not step past a value you have not read.

What this pattern is

The values are already small whole numbers. They are not weights you need to sort. A value `v` in `1..n` wants the seat at index `v - 1`. A value in `0` through `n - 1` wants the seat at index `v`. You swap until each value sits at home, or until it cannot, because a copy is already there.

A **half-open range** includes its left end and stops before its right end. You will use that shape for regions. `[0, lo)` means index 0 up to, but not including, `lo`.

Picture numbered lockers. You pick up a tag and carry it to the locker with that number. You take whatever was there and repeat. You stop on a locker that already holds the same number. Sometimes the array holds only 0, 1, and 2. Then you do not send each value to a home index. You grow three regions from the outside in.

What you already know

Chapter 6 reads a cell and writes a cell in one pass. Chapter 12 sorts when you have to build an order. Here you do not sort. The value itself is the index. A swap is the whole move. If the problem forbids writes, the same home index becomes a step in a cycle, and you follow the cycle instead of swapping.

A worked example

Sort the colors in `[2, 0, 1, 2, 1]`. You keep three regions, and you read one unseen cell at a time.

`[0, lo)` holds 0. `[lo, mid)` holds 1. `(hi, n)` holds 2, meaning every index strictly after `hi`. `[mid, hi]` is still unseen. At the start, `lo` and `mid` are 0 and `hi` is 4, so the whole array is unseen.

Read `mid`, index 0. The value is 2. A 2 swaps with `hi`, index 4, which holds 1. The array becomes `[1, 0, 1, 2, 2]`. `hi` moves to 3. `mid` stays at 0. The 1 that arrived from the right has not been read.

The shorter input `[2, 0, 1]` is the same first swap. It becomes `[1, 0, 2]`, and `hi` becomes 1. If you also moved `mid` to 1, you would skip the 1 now sitting at index 0. That 1 arrived unseen, so the next check has to read it.

Back on the five values, read index 0 again. The value is 1. A 1 belongs in the middle region. Advance `mid` to 1. You do not swap.

Read index 1. The value is 0. A 0 swaps with `lo`. `lo` is still 0, so index 0 and index 1 trade. The array becomes `[0, 1, 1, 2, 2]`. The value that came from `lo` was a 1 you have already accepted. Advance both `lo` and `mid`. `lo` becomes 1 and `mid` becomes 2.

Read index 2. The value is 1, so advance `mid` to 3. Read index 3. The value is 2, and `hi` is also 3, so the swap is the cell with itself. `hi` moves to 2. `mid` stays at 3. A swap with `hi` does not advance `mid`, even when both indexes are the same cell.

`mid` is now past `hi`. The unseen span is empty. You stop. The array is `[0, 1, 1, 2, 2]`.

```
[2, 0, 1, 2, 1]
mid=0 sees 2, swap with hi → [1, 0, 1, 2, 2], hi = 3, mid stays 0
mid=0 sees 1, mid = 1
mid=1 sees 0, swap with lo → [0, 1, 1, 2, 2], lo = 1, mid = 2
mid=2 sees 1, mid = 3
mid=3 sees 2, swap with hi (same cell), hi = 2, mid stays 3
mid > hi, stop
on [2, 0, 1], advancing mid after the hi swap skips the 1
```

How you recognise it

- The values lie in 1..n or in 0..n, and the question is a missing number, a duplicate, or the first missing positive.
- You must place zeros, then ones, then twos, in one pass, with a fixed amount of extra memory.
- You are allowed to modify the array. If the problem forbids writes, the duplicate is a cycle in the step from i to nums[i].
- One value appears once, and every other value appears twice or three times. XOR, or a count of bits, replaces the placement.
- The home of a value is an index you can compute, so a search for that home is wasted work.
- Use chapter 12 when the values are not legal indexes and a sort, then a scan, is the tool.
- Use chapter 6 when you overwrite cells from a count array, and you do not swap a value onto a home index.
- Use chapter 14 when the array is sorted and you throw away half of an index range.

What to say out loud

Values that already sit at their home index stay there. A swap either moves some value onto its home, or it tells you a duplicate is already home. After you swap with the high end of a three-color array, mid stays put, because the new middle value is still unseen.

The template

AFTER A SWAP WITH HI, MID STAYS. THE NEW NUMS[MID] IS UNPROCESSED.

```
function sortColors(nums) {
  let lo = 0;
  let mid = 0;
  let hi = nums.length - 1;
  while (mid <= hi) {
    if (nums[mid] === 0) {
      const tmp = nums[lo];
      nums[lo] = nums[mid];
      nums[mid] = tmp;
      lo += 1;
      mid += 1;
    } else if (nums[mid] === 1) {
      mid += 1;
    } else {
      const tmp = nums[mid];
      nums[mid] = nums[hi];
      nums[hi] = tmp;
      hi -= 1;
    }
  }
  return nums;
}
```

The invariant: `[0, lo)` is 0, `[lo, mid)` is 1, `(hi, n)` is 2, and `[mid, hi]` is unseen. You have not read the unseen span, so a value inside it can be any of the three colors.

The off-by-one is the loop test. `mid` runs while it is less than or equal to `hi`, because the cell at `hi` is still unseen. Stopping when `mid` merely equals `hi` would leave that last cell unread.

Update order: a 0 swaps with `lo`, then both `lo` and `mid` advance. A 1 advances only `mid`. A 2 swaps with `hi`, `hi` decreases, and `mid` does not move. On `[2, 0, 1]`, moving `mid` after the high swap skips the 1 that just arrived at index 0.

Cyclic placement

A value `v` in `1..n` wants index `v - 1`. You swap while `nums[dest]` is not already `v`. If you swap when the two cells are already equal, they trade the same value forever.

Take `[3, 4, -1, 1]`. The length is 4, so a useful value is in 1 through 4. Index 0 holds 3, which wants index 2. Swap. The array is `[-1, 4, 3, 1]`. Index 0 now holds -1, which is outside 1 through 4, so you advance. Index 1 holds 4, which wants index 3. Swap. The array is `[-1, 1, 3, 4]`. Index 1 now holds 1, which wants index 0, and index 0 holds -1, not 1. Swap. The array is `[1, -1, 3, 4]`. Index 0 now holds 1, which is home, so you advance. Index 1 holds -1, so you advance. Index 2 holds 3, which is home. Index 3 holds 4, which is home.

Scan. Index 1 holds -1, not 2, so the first missing positive is 2. If every index `j` held `j + 1`, the answer would be `n + 1`.

IGNORE VALUES OUTSIDE 1..N. V WANTS INDEX V − 1.

```
function firstMissingPositive(nums) {
  const n = nums.length;
  let i = 0;
  while (i < n) {
    const v = nums[i];
    const dest = v - 1;
    if (v >= 1 && v <= n && nums[dest] !== v) {
      const tmp = nums[i];
      nums[i] = nums[dest];
      nums[dest] = tmp;
    } else {
      i += 1;
    }
  }
  for (let j = 0; j < n; j++) {
    if (nums[j] !== j + 1) return j + 1;
  }
  return n + 1;
}
```

```
[3, 4, -1, 1]
3 belongs at index 2: [-1, 4, 3, 1]
-1 is outside 1..4, skip
4 belongs at index 3: [-1, 1, 3, 4]
1 belongs at index 0: [1, -1, 3, 4]
scan: index 1 holds -1, not 2, so the answer is 2
```

Update order: read `v`, set `dest` to `v - 1`, and swap only when `v` is inside 1 through `n` and `nums[dest]` is not `v`. Otherwise add 1 to `i`. On a successful swap you do not add 1 to `i`, because the value that arrived still needs a home. The inequality is also the duplicate test. If writes are allowed and `nums[dest] === v` while `dest !== i`, that `v` is a duplicate and you advance.

Floyd, when you must not write

Problem 287 says you must not modify the array. The length is `n + 1` and the values lie in `1..n`, so some value repeats. Treat the array as the step from `i` to `nums[i]`. The walk from index 0 enters a cycle. The entrance of the

cycle is the duplicate.

Take `[1, 3, 4, 2, 2]`. Follow the steps: index 0 goes to 1, 1 goes to 3, 3 goes to 2, 2 goes to 4, and 4 goes to 2. The cycle is 2, then 4, then 2. The entrance is 2, and 2 is the repeated value.

Slow moves one step. Fast moves two steps. They start at `nums[0]` and move until they stand on the same index inside the cycle. Then you set one walker back to `nums[0]` and move both one step at a time. They meet at the entrance. You never write into the array.

287, NO WRITES. THE CYCLE ENTRANCE IS THE REPEATED VALUE.

```
function findDuplicate(nums) {
  let slow = nums[0];
  let fast = nums[0];
  do {
    slow = nums[slow];
    fast = nums[nums[fast]];
  } while (slow !== fast);
  slow = nums[0];
  while (slow !== fast) {
    slow = nums[slow];
    fast = nums[fast];
  }
  return slow;
}
```

Update order: both walkers start at `nums[0]`, and the first loop steps them before it compares, because a do-loop checks at the bottom. Fast applies the step twice. After they meet, set `slow` to `nums[0]`, not to 0, and then move each walker one step. On `[1, 3, 4, 2, 2]`, a reset to 0 makes the walkers chase each other and miss the duplicate. Read the constraint before you pick this loop over the swap.

One value alone, twice or three times

XOR is its own inverse. `x ^ x` is 0, and `x ^ 0` is `x`. If every value except one appears twice, XOR the whole array. Pairs vanish. The singleton remains. Order does not matter. On `[4, 1, 2, 1, 2]` the running XOR is 4, then 5, then 7, then 6, then 4. The answer is 4.

XOR does not cancel a value that appears three times. Keep two masks. `ones` holds bits seen `3t + 1` times. `twos` holds bits seen `3t + 2` times. A third sight of a bit clears it out of both masks.

Take `[2, 2, 2, 3]`. The first 2 sets `ones` to 2, and `twos` stays 0. The second 2 clears `ones` and sets `twos` to 2. The third 2 sees that bit in `twos` and clears it out of both masks. Then 3 is a new bit, so `ones` becomes 3. The function returns 3.

SECOND PASS. JAVASCRIPT BITWISE OPS ARE 32-BIT SIGNED. THESE INPUTS FIT.

```
function singleNumber(nums) {
  let ones = 0;
  let twos = 0;
  for (const x of nums) {
    ones = (ones ^ x) & ~twos;
    twos = (twos ^ x) & ~ones;
  }
  return ones;
}
```

Update order: write `ones` first. The old `twos` blocks a bit that is moving on. Then write `twos`, using the new `ones`. If the second line used the old `ones`, a bit could sit in both masks. JavaScript bitwise ops are 32-bit and signed. The usual inputs fit in that width.

Missing number, three ways

Values from 0 through `n` live in an array of length `n`. One number is absent. Take `[3, 0, 1]`, so `n` is 3 and the missing number is 2. Mention all three ways. Write the one you can finish.

Cyclic placement: 0 wants index 0, and 1 wants index 1. 3 wants index 3, which is past the end of an array of length 3, so 3 cannot be placed. After the swaps that do fit, index 2 does not hold 2. That hole is the answer.

XOR: fold every index from 0 through `n` together with every value. Pairs cancel. `0 ^ 1 ^ 2 ^ 3 ^ 3 ^ 0 ^ 1` leaves 2, the index that never appeared as a value.

Sum: the closed formula `n * (n + 1) / 2` is 6. The array sums to 4. The difference is 2. The sum fits in a safe integer for the usual constraints.

Marks in the sign

When every value is in `1..n` and you may modify, the sign of `nums[v - 1]` can record that you have seen `v`. Read `Math.abs` before you use the value as an index. A raw negative cell is not an index.

Disappeared numbers on `[4, 3, 2, 7, 8, 2, 3, 1]`: seeing 4 negates index 3, seeing 3 negates index 2, seeing 2 negates index 1, seeing 7 negates index 6, seeing 8 negates index 7, and seeing 1 negates index 0. The second visits to 2 and 3 find a cell that is already negative, so you leave it. Indexes 4 and 5 stay positive. The missing numbers are 5 and 6.

All duplicates uses the same marks from the other side. A cell that is already negative means the value you just saw is a duplicate. Set mismatch returns both. On `[1, 2, 2, 4]`, the first 1 negates index 0, the first 2 negates index 1, and the second 2 finds index 1 already negative, so 2 is the duplicate. Then 4 negates index 3. Index 2 stayed positive, so the missing value is 3. The answer is `[2, 3]`.

Two singletons

Two values appear once. Everything else appears twice. XOR the whole array. The result is `a ^ b`, which is not 0, so some bit differs between `a` and `b`. Take the lowest set bit, `xor & -xor` in 32-bit arithmetic. Split the numbers into two groups by that bit, and XOR inside each group. Each group reduces to one singleton.

Take `[1, 2, 1, 3, 2, 5]`. The XOR of the whole array is `3 ^ 5`, which is 6, binary 110. The lowest set bit is 2. Numbers that have that bit are 2, 3, and 2. Their XOR is 3. Numbers without that bit are 1, 1, and 5. Their XOR is 5. The two singles are 3 and 5.

Time and extra memory

The flag, the cyclic placement, the sign marks, and the bit passes are O(n) time and O(1) extra memory. Floyd is also O(n) time and O(1) extra memory, and it does not write. The final scan that hunts the first mismatch is another linear pass.

Mistakes

- On [2, 0, 1], if you advance mid after the swap with hi, the array is [1, 0, 2] and mid moves to 1. You never read the 1 at index 0. On [2, 1, 2], that same extra step stops while the array is still [2, 1, 2].
- On a cyclic place, if nums[dest] already equals v and you swap anyway, the two cells trade the same value forever. The loop never ends. The inequality is the stop, and it is also how you notice a duplicate.
- On [4, 3, 2, 7, 8, 2, 3, 1], using a raw negative cell as an index reads before the start of the array. Take the absolute value first, then subtract 1.
- On [1, 3, 4, 2, 2], writing the cyclic swaps modifies an array that problem 287 forbids. Running Floyd when the problem allows writes ignores a constraint the judge stated. Read the constraint, then pick the loop.
- On [2, 2, 3, 2], a plain XOR returns 1, because a triple does not cancel. XOR cancels pairs. Triples need the ones and twos masks, or a count of each bit modulo 3.

Practice

Stay before chapter 14. Then before you leave the chapter. The two bit problems are a second pass.

75. Sort Colors MEDIUM STAY
Three regions. Do not advance mid after swapping with hi.

268. Missing Number EASY STAY
Cyclic placement, or the sum formula, or XOR of indices and values.

136. Single Number EASY STAY
XOR everything. Pairs cancel.

448. Find All Numbers Disappeared in an Array EASY STAY
Negate nums[abs(x) − 1]. Positive cells at the end are the missing numbers.

442. Find All Duplicates in an Array MEDIUM THEN
The same marks. A cell that is already negative means the value is a duplicate.

645. Set Mismatch EASY THEN
One pass finds the duplicate by a second visit. The positive cell is the missing value.

287. Find the Duplicate Number MEDIUM THEN
Do not modify. Floyd on i → nums[i]. The entrance is the duplicate.

41. First Missing Positive HARD THEN
Place v at v − 1 when v is inside 1..n. The first mismatch is the answer.

137. Single Number II MEDIUM SECOND PASS
ones and twos. A bit seen three times clears out of both masks.

260. Single Number III MEDIUM SECOND PASS
XOR all, split by one set bit of that XOR, XOR inside each group.

REWRITE FROM MEMORY
Redraw the three Dutch-flag regions and rewrite the loop until the swap with the high end leaves mid alone.

Carry forward. Chapter 14 searches a sorted index range, and then it searches the answer itself, with one loop shape.

CHAPTER 14 · BUILDS ON CHAPTERS 5 AND 13

Binary search

You keep the index range where the answer can still hide. Each check throws away the half that fails a yes-or-no test.

What this pattern is

A **half-open range** is written `[lo, hi)`. It includes `lo`. It does not include `hi`. `hi` is a fence, not a candidate.

A **predicate** is a yes-or-no test. Call it `ok`. You use it when the answers have a shape: once the test turns true, it stays true as the index grows. That shape lets you throw away half the range.

Picture a dictionary. You open the middle. If the word you see is already past the one you want, the first good page is this page or somewhere to its left, so you drop the right half and you keep this page as the new fence. If the word is still too early, this page failed, so you drop it and everything before it.

This book has one loop. It runs while `lo` is still below `hi`. The middle index rounds down, so `mid` is always strictly below `hi` while the range is not empty. Setting `hi` to `mid` shrinks the range. Setting `lo` to `mid + 1` shrinks it too.

What you already know

Chapter 5 moves two ends because a sum is too small or too large. Chapter 13 puts a value at the index the value names. Binary search does neither. The array, or the answer you are hunting, is ordered. You discard a half because a predicate says the answer cannot live there. You keep the half that can.

A worked example

Find the first index of 3 in `[1, 3, 3, 5, 8]`. The range starts as `[0, 5)`. The fence is the length, 5. The last index is still a candidate.

`mid` is `0 + floor((5 − 0) / 2)`, which is 2. `nums[2]` is 3, and 3 is already at least the target. The first good index is at 2 or to its left. Set `hi` to 2. You keep `mid`, because it might be the first success. The range is now `[0, 2)`.

`mid` is `0 + floor(2 / 2)`, which is 1. `nums[1]` is 3, still large enough. Set `hi` to 1. The range is `[0, 1)`.

`mid` is 0. `nums[0]` is 1, which is too small. The answer is strictly to the right of 0, so this index cannot be the answer. Set `lo` to `mid + 1`, which is 1. The range is `[1, 1)`. `lo` equals `hi`, so you stop. Index 1 is the first 3.

The failing branch has to be `lo = mid + 1`. Here is the loop that never ends. Two indexes remain: `lo` is 0 and `hi` is 1. Then `mid` is 0, which equals `lo`. The value at `mid` fails. If you set `lo = mid`, `lo` stays 0. The range is still `[0, 1)` on the next check. It never shrinks, so `while lo < hi` never stops.

```
nums = [1, 3, 3, 5, 8], target 3
[0, 5) mid 2, nums[2] = 3 >= 3, hi = 2
[0, 2) mid 1, nums[1] = 3 >= 3, hi = 1
[0, 1) mid 0, nums[0] = 1 < 3, lo = 1
lo === hi === 1, the first 3

the infinite loop, so you never write it:
lo = 0, hi = 1, mid = 0
if the failing branch sets lo = mid, lo stays 0 forever
the failing branch must be lo = mid + 1
```

How you recognise it

- The array is sorted, and you want the first index where a predicate becomes true.

- Search insert, the first occurrence, the first bad version, and the integer square root are all that same lower bound.
- The thing you search is a speed, a capacity, a day, or a largest sum. If "can I achieve X?" is false below some boundary and true above it, you can binary-search X.
- A rotated sorted array still has one sorted half. You name that half before you discard anything.
- A peak tells you which neighbor is higher, and you move toward that neighbor.
- Use chapter 5 when both ends move on a sum, and you are not throwing away a half by a yes-or-no test.
- Use chapter 13 when the value is the index, so a swap places it and a search is wasted.
- Use chapter 7 when you grow and shrink a contiguous window, and every index in the middle still matters.

What to say out loud

The answer is inside the half-open range from lo up to hi. If ok(mid) is true, the answer is at mid or to its left, so hi becomes mid. If ok(mid) is false, the answer is strictly to the right, so lo becomes mid + 1. You never set lo to mid. That loop does not end.

The template

THE ONLY TEMPLATE. OK(MID) MEANS "MID IS ALREADY LARGE ENOUGH."

```
function lowerBound(nums, target) {
  // [lo, hi). First index with nums[i] >= target, or nums.length.
  let lo = 0;
  let hi = nums.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] >= target) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
```

The invariant: the first index with `nums[i]` at least the target, or `nums.length` if none exists, lies in `[lo, hi)`.

The off-by-one is where `hi` starts. It starts at `nums.length`, not at the last index. The last index is a candidate. The fence is not. `mid` is `lo + floor((hi − lo) / 2)`. While the range is not empty, that `mid` is at least `lo` and strictly below `hi`.

Update order: if `nums[mid]` is at least the target, set `hi = mid`. You keep `mid`, because it might be the first success. Otherwise set `lo = mid + 1`. You drop `mid`, because it failed. When two indexes remain, `mid` equals `lo`. The failing branch `lo = mid` never ends. Search insert position is this function. The insertion index may equal the length.

Search in a rotated array

The values are unique. At `mid`, compare `nums[lo]` with `nums[mid]` and name the sorted half before you discard anything. If `nums[lo]` is at most `nums[mid]`, the left half from `lo` through `mid` is the sorted one. The target is inside it only when it sits between `nums[lo]` and `nums[mid]`. Otherwise the target sits in the other half. When the left half is not sorted, the right half is the sorted one. Use `nums[hi − 1]` as the rightmost included value, because `hi` is exclusive.

Take `[4, 5, 6, 7, 0, 1, 2]` and target 0. The range is `[0, 7)`. `mid` is 3, value 7. `nums[0]` is 4, and 4 is at most 7, so the left half `[4, 7]` is sorted. 0 is not inside it. Discard that half. `lo` becomes 4.

The range is `[4, 7)`. `mid` is 5, value 1. `nums[4]` is 0, and 0 is at most 1, so the left half is sorted. 0 sits between `nums[lo]` and `nums[mid]`. Keep that half. `hi` becomes 5. The range is `[4, 5)`. `mid` is 4, value 0, which equals the target. Return 4.

UNIQUE VALUES. THE HALF YOU JUST TESTED IS THE SORTED ONE.

```
function search(nums, target) {
  let lo = 0;
  let hi = nums.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] === target) return mid;
    if (nums[lo] <= nums[mid]) {
      if (nums[lo] <= target && target < nums[mid]) hi = mid;
      else lo = mid + 1;
    } else if (nums[mid] < target && target <= nums[hi - 1]) {
      lo = mid + 1;
    } else {
      hi = mid;
    }
  }
  return -1;
}
```

```
[4, 5, 6, 7, 0, 1, 2], target 0
[0, 7) mid 3 value 7, left half [4, 7] is sorted, 0 is not in it, lo = 4
[4, 7) mid 5 value 1, left half [0, 1] is sorted, 0 is in it, hi = 5
[4, 5) mid 4 value 0, return 4
```

Update order: if `nums[mid]` equals the target, return `mid` immediately. Else if the left side from `lo` through `mid` is sorted, keep it only when `nums[lo]` is at most the target and the target is strictly below `nums[mid]`. Otherwise set `lo` to `mid + 1`. In the other branch the right side is sorted. Keep it only when the target is strictly above `nums[mid]` and at most `nums[hi − 1]`, and then set `lo` to `mid + 1`. Otherwise set `hi` to `mid`. You name the sorted half before either discard.

Rotated duplicates

Rotated array II may contain duplicates. When the value at `lo`, the value at `mid`, and the included right end are equal, you cannot tell which half is sorted. Shrink one end by one. Set `lo` to `lo + 1`, and continue.

Take `[2, 2, 2, 2, 2]` and target 3. Every mid you land on matches both ends. Each step throws away one index. The range empties and you return -1. That walk visits every index. The worst case is `O(n)`, on an array of identical values with the target missing or sitting at the end. Say that cost out loud before you write the shrink.

Koko eats at a speed

Koko eats at speed `s`. A pile takes `ceil(pile / s)` hours. You want the smallest `s` that finishes within `h` hours. The low end is 1. The high fence is one more than the largest pile. `ok(s)` sums the hours and asks whether the sum is at most `h`.

Take piles `[3, 6, 7, 11]` and `h = 8`. The range is `[1, 12)`. The first mid is 6. Speed 6 costs `ceil(3/6) + ceil(6/6) + ceil(7/6) + ceil(11/6)`, which is `1 + 1 + 2 + 2 = 6` hours. 6 is within 8, so a slower speed might work too. Set `hi` to 6. The next mid is 3. Speed 3 costs `1 + 2 + 3 + 4 = 10`, which is past 8. Set `lo` to 4. The next mid is 5. Speed 5 costs `1 + 2 + 2 + 3 = 8`, which meets the limit, so set `hi` to 5. The next mid is 4. Speed 4 also costs 8, so set `hi` to 4. `lo` meets `hi` at 4. The answer is 4.

Update order is the lower-bound update. `ok(mid)` means this speed is fast enough, so `hi = mid`. A fail sets `lo = mid + 1`. You are searching the answer, which is a speed, and the index template is only the shape of the loop.

Ship capacity

The ship must carry the heaviest box, so the low end is that box. The high fence is one more than the total weight. `ok(cap)` counts days: fill the current ship until the next box does not fit, then open another day.

Take weights `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` and 5 days. The low end is 10. The total is 55, so the fence is 56. Capacity 15 packs as `1+2+3+4+5`, then `6+7`, then 8, then 9, then 10. That is 5 days, so 15 is fast enough and you move the fence down. Capacity 14 packs as 10, then 11, then 7, then 8, then 9, then 10. That is 6 days, so 14 fails and you move `lo` to 15. The smallest working capacity is 15.

Bloom day

If `m * k` exceeds the number of flowers, return -1. Otherwise search the day. A day can make a bouquet when `k` adjacent flowers have all bloomed by that day. Reset the adjacent run when a flower is late.

Take bloom days `[1, 10, 3, 10, 2]`, with `m = 3` and `k = 2`. `m * k` is 6, and there are only 5 flowers, so you return -1 before any search. Change `k` to 1. You need 3 single flowers. On day 1 only index 0 is ready. On day 2, indexes 0 and 4 are ready. On day 3, indexes 0, 2, and 4 are ready. The smallest such day is 3.

Adjacency matters on `[7, 7, 7, 7, 12, 7, 7]` with `m = 2` and `k = 3`. On day 7 the first four flowers are ready, which makes one bouquet of 3 with one flower left over. The 12 is late, so the run resets. The last two flowers make no bouquet. On day 12 every flower is ready, one run of 7 makes two bouquets, and the answer is 12.

Split array, same predicate as the ship

You cover the array in at most `m` contiguous parts, and you minimize the largest part sum. The low end is the largest element, because that element is a part by itself. The high fence is one more than the total. `ok(cap)` is true when you can cover the array in at most `m` parts, each summing to at most `cap`.

Take `[7, 2, 5, 10, 8]` and `m = 2`. The low end is 10. The total is 32, so the fence is 33. Capacity 18 packs `7+2+5` and then `10+8`. Two parts, so it works. Capacity 17 packs `7+2+5`, then 10, and then 8 needs a third part because `10+8` is 18. Three parts, so it fails. The smallest working cap is 18. It is the ship again, and it is a second-pass problem.

Square root

Search `[0, x + 1)` for the first integer whose square is greater than `x`, then subtract 1. For `x = 8` the squares of 0, 1, 2, 3 are 0, 1, 4, 9. The first square greater than 8 is 9, at 3. Subtract 1. The floor root is 2.

For `x` up to `2^31 − 1`, `mid * mid` is a safe JavaScript number. If you ever search a larger domain, compare with `mid > x / mid` instead of multiplying. A perfect square is the integer square root, squared, equal to the input. 0 and 1 are squares. If your predicate was "the square is already too big" and you return the boundary without subtracting 1, you return 3 for input 8.

Insert, a range of matches, and the first bad version

Search insert on `[1, 3, 5, 6]` is `lowerBound`. Target 2 lands at index 1, the first value that is at least 2. Target 7 lands at index 4, which equals the length. Target 0 lands at index 0.

The first and last position of a target are `lowerBound(target)` and `lowerBound(target + 1) − 1`. On `[5, 7, 7, 8, 8, 10]`, target 8 starts at index 3. `lowerBound(9)` is index 5, minus 1 is 4. The answer is `[3, 4]`. Target 6 has its first index at 2, and `nums[2]` is 7, not 6, so the answer is `[−1, −1]`. If the first index is past the end, the answer is the same pair of -1s.

First bad version is `lowerBound` on versions `1..n`, with `hi = n + 1` and `ok` equal to `isBadVersion`. For `n = 5` and first bad version 4, the range starts at `[1, 6)`. `mid` is `1 + floor((6 − 1) / 2)`, which is 3. Version 3 is

still good, so `lo` becomes 4. The range is `[4, 6)`. `mid` is 5. Version 5 is bad, so `hi` becomes 5. The range is `[4, 5)`. `mid` is 4. Version 4 is bad, so `hi` becomes 4. You stop at 4. Call `isBadVersion` only as often as the loop does.

Minimum of a rotated array

The minimum is the only place the sequence drops. This loop uses a closed right end, so `hi` is a candidate. While `lo` is below `hi`, if `nums[mid]` is greater than `nums[hi]`, the minimum is strictly right of `mid`, and `lo` becomes `mid + 1`. Otherwise `mid` might be the minimum, so `hi` becomes `mid`. The range still shrinks, because `mid` is computed below `hi`.

Take `[3, 4, 5, 1, 2]`. `lo` is 0 and `hi` is 4. `mid` is 2, value 5. 5 is greater than `nums[4]`, which is 2, so `lo` becomes 3. `mid` is now 3, value 1. 1 is not greater than `nums[4]`, so `hi` becomes 3. `lo` meets `hi`. The minimum is `nums[3]`, which is 1.

A peak

A peak is an index greater than or equal to its neighbors. An end is a peak if it beats its one neighbor. The missing neighbor counts as smaller. If `nums[mid]` is greater than `nums[mid + 1]`, a peak exists at `mid` or to its left. Otherwise a peak exists to the right. Move toward the higher neighbor.

Take `[1, 2, 3, 1]`. At index 1 the value is 2, and the right neighbor is 3, which is higher, so you move right. At index 2 the value is 3, and the right neighbor is 1, which is lower, so a peak exists at index 2 or to its left. Index 2 is a peak. A mountain array has one peak, so the same neighbor test returns that unique index.

One element between pairs

A single element in a sorted array sits between pairs. Before that element, pairs occupy an even index and the next odd index. Force `mid` even. If `nums[even]` equals `nums[even + 1]`, the single element is to the right, and you set `lo` to `even + 2`. Otherwise it is at this even index or to the left, and you set `hi` to that even index.

Take `[1, 1, 2, 3, 3, 4, 4, 8, 8]`. The single element is 2, at index 2. Start with `lo` 0 and closed `hi` 8. `mid` 4 holds 3, and index 5 holds 4, so they differ. The single is at this even index or to the left. `hi` becomes 4. `mid` 2 holds 2, and index 3 holds 3, so they differ. `hi` becomes 2. The next even mid is 0. Indexes 0 and 1 are both 1, so the single is to the right. `lo` becomes 2. `lo` meets `hi`. The answer is 2.

K closest elements

Binary-search the start of a window of length `k`. The legal starts are `0` through `n - k`. Prefer the window that starts at `mid` over the one that starts at `mid + 1` when `x - arr[mid]` is less than or equal to `arr[mid + k] - x`. An equal distance keeps the left window, which holds the smaller values.

Take `[1, 2, 3, 4, 5]`, `k = 4`, and `x = 3`. The legal starts are 0 and 1. The window at 0 is `[1, 2, 3, 4]`. The window at 1 is `[2, 3, 4, 5]`. At start 0, `x - arr[0]` is 2, and `arr[4] - x` is 2. The distances tie, so you keep the left window. The answer is `[1, 2, 3, 4]`.

Median of two sorted arrays

Binary-search the cut on the shorter array. Take `i` elements from it and `half - i` from the longer one. `half` is `floor((m + n + 1) / 2)`. The cut is correct when the left side of each array is at most the right side of the other. If you took too much from the shorter array, move the cut left. An odd total returns the max of the two left edges. An even total averages that with the min of the two right edges. An empty side uses a very small left edge or a very large right edge, so the comparison still has a number.

Take `[1, 3]` and `[2]`. The shorter array is `[2]`. The total length is 3, so `half` is 2. Try `i = 0` from the shorter array, which means you take 2 from the longer one. The longer left edge is 3. The shorter right edge is 2. 3 is not at most 2, so you took too little from the shorter array and you move the cut right. Try `i = 1`. You take 2 from the

short array and 1 from the long one. The left edges are 1 and 2. The long right edge is 3. 2 is at most 3, and the short array has nothing on the right. The cut is correct. The total is odd, so the median is the larger left edge, which is 2.

Find in a mountain array

Find the peak with the neighbor test, using the class method `get`. Binary-search the ascending side for the target. If it is absent, binary-search the descending side. On a descent, a value that is too small means the target is toward the peak, so you move left. Cache nothing the API forbids.

Take `[1, 2, 3, 4, 5, 3, 1]` and target 3. The peak is index 4, value 5. The ascending side holds 3 at index 2, so you return 2 and you do not search the right side. If the target lived only on the descent, a mid that reads 3 while the target is 4 would move left, because 3 is below 4 and the slope falls as you move right.

Time and extra memory

The index searches use O(log n) comparisons and O(1) extra memory. A feasibility test wrapped around the same loop can make the whole algorithm O(n log A), where A is the size of the answer range. Koko, the ship, bloom day, and the split sum all pay that, because each ok walks the array. Rotated array II is O(n) in the worst case.

Mistakes

- On a search for 3 in [1, 3], the range can shrink to lo = 0 and hi = 1, so mid = 0. The value 1 fails. If you set lo = mid, lo stays 0 and the loop never ends. The failing branch sets lo to mid + 1.
- On [1, 3, 5], if hi starts at the last index and you also use the half-open update hi = mid, you drop or keep the wrong end. Pick one range, say whether hi is included, and use the update that matches that sentence.
- On [4, 5, 6, 7, 0, 1, 2], target 0, treating the whole array as sorted walks into the large left half and misses index 4. Name the sorted half before you discard anything.
- On [2, 2, 2, 2, 2], target 3, assuming a sorted half when both ends and mid match can discard the side that still holds a different value. Shrink one end. The worst case is linear.
- On sqrt(8), if your predicate is "the square is already too big" and you return the boundary 3, the square is 9. Subtract 1 to get the floor root 2.

Practice

Stay before chapter 15. Then before you leave the chapter. Three problems are a second pass.

704. Binary Search EASY STAY
The template, then check that nums[lo] equals the target. The classic version returns the index or −1.

35. Search Insert Position EASY STAY
lowerBound itself. The insertion index may equal nums.length.

34. Find First and Last Position of Element in Sorted Array MEDIUM STAY
lower(target) and lower(target + 1) − 1. Absent target returns [-1, -1].

875. Koko Eating Bananas MEDIUM STAY
Search the speed. ok sums ceil(pile / speed) and compares with h.

33. Search in Rotated Sorted Array MEDIUM STAY
Unique values. One half is sorted. Discard the half that cannot hold the target.

153. Find Minimum in Rotated Sorted Array MEDIUM STAY
If nums[mid] > nums[hi], the min is to the right. Otherwise hi = mid.

278. First Bad Version EASY THEN
lowerBound on 1..n with hi = n + 1. ok is isBadVersion. Call it as rarely as the loop does.

69. Sqrt(x) EASY THEN
First integer whose square exceeds x, then subtract 1.

367. Valid Perfect Square EASY THEN
The integer square root, squared, equals the input. 0 and 1 are squares.

1011. Capacity To Ship Packages Within D Days MEDIUM THEN
Low is the heaviest package. ok counts the days a capacity needs.

1482. Minimum Number of Days to Make m Bouquets MEDIUM THEN
Adjacent flowers. Return −1 when m * k > n.

81. Search in Rotated Sorted Array II MEDIUM THEN
When both ends equal mid, shrink an end. Worst case O(n).

162. Find Peak Element MEDIUM THEN
Move toward the higher neighbor. An end can be the peak.

852. Peak Index in a Mountain Array MEDIUM THEN
The same neighbor test. A mountain has one peak, so the answer is unique.

540. Single Element in a Sorted Array MEDIUM THEN
Force mid even. Equal pair means the single element is to the right.

658. Find K Closest Elements MEDIUM THEN
Binary-search the window start. A distance tie keeps the smaller values.

410. Split Array Largest Sum HARD SECOND PASS
Same predicate as the ship. Minimize the largest part sum, using at most m parts.

1095. Find in Mountain Array HARD SECOND PASS
Peak, then ascending search, then descending search. Use get, and cache nothing the API forbids.

4. Median of Two Sorted Arrays HARD SECOND PASS
Partition the shorter array. Left edges <= right edges. Odd and even totals differ.

REWRITE FROM MEMORY

Rewrite lowerBound, and say the infinite loop that lo = mid creates when two indexes remain and the range stops shrinking.

Carry forward. Chapter 15 keeps a stack of indexes in a forced order, so the next greater element is the value that pops you.

CHAPTER 15 · BUILDS ON CHAPTERS 2 AND 7

Monotonic stack

You keep a stack of indexes whose values decrease from bottom to top. The value that pops an index is its next greater element.

What this pattern is

A **monotonic stack** is a stack whose values stay in one forced order. In this chapter the order is decreasing from bottom to top. The bottom is an older large value. The top is a newer small value.

A running maximum tells you the greatest value anywhere to the right. The next greater element is a nearer question. It is the first value to the right that is strictly larger. A single maximum cannot answer that, because a medium value can stand between you and the maximum, and that medium value is the one that matters.

Picture a line of people facing right. You want the first person taller than you. You remember the people who are still waiting, from taller to shorter. When a taller person arrives, everyone shorter on the top of your memory can see them. You give those people their answer and forget them. The taller person then waits. Each index is pushed once and popped at most once.

What you already know

Chapter 2 can walk once and remember the best value to the right. Chapter 7 can slide a window when the rule is a sum or a count. Neither of those stores a chain of values that are still waiting for a greater neighbor. This chapter stores that chain as indexes. The value stays in the array, so you can still read it, and the index lets you write a distance.

A worked example

Take `[2, 1, 2, 4, 3]`. The answers start as `[-1, -1, -1, -1, -1]`. The stack stores indexes. It starts empty.

Index 0 holds 2. Nothing is on the stack, so nothing pops. Push 0. The stack is `[0]`.

Index 1 holds 1. The top index is 0, and `nums[0]` is 2, which is not strictly smaller than 1. You do not pop. Push 1. The stack is `[0, 1]`. From bottom to top the values are 2, then 1. That is decreasing.

Index 2 holds 2. The top is index 1, value 1, and 1 is strictly smaller than 2. Pop index 1. Its next greater value is 2, so `ans[1]` becomes 2. The new top is index 0, value 2, which is not strictly smaller than 2. An equal neighbor is not a greater neighbor. Push 2. The stack is `[0, 2]`.

Index 3 holds 4. The top index 2 holds 2, which is smaller than 4. Pop it. `ans[2]` becomes 4. The top index 0 also holds 2, which is smaller than 4. Pop it. `ans[0]` becomes 4. Push 3. The stack is `[3]`.

Index 4 holds 3. The top value is 4, which is not strictly smaller than 3, so you do not pop. Push 4. Indexes 3 and 4 stay on the stack, because nothing to their right is strictly greater. Their answers stay -1. The answer is `[4, 2, 4, -1, -1]`. Every index was pushed once. Indexes 0, 1, and 2 were popped once.

```
[2, 1, 2, 4, 3]
i=0 push 0
i=1 value 1, top 2 is not smaller, push 1
i=2 value 2 pops index 1, ans[1] = 2, top 2 is not smaller, push 2
i=3 value 4 pops index 2, ans[2] = 4, pops index 0, ans[0] = 4, push 3
i=4 value 3, top 4 is not smaller, push 4
ans = [4, 2, 4, -1, -1]
```

How you recognise it

- You want the next greater value, the next smaller value, the previous smaller value, or the number of days until a warmer temperature.
- A span asks how many values to the left are less than or equal to today. You answer by collapsing those values into one stack entry.
- A histogram bar is the shortest bar of some rectangle, and you need the first shorter bar on each side.
- You want the maximum inside every window of a fixed length. One variable cannot recover the second maximum after the first one slides out.
- The array can contain negatives, and you want the shortest subarray whose sum is at least K. A plain window dies, because dropping the left end can make the sum worse.
- Use chapter 2 when any greater value to the right is enough, and you do not need the nearest one.
- Use chapter 7 when the window is ruled by a sum or a count, and the maximum inside the window is not the question.
- Use chapter 8 when a prefix sum, looked up in a map, answers the subarray, and you never need the nearest greater index.

What to say out loud

The stack decreases from bottom to top. Everything below an index is greater than it and is still waiting for its next greater value. Everything you have popped already has an answer. You store indexes, so you can still read the value and the distance.

The template

POP WHILE THE TOP IS STRICTLY SMALLER. THE STACK STORES INDICES.

```
function nextGreater(nums) {
  const n = nums.length;
  const ans = new Array(n).fill(-1);
  const stack = [];
  for (let i = 0; i < n; i++) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[i]) {
      ans[stack.pop()] = nums[i];
    }
    stack.push(i);
  }
  return ans;
}
```

The invariant: from bottom to top, the values at the stacked indexes are strictly decreasing. Each popped index has its answer written. Each index still on the stack has no strictly greater value among the indexes you have visited.

The off-by-one is the compare. It is a strict less-than. An equal neighbor does not pop, so it is not recorded as next greater. An index with no greater neighbor keeps the -1 you wrote when you filled the array.

Update order: pop first, while the stack is not empty and the top value is strictly smaller than `nums[i]`, and write `nums[i]` into that popped index. Then push `i`. If you push before you pop, the new index sits on top of values it should have answered, and those values never see it.

The window maximum

A deque is a line you can add to or remove from at either end. This one stores indexes. The values decrease from the front to the back. A head pointer marks the front. You never call `Array.shift`. `shift` moves every later element, and the window maximum becomes `O(nk)`. Moving the head is `O(1)`.

Take `[1, 3, -1, -3, 5, 3, 6, 7]` and `k = 3`. Index 0 pushes 0. Index 1 holds 3, which is at least the back value 1, so you pop 1. 3 is later and at least as large, so 1 can never be the max of a later window. Push 1. Index 2 holds -1. 3 is larger, so you keep it and push 2. The window is full. The front value is 3. Emit 3.

Index 3 holds -3. The front index 1 is still inside the window, because it is greater than `3 - 3`. The back value -1 is greater than -3, so you push 3 and emit 3 again. Index 4 holds 5. The front index 1 has now fallen to the edge `i - k`, so the head advances past it. Every back value is at most 5, so you pop them and push 4. Emit 5. The same front-drop and back-pop steps produce the rest: 5, 6, 7. The full output is `[3, 3, 5, 5, 6, 7]`.

INDICES, DECREASING VALUES. A HEAD POINTER REPLACES ARRAY.SHIFT.

```
function maxSlidingWindow(nums, k) {
  const dq = [];
  let head = 0;
  const out = [];
  for (let i = 0; i < nums.length; i++) {
    while (head < dq.length && dq[head] <= i - k) head += 1;
    while (head < dq.length && nums[dq[dq.length - 1]] <= nums[i]) dq.pop();
    dq.push(i);
    if (i >= k - 1) out.push(nums[dq[head]]);
  }
  return out;
}
```

Update order: first drop a front index that has fallen out, while the head points at an index at most `i - k`. Then pop from the back while the back value is less than or equal to `nums[i]`. Then push `i`. When `i` is at least `k - 1`, the window is full and you read the front. Dropping the front after you read it would emit a value that has already left the window.

Histogram, with a zero on each end

A **sentinel** is an extra value you place on purpose so the loop has a known stopper. Here you put 0 on both ends of the heights. The right 0 is shorter than every positive bar, so it pops every real bar. The left 0 stays under them, so the stack is never empty when you read the left edge.

The stack stores indexes. The bar you pop is the shortest bar of a rectangle that runs from the new top, exclusive, to `i`, exclusive. The width is `i - left - 1`.

Take heights `[2, 1, 2]`. With sentinels they become `[0, 2, 1, 2, 0]`. The stack starts as `[0]`, the left sentinel. Index 1 holds 2. The top height 0 is not greater than 2, so you push 1. Index 2 holds 1. The top height 2 is greater than 1, so you pop that bar. The new top is the left sentinel, index 0. The width is `2 - 0 - 1`, which is 1. The area is 2. You then push index 2.

Index 3 holds 2. The top height 1 is not greater, so you push 3. Index 4 holds the right sentinel 0. You pop the 2 at index 3. The left edge is index 2. The width is `4 - 2 - 1`, which is 1. The area is 2. You pop the 1 at index 2. The left edge is the leading 0. The width is `4 - 0 - 1`, which is 3. The area is 3. The best area is 3.

THE SENTINELS MAKE EVERY BAR POP, AND THE STACK IS NEVER EMPTY WHEN YOU READ THE LEFT EDGE.

```
function largestRectangleArea(heights) {
  const h = [0, ...heights, 0];
  const stack = [0];
  let best = 0;
  for (let i = 1; i < h.length; i++) {
    while (h[stack[stack.length - 1]] > h[i]) {
      const mid = stack.pop();
      const left = stack[stack.length - 1];
      const width = i - left - 1;
      const area = h[mid] * width;
      if (area > best) best = area;
```

```
    }
    stack.push(i);
  }
  return best;
}
```

```
heights [2, 1, 2] become [0, 2, 1, 2, 0]
i=1 value 2, push
i=2 value 1 pops the 2: left is the leading 0, width 1, area 2
i=3 value 2, push
i=4 value 0 pops the 2: width 1, area 2
         pops the 1: left is the leading 0, width 3, area 3
best 3
```

Update order: while the top bar is strictly taller than `h[i]`, pop it, read the left index that is now on top, and score height times `i - left - 1`. After that loop, push `i`. Strict greater-than against the new bar, with these sentinels, lets equal bars share a rectangle. If you change the compare, the width formula has to change with it. You do not special-case an empty stack in the middle. The left sentinel is the reason the left read is safe.

Distances, and a circular array

Daily temperatures stores the same indexes and writes `i - j`, the distance, instead of the temperature. A 0 means no warmer day. Take `[73, 74, 75, 71, 69, 72, 76, 73]`. Day 0 is 73, and 74 is one day later. Day 1 waits one day for 75. Day 2 waits four days for 76. Day 3 waits two days for 72. Day 4 waits one day for 72. Day 5 waits one day for 76. Days 6 and 7 have nothing warmer. The answer is `[1, 1, 4, 2, 1, 1, 0, 0]`.

A circular array scans `i` from 0 to `2n - 1` and reads `nums[i % n]`. Push an index only when `i` is below `n`, so each position enters the stack once. Write answers only for those real indexes. On `[1, 2, 1]` the next greater values are `[2, -1, 2]`. The last 1 wraps around and sees the first 2. The middle 2 never finds a greater value.

Stock span

Online stock span stores pairs `[price, span]`. The span starts at 1, because today includes itself. Pop while the top price is less than or equal to today's price, and add the popped spans to today.

Take prices 100, 80, 60, 70, 60, 75, 85. 100 starts a span of 1. 80 is below 100, so its span is 1. 60 is below 80, so its span is 1. 70 pops the 60 and adds that span, so 70 has span 2. The next 60 has span 1. 75 pops that 60 and the 70, adding 1 and 2, so 75 has span 4. 85 pops the 75 and the 80, adding 4 and 1, so 85 has span 6. The spans are `[1, 1, 1, 2, 1, 4, 6]`.

Shortest subarray with sum at least K

Negatives are why a chapter 7 window dies. Build the prefix from chapter 8. Keep a deque of prefix indexes whose prefix values increase. A later, smaller prefix makes an earlier prefix useless as a start, because it is both larger and older. When `prefix[i] - prefix[head]` is at least `k`, that head is a candidate start. A later end would only make a longer subarray for the same start, so you pop the head and record the length. The answer is the minimum length, or -1.

Take `[2, -1, 2]` and `k = 3`. The prefix, with a 0 in front, is `[0, 2, 1, 3]`. Index 0 pushes 0. Index 1 holds prefix 2. `2 - 0` is below 3, and 0 is below 2, so you push 1. Index 2 holds prefix 1. The back prefix 2 is at least 1, so you pop index 1. The prefix 0 is smaller than 1, so you push 2. Index 3 holds prefix 3. `3 - 0` is 3, which meets `k`, so the length 3 is a candidate and you pop the head. `3 - 1` is 2, which is below `k`. The best length is 3, the whole array.

Absolute difference within a limit

Keep two deques and the same head-pointer trick. One decreases, and it tracks the max. One increases, and it tracks the min. Shrink `l` while max minus min exceeds the limit. When a head index equals `l`, that end has left the window, so you drop it.

Take `[8, 2, 4, 7]` and limit 4. The pair 8 and 2 differs by 6, so you shrink until the window is just 2. Adding 4 makes `[2, 4]`, which differs by 2. Adding 7 makes `[2, 4, 7]`, which differs by 5, so you shrink to `[4, 7]`, which differs by 3. The longest window has length 2.

Sum of subarray minimums

For each index, count the subarrays that have that index as their minimum. Measure the distance to the previous strictly smaller bar, and the distance to the next smaller-or-equal bar. A strict test on one side and a non-strict test on the other counts each equal minimum once. The contribution is `value * left * right`, summed modulo `1e9+7`.

Take `[3, 1, 2, 4]`. The 3 owns only the subarray `[3]`, contribution 3. The 1 is the minimum of every subarray that includes it, which is 6 of them, contribution 6. The 2 owns `[2]` and `[2, 4]`, contribution 4. The 4 owns `[4]`, contribution 4. The total is 17.

On `[1, 1]`, the left 1 stops at the next equal bar, so it owns only its own cell. The right 1 treats the previous equal as not strictly smaller, so its left distance steps over the tie and it owns the pair plus its own cell. The three subarrays are counted once. If both sides are strict, both copies claim the pair, and the modulo answer is wrong.

The 132 pattern

Scan from the right. The stack holds possible 3s, and it stays decreasing. While the top is smaller than the current value, pop that top into `ak`. `ak` is the best 2 you have seen, and it had a larger value to its left, the 3 that popped it. When the current value is smaller than `ak`, that current value is a 1 further left, and you return true.

Take `[3, 1, 4, 2]`. Start at 2. The stack is empty, so you push 2. Next, from the right, the value is 4. 4 is greater than 2, so you pop 2 into `ak` and push 4. Next the value is 1. 1 is smaller than `ak`, so you return true. The pattern is 1, then 4, then 2. On `[1, 2, 3, 4]` every new value from the right is smaller than the top, so `ak` never becomes a real 2, and you return false.

Time and extra memory

Each index is pushed once and popped once, so the stack scans are O(n) time and O(n) extra memory. The window deques are the same bound. Array.shift on the deque is the operation that turns the window maximum into O(nk). A head pointer keeps the front drop O(1).

Mistakes

- On temperatures [73, 74], if you store the value 73, you can see that 74 is warmer, but you cannot write the distance 1. Store the index, and read the value from the array.
- On a window of length 3 and a long array, Array.shift drops the front by moving every later index. The run becomes O(nk). A head pointer drops the front in O(1).
- On heights [2, 1, 2], popping with a compare you picked at random, or using a width other than i - left - 1, scores the bar against the wrong neighbor. Strict greater-than against the new bar matches these sentinels. Stay with that pair.
- On heights [2, 1, 2] with no sentinel zeros, the stack can go empty when you pop the last bar. The read of the left edge crashes, or a special case in the middle of the loop gets the width wrong.
- On [1, 1], counting both sides of an equal minimum as strict lets both copies claim the pair. The sum double-counts that pair, and the modulo answer is wrong.

Practice

Stay before chapter 16. Then before you leave the chapter. Three problems are a second pass.

739. Daily Temperatures MEDIUM STAY
Stack of indices. The answer is the distance, and 0 means no warmer day.

496. Next Greater Element I EASY STAY
Next greater on nums2, then look up each value of nums1. A map from value to answer avoids a second search.

239. Sliding Window Maximum HARD STAY
Deque of indices, head pointer, pop the back while it is smaller or equal.

84. Largest Rectangle in Histogram HARD STAY
Sentinel 0s. Width is i − left − 1. The popped bar is the height.

503. Next Greater Element II MEDIUM THEN
Scan to 2n − 1. Push only while i < n.

901. Online Stock Span MEDIUM THEN
Stack of [price, span]. Pop prices that are <= today and add their spans.

1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit MEDIUM THEN
One deque for the max, one for the min. Shrink while the difference exceeds limit.

862. Shortest Subarray with Sum at Least K HARD SECOND PASS
Prefix plus an increasing deque of prefix indices. Negatives are why the window died.

907. Sum of Subarray Minimums MEDIUM SECOND PASS
Strict previous smaller, non-strict next smaller. Contribution modulo 1e9+7.

456. 132 Pattern MEDIUM SECOND PASS
Right to left. ak is the best 2 popped by a larger 3. A later 1 that is smaller than ak finishes it.

REWRITE FROM MEMORY
Rewrite next-greater and the histogram with a zero sentinel on both ends. Say why Array.shift is not the way you drop the front of a deque.

Carry forward. Chapter 16 is a scan that keeps a reach or a tank, and it never needs a stack.

CHAPTER 16 · BUILDS ON CHAPTERS 2 AND 15

Greedy scans

You keep the farthest index you can reach. If the index you are standing on is past that reach, you stop.

What this pattern is

Some problems only need the farthest index you can currently land on. You do not keep a stack of waiting values. You walk from left to right. You keep one number. A failure is the moment your foot steps past that number.

Picture stones in a river. Each stone tells you how far you may leap from it. You remember the farthest stone any leap so far can reach. If the next stone is already beyond that memory, you are in the water. You do not leap from a stone you cannot stand on.

The same shape covers a gas circuit, a flower bed, and one allowed edit. Each of them keeps a small fact, updates it at the current index, and stops when that fact says the walk has failed.

What you already know

Chapter 2 walks once and updates a total. Chapter 15 keeps a stack because the nearest greater neighbor matters. Here the neighbor does not matter. Only the farthest reach matters, or a running tank, or a single edit you have not spent yet. A reach does not shrink. A window from chapter 7 does.

A worked example

Ask whether you can reach the last index of `[2, 3, 1, 1, 4]`. `reach` starts at 0. You can stand on index 0, because 0 is not past `reach`.

From index 0 you may leap 2, so you can land as far as `0 + 2`. `reach` becomes 2.

Index 1 holds 3. 1 is still within 2, so you may stand there. You test that before you extend. From index 1 you can land as far as `1 + 3`, which is 4. `reach` becomes 4.

Index 2 holds 1. 2 is within 4, so you may stand there. The leap `2 + 1` is 3, which does not beat 4, so `reach` stays 4.

Index 3 is within 4. Index 4 is the last stone, and 4 equals `reach`, so you may stand there too. You never step past the number. The answer is true.

Now take `[3, 2, 1, 0, 4]`. From index 0, `reach` becomes 3. Index 1 can leap to `1 + 2`, which is still 3. Index 2 can leap to `2 + 1`, which is still 3. Index 3 can leap to `3 + 0`, which is still 3. Index 4 is greater than 3.

You return false at index 4 before you extend `reach` from that index. You cannot stand on 4, so its leap does not count. If you extended first, `4 + 4` would hide the failure and the function would say true.

```
[2, 3, 1, 1, 4]
i=0 reach = 2
i=1 reach = max(2, 1+3) = 4
i=2, 3, 4 are all <= 4
true

[3, 2, 1, 0, 4]
i=0 reach 3
i=1 reach 3
i=2 reach 3
i=3 reach 3
i=4 > 3, false
```

How you recognise it

- From each index you may leap forward by at most nums[i], and you ask whether the last index is reachable, or how many leaps you need.
- You drive a circuit of gas stations. If the total gas is less than the total cost, no start works. Otherwise one start does.
- You may place a flower only in an empty slot whose two neighbors are empty, or off the end of the bed.
- You may change an array in at most one place to make it non-decreasing, and the cell two steps back decides which value you edit.
- The choice at the current index depends on a reach, a tank, or a single spent change.
- Use chapter 15 when you need the next greater value or the widest rectangle. A single reach cannot store those waiting indexes.
- Use chapter 2 when one running total is the whole answer, and there is no test that the current index is legal to stand on.
- Use chapter 7 when you shrink a window from the left to repair a sum. A reach does not shrink.

What to say out loud

Reach is the farthest index the jumps you have already seen can land on. If i itself is beyond reach, every later index is impossible too. You test that before you extend reach. You do not jump from an index you cannot stand on.

The template

CHECK I AGAINST REACH BEFORE YOU EXTEND IT.

```
function canJump(nums) {
  let reach = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}
```

An **invariant** is a fact that stays true every time the loop comes back to check it. Here `reach` is the farthest index you can land on using jumps from indexes you were allowed to stand on. Every index from 0 through `reach` is reachable.

The off-by-one is the test. You fail when `i` is greater than `reach`. Standing on `reach` is legal. The last index is a success when it equals `reach`.

Update order: test `i > reach` first. If it is true, return false. Only then set `reach` to the max of `reach` and `i + nums[i]`. Extending first lets a forbidden index stretch `reach` and hide the failure. On `[3, 2, 1, 0, 4]` that forbidden index is 4.

How many jumps

Jump II asks for the number of jumps. Think of the array as layers. `end` is the right edge of the layer you are jumping from. `far` is the furthest anything in this layer can reach. When `i` arrives at `end`, you must take one jump, and the next layer ends at `far`. The loop stops at `n - 2`. The last index does not need a jump of its own.

Take `[2, 3, 1, 1, 4]`. At index 0, `far` becomes 2. `i` equals `end`, so you take a jump. `jumps` is 1 and `end` becomes 2. At index 1, `far` becomes `max(2, 1 + 3)`, which is 4. Index 1 is not the end of the layer. At index 2, `far` stays 4, and `i` equals `end`, so you take a second jump and `end` becomes 4. The loop stops. It does not visit index 4. The answer is 2. The jumps land 0 to 1, then 1 to 4.

ONE JUMP EACH TIME YOU EXHAUST THE CURRENT LAYER.

```
function jump(nums) {
  let jumps = 0;
  let end = 0;
  let far = 0;
  for (let i = 0; i < nums.length - 1; i++) {
    far = Math.max(far, i + nums[i]);
    if (i === end) {
      jumps += 1;
      end = far;
    }
  }
  return jumps;
}
```

Update order: extend `far` on every `i`, including the index that equals `end`. Then, if `i === end`, add one jump and set `end = far`. If you moved `end` before you folded this index into `far`, you would close the layer without the leap from the edge. The loop condition is `i < n - 1`. Running through the last index takes a jump the last stone does not need. On this array that extra jump would return 3.

The gas tank

If the total gas is at least the total cost, a circuit exists. The start is the index after the last time the running tank went negative. Any earlier start dies at that same failure, because the tank arrived there even lower. If the total is negative, return -1. The local reset does not create fuel.

Take gas `[1, 2, 3, 4, 5]` and cost `[3, 4, 5, 1, 2]`. The gains are -2, -2, -2, 3, 3. At index 0 the tank is -2, so every start up through 0 fails. Set `start` to 1 and the tank to 0. Index 1 fails the same way, and `start` becomes 2. Index 2 fails, and `start` becomes 3. Index 3 gains 3. Index 4 gains 3. The total gain is 0, which is not below 0, so a circuit exists. The start is 3, the index after the last negative tank. From 3 the gains are 3, 3, -2, -2, -2. The tank never goes negative on that circuit.

A NEGATIVE TANK MEANS EVERY INDEX UP TO I FAILS AS A START.

```
function canCompleteCircuit(gas, cost) {
  let total = 0;
  let tank = 0;
  let start = 0;
  for (let i = 0; i < gas.length; i++) {
    const gain = gas[i] - cost[i];
    total += gain;
    tank += gain;
    if (tank < 0) {
      start = i + 1;
      tank = 0;
    }
  }
  return total < 0 ? -1 : start;
}
```

Update order: add the gain into `total` and into `tank` first. If the tank is then negative, set `start` to `i + 1` and set the tank back to 0. Resetting before you add the gain hides the failure at `i`. After the loop, if `total` is below 0 return -1. Otherwise return `start`.

Flowers

A slot is free when it is 0 and both neighbors are 0 or off the end of the bed. Plant it, so the next check sees the new flower. Count the plant against `n`. Stop early when you have planted `n`. You may mutate the bed.

Take the bed `[1, 0, 0, 0, 1]` and `n = 1`. Index 0 is already 1. Index 1 is 0, but its left neighbor is 1, so it is not free. Index 2 is 0, its left neighbor is 0, and its right neighbor is 0. Plant it. The bed becomes `[1, 0, 1, 0, 1]`, and `n` drops to 0. You stop. The answer is true.

If `n` were 2, you would have planted one flower and still need another. Index 3 is 0, but its left neighbor is now 1. Index 4 is 1. You cannot plant a second flower. The answer is false.

One edit

Walk from the left. When `nums[i]` is less than `nums[i - 1]`, you must spend your one change. If `i` is 1, or `nums[i]` is at least `nums[i - 2]`, lower `nums[i - 1]` down to `nums[i]`. That keeps the pair before `i` legal. Otherwise raise `nums[i]` up to `nums[i - 1]`. Lowering the previous cell would break the cell two steps back. A second violation returns false.

Take `[4, 2, 3]`. At index 1, 2 is less than 4. This is the first pair, so you lower index 0 down to 2. The array becomes `[2, 2, 3]`. There is no second drop. The answer is true.

Take `[3, 4, 2, 3]`. At index 2, 2 is less than 4. Index 2 is not the first pair. `nums[2]` is 2 and `nums[0]` is 3, so 2 is below the cell two steps back. You raise index 2 up to 4. The array becomes `[3, 4, 4, 3]`. At index 3, 3 is less than 4. That is a second violation. The answer is false. If you had lowered the 4 instead, the array would be `[3, 2, 2, 3]`, and the new drop sits at index 1, where you have already spent the only change.

Time and extra memory

Jump I, jump II, the gas circuit, the flower bed, and the one edit are all O(n) time and O(1) extra memory. You may mutate the flower bed and the array you are making non-decreasing.

Mistakes

- On [3, 2, 1, 0, 4], if you extend reach before you test i greater than reach, index 4 sets reach to 8 before you notice you cannot stand there. The function returns true. The jump is illegal.
- On [2, 3, 1, 1, 4], if the jump-II loop runs through the last index, you take a jump when i hits the end of the last layer. You return 3. The loop ends at n - 2, and the answer is 2.
- On gas [1, 2, 3, 4, 5] and cost [3, 4, 5, 1, 3], the gains are -2, -2, -2, 3, 2. The tank resets and leaves a start of 3, but the total gain is -1. Returning 3 pretends the reset created fuel. total below 0 returns -1.
- On [3, 4, 2, 3], always lowering nums[i - 1] changes 4 to 2 and leaves [3, 2, 2, 3]. The new drop is a second violation. Compare nums[i] with nums[i - 2] before you choose which cell to edit.

Practice

Stay before chapter 17. Then before you leave the chapter.

55. Jump Game MEDIUM STAY
If i > reach, return false. Then reach = max(reach, i + nums[i]).

45. Jump Game II MEDIUM STAY
Current end of the layer, and far. One jump when i hits end. Loop to n − 2.

134. Gas Station MEDIUM STAY
Total gas < total cost returns −1. Otherwise start after the last negative tank.

605. Can Place Flowers EASY THEN
Plant when the slot and both neighbors are empty. Decrement n as you plant.

665. Non-decreasing Array MEDIUM THEN

One change. Compare nums[i] with nums[i − 2] before you choose which cell to edit.

REWRITE FROM MEMORY

Rewrite jump I and the gas-station reset. Say which cell the one-edit problem changes, and why the value two steps back decides.

Carry forward. Chapter 17 does not add a template. It asks you to name, from the wording alone, which chapter you just finished.

CHAPTER 17 · BUILDS ON CHAPTERS 0 THROUGH 16

Recognition sheet

Recognition is choosing the chapter before you write any code. You ask the questions in order, and you stop at the first one that fits.

What recognition is

Recognition is the choice of a chapter. You make that choice before you write code. You are not hunting for a new trick. You are matching the shape of the prompt to a template you already own.

You read the prompt once. Then you ask a fixed list of questions, from the top. The first question that fits is the chapter you use. You stop. A later question can also sound clever. You leave it. The first fit is the tool that was built for this constraint. A later tool usually forgets that constraint.

You say the chapter number out loud. Then you say what the slow version would cost: every pair, every band, or every row. Then you write the template from that chapter. If your hands are typing a loop before the number, you are guessing from a word you have seen before.

The same word sits in several chapters. Sum can be a window, a prefix, or Kadane. Sorted can be two ends, or a binary search. The questions exist so the shared word does not decide.

You will sometimes say yes too early. The repair is to name the question you accepted, and to read the next sentence of that question. A wrong chapter is a wrong yes. It is not a missing formula.

These are the numbers you are choosing. Chapter 2 is one pass. Chapter 3 is two loops. Chapter 4 is the map. Chapter 5 is pointers from the ends. Chapter 6 is a read index and a write index. That write is in-place: you change the given array, and you do not build a second array of the same length. Chapter 7 is the window. Chapter 8 is the prefix. Chapter 9 is the difference array. Chapter 10 is two passes. Chapter 11 is Kadane. Chapter 12 is sort, then scan. Chapter 13 is placement. Chapter 14 is binary search. Chapter 15 is the stack. Chapter 16 is the greedy scan. Chapters 18 through 25 are the matrix: indexing, the walk, in-place marks, the two-dimensional prefix, search in a sorted matrix, regions, the order you fill cells, and a four-week review. You can name a later chapter before you have studied its template. The number is the decision. The code comes after.

How to walk the questions

Start at the first question. If the input is a grid, you stay inside that question and you pick the branch that names the job. You do not drop into the array questions while you are still holding a matrix. If the input is not a grid, you go to the next question and you do the same thing: read it all, including the second sentence, then stop or move on.

A few names in the list, so the words are not bare. A window is a contiguous band. You grow it from the right, and you shrink it from the left. A prefix sum at index i is the total of every value from index 0 through i. A half-open range keeps the low end and excludes the high end. Kadane keeps a running sum, and throws that sum away when it is negative, because a negative start only lowers what comes after.

When a question fits, you stop. You say the chapter number. You say the brute cost. Then you code.

1. If the input is a grid, stay on this question and choose the job. If the job is a region, a flood, or a distance in layers (the number of steps out from a start cell), use chapter 23. If a cell is the sum or the min of earlier cells, use chapter 24. If you sum a rectangle, use chapter 21. If each row and each column is sorted, or each row is sorted and the rows themselves are ordered, use chapter 22. If you walk, rotate, or spiral, use chapter 19. If the next state has to be stored in the cell, use chapter 20. Otherwise you are still indexing, and you use chapter 18.
2. Does each index want the nearest greater or smaller neighbor, a span, a histogram rectangle, or the max of a window after the max can leave? Use a stack or a deque, chapter 15. A deque is the double-ended queue that

chapter 15 uses when the window maximum can fall out of the band.

3. Are the values in 1 through n or 0 through n, and is the question a missing value, a duplicate, or a first positive? Place each value at its index, chapter 13. A single number among pairs is XOR, the chapter 13 bit trick, and that is still chapter 13.
4. Is the thing you search a speed, a capacity, a day, or a boundary in a sorted array? Use the half-open loop, chapter 14.
5. Do you want the maximum sum of a contiguous segment, with the empty segment forbidden, with negatives allowed, and with no other constraint? Use Kadane, chapter 11.
6. Do you add a value onto many ranges and then read the array? Use a difference array, chapter 9.
7. Do you need a range sum, or the count of subarrays with sum K, and can a negative repair a sum? Use a prefix map, chapter 8.
8. Is the band contiguous, and does growing it only move you toward invalid while shrinking repairs it? Use a window, chapter 7.
9. Is the array sorted, or are you comparing two ends, so that one end can be discarded forever? Use pointers from the ends, chapter 5.
10. Are you overwriting the array, with a read index and a write index that both move right? That is the in-place write from chapter 6.
11. Do you need every pair in order, with duplicate triples removed, or are the items intervals? Sort, then scan, chapter 12.
12. Are you asking whether you have seen the complement, on unsorted data? Use a map, chapter 4.
13. Is n small enough that every pair or every subarray is acceptable? Use two loops, chapter 3.
14. Does the answer at i need only nums[i] and a summary of one side? Use one pass, chapter 2. A reach, a jump layer (the frontier of indexes you can still land on), or a tank is the greedy scan, chapter 16. Two full sides, left and right, are chapter 10.

The list gives you one chapter. The pairs below are the places where a shared word makes you rush the next question. Read the pair that matches the word you almost said yes to.

The distinctions

Window and prefix. Use a window when adding the next element only moves the range toward invalid, and dropping the left end repairs it. The values that the condition cares about are non-negative, or they are counts you can undo. Use a prefix map when a negative value can repair a sum you have already walked past, or when you need the number of subarrays whose sum is K. The window throws the left end away and never needs that end again. The map keeps every earlier prefix sum, because a later negative can still make an old total equal K.

Ends and window. Use pointers from the ends when the array is sorted, or you are comparing two ends, and each step can discard one end forever. Use a window, rather than those ends, when both indices only move right and the answer is a contiguous band. The two ends move toward each other, and each move deletes one side for good. A window's two indices both move only forward, and the cells between them are the band you are judging.

Kadane and window. Use Kadane when you want the best sum of a non-empty contiguous segment and a negative prefix should be thrown away. Use a window for a sum only when a separate bound exists and the values are non-negative. Kadane has no second bound such as "at most K". The only decision is whether the sum so far helps the next cell. A window has that second bound, and it can repair the bound by dropping the left end, which Kadane never does.

Map and sort. Use a hash map when the pair is unsorted and you need a complement in expected linear time. The complement of x is the value that finishes the pair, usually the target minus x. Use a sort when you need every pair in order, duplicates skipped, or the objects are intervals, and n log n is acceptable. The map answers

one question: have I seen the partner? The sort puts equal values together, so a scan can skip duplicates, and it puts intervals in an order a single pass can merge.

Index search and answer search. Use binary search on an index when a sorted half tells you which side cannot hold the target. You are searching for a position that already exists in the array. Use binary search on the answer when feasibility is monotone: a faster speed, a larger ship, a later day, and you can test one candidate in linear time. Monotone means the yes and no answers do not flip back and forth. Once a speed is fast enough, every faster speed is fast enough too. You are searching a number you chose, and the array is only the test.

Stack and a right-to-left max. Use a stack when the answer at i is the nearest greater element, and a popped index can never be the answer for anyone later. Use a plain right-to-left pass when the answer is the greatest element anywhere to the right, not the nearest one. That pass carries one champion as it walks backward. The stack carries every index that can still be the nearest greater value for a cell you have not reached. When a taller value arrives, the shorter indexes behind it are finished, so you pop them.

Cyclic placement and a set. Cyclic placement writes each value onto the index that value names, then follows that cycle until the value is home or a duplicate is already there. For a value v in 1 through n, the home index is v minus 1. Use cyclic placement when each value in 1..n has a home index. Use a set when you only need membership and the values do not fit in the index range, or you must not write into the array.

Answer key for the eight drills

These eight lines are the drills from chapter 1. Cover the table until you have written a chapter number for each line. Then look. A miss means you said yes to the wrong question above, so go back to that question and read the branch you skipped.

DRILL	USE	CHAPTER
Longest subarray, sum ≤ K, all positive	Variable window, shrink while the sum is too big	**07**
Count subarrays with sum K, negatives allowed	Prefix frequency, seed 0, query before update	**08**
Two indices, sorted, sum to target	Pointers from the ends	**05**
Remove duplicates from a sorted array, in place	Read r, write w	**06**
Next warmer day	Monotonic stack of indices, answer is the distance	**15**
Number of islands	Grid search, mark a cell when you enter it	**23**
Add a value on many ranges, then read	Difference array, +v at L and −v at R+1	**09**
Minimum eating speed	Binary search the speed, test hours in linear time	**14**

Twelve prompts

Do these twelve the same way. Write the chapter number next to the prompt. Write the pattern name if you want a second word. Do not write code. Code would only practice a template you have already chosen, and this page is the choice. The names are on the next page. Leave that page covered until all twelve numbers are written down.

1. A string of 0s and 1s. You may flip at most K zeros. What is the longest all-ones stretch you can produce?
2. Values may be negative. How many contiguous segments sum to exactly K?
3. A sorted array. Find two indices that add to a target.
4. Daily prices. One buy and one later sell. Maximum profit.
5. Unsorted integers. The longest streak of consecutive values. The order in the array does not matter.
6. Split the array into at most M contiguous parts. Minimize the largest part sum.
7. For each day, how many days until a strictly warmer temperature?
8. Add v to every index in many inclusive ranges, then print the array.

9. A grid of land and water. How many connected groups of land are there, 4-directionally?
10. The contiguous segment with the largest sum. Negatives exist. The empty segment is forbidden.
11. Each row of a matrix is sorted, and the first cell of a row is greater than the last cell of the row above. Is target present?
12. A sorted array. Remove duplicates in place so each remaining value appears once.

Stop here. The next page is the key, and it starts on a new page so a glance does not give you the number.

Pattern names

Uncover this only after the twelve prompts have a written answer.

1. Sliding window, at most K zeros. Chapter 7.
2. Prefix frequency map. Chapter 8.
3. Pointers from the ends. Chapter 5.
4. One pass, minimum price so far, profit before the update. Chapter 2.
5. Set, start a run only when x − 1 is absent. Chapter 4.
6. Binary search on the answer. Chapter 14.
7. Monotonic stack. Chapter 15.
8. Difference array. Chapter 9.
9. Depth-first search, mark on entry. Chapter 23.
10. Kadane. Chapter 11.
11. Binary search on the flat index. Chapter 22.
12. Same-direction read and write. Chapter 6.

REWRITE FROM MEMORY

Close the page and walk the twelve prompts again. You are done when the chapter number comes out before any code does.

Carry forward. Chapter 18 starts the matrix. The index becomes a row and a column, and the shared-array bug from chapter 0 becomes visible.

CHAPTER 18 · BUILDS ON CHAPTER 0

The matrix from zero

A matrix is an array of rows. You name a cell by its row and its column, and you give each row its own array.

What this pattern is

A matrix is an array of arrays. One index was enough for a list. A cell in a grid needs two. A row is one horizontal line of cells. A column is one vertical line of cells.

You write `matrix[r][c]` for the cell in row r and column c. The first index is the row. The second index is the column. There are `m = matrix.length` rows. Once you know the matrix is not empty, there are `n = matrix[0].length` columns. The last row index is m minus 1. The last column index is n minus 1. You read `matrix.length` before you touch row 0, because an empty matrix has no row 0.

What you already know

Chapter 0 said an array is an object. Two names can point at the same array. You have written a loop, you have used a length, and you have checked an index before a read. A matrix adds the second index. The object rule is the part that surprises people, and the next section shows the write that reveals it.

A worked example

Take a grid with two rows and three columns. Row 0 is 1, 2, 3. Row 1 is 4, 5, 6. So m is 2 and n is 3. The last row index is 1. The last column index is 2.

You touch `matrix[0][0]` when you want the top-left cell. That index is row 0, column 0, and the value is 1. You touch `matrix[1][2]` when you want the bottom-right cell. You go down one row, then across two columns, and the value is 6. A row-major walk finishes a row before it starts the next one. The indexes are `[0][0]`, `[0][1]`, `[0][2]`, then `[1][0]`, `[1][1]`, `[1][2]`. A column-major walk holds the column still and moves the row. Column 0 is `[0][0]` then `[1][0]`, values 1 then 4, before you ever touch column 1.

The neighbor below `matrix[0][1]` (the 2) is `matrix[1][1]` (the 5). You add 1 to the row and you leave the column alone. The neighbor above that same cell would be row minus 1. That row is not in the grid, so you do not read it.

Now build two rows of three zeros, and watch the object rule. `Array(3).fill(0)` builds one inner array. `Array(2).fill` does not call that builder twice. It copies the same array into both slots of the outer array. `broken[0]` and `broken[1]` are two names for one row.

Write `broken[0][0] = 9`. You meant to change row 0, column 0. Because both slots share that one inner array, row 1 column 0 becomes 9 as well. You can read it: `broken[1][0]` is 9.

The repair calls a function once per row. `Array.from({ length: 2 }, ...)` runs that function for row 0, then again for row 1, and each run builds a new `Array(3).fill(0)`. After `rows[0][0] = 9`, row 0 column 0 is 9 and `rows[1][0]` is still 0.

ONE INNER ARRAY, SHARED, VERSUS A FRESH INNER ARRAY PER ROW.

```
const broken = Array(2).fill(Array(3).fill(0));
broken[0][0] = 9;
// broken[1][0] is 9 as well. fill copied one reference.

const rows = Array.from({ length: 2 }, () => Array(3).fill(0));
rows[0][0] = 9;
// rows[1][0] stays 0. Each call builds a new inner array.
```

After the first snippet, both rows show 9 in column 0. After the second, only row 0 does. The order is the whole bug: `fill` copies a reference it already has, and `Array.from` builds a new row on every call.

How you recognise it

The input is a grid of numbers, not one list. The job is a sum along a row, a check along a diagonal, a count of cells, or a copy into a new shape. Plain row order is enough. You are not peeling a frame, and you are not storing a second state inside the cell. If the prompt says spiral, rotate, or diagonal order, the walk itself is the problem. If the cell must remember a next state, the mark is the problem. If many rectangles want a sum, the prefix grid is the problem.

- Use chapter 18 when you index cells, sum a row, check a diagonal, reshape, or count a perimeter.
- Use chapter 19 when the order is a spiral, a rotation, or a diagonal walk.
- Use chapter 21 when the same still matrix answers many rectangle sums.

What to say out loud

An invariant is a fact that stays true at the end of every step. Yours is: every row I allocate is a new array. I read the number of rows before I touch row 0. I check the row and the column against m and n before I read a neighbor.

The template

ROW-MAJOR, COLUMN-MAJOR, THE MAIN DIAGONAL, AND THE FOUR NEIGHBORS.

```
function eachRow(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      const cell = matrix[r][c];
    }
  }
}

function eachColumn(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let c = 0; c < n; c++) {
    for (let r = 0; r < m; r++) {
      const cell = matrix[r][c];
    }
  }
}

function mainDiagonal(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  for (let i = 0; i < Math.min(m, n); i++) {
    const cell = matrix[i][i];
  }
}

const DIRS = [[1, 0], [-1, 0], [0, 1], [0, -1]];

function inBounds(r, c, m, n) {
  return r >= 0 && c >= 0 && r < m && c < n;
}
```

The invariant of these loops is that you never read an index until it sits inside the grid, and that each row array was created by its own call. Row-major puts the row loop outside and the column loop inside, so you finish row 0

of the example before you read the 4. Column-major swaps those loops. The column stays fixed while r runs, which is how you copy a column. If you swap the meaning and read `matrix[c][r]` while r is the one that moves, you walk a row, or you step off a grid that is not square.

The main diagonal uses one index for both positions, and it stops at the smaller of m and n. On the 2 by 3 grid that is i = 0 and i = 1: the 1, then the 5. There is no row 2, so the loop does not ask for one. On a square, the anti-diagonal is `matrix[i][n - 1 - i]`. For the square 1..9 that is 3, then 5, then 7. If you add both diagonals and n is odd, the center cell sits on both, so you added it twice and you subtract it once.

A Toeplitz diagonal is constant from the top-left toward the bottom-right. A cell that has a parent must equal `matrix[r - 1][c - 1]`. In a grid whose first row is 1, 2 and whose second row is 3, 1, the cell at row 1 column 1 is 1 and its parent is 1, so that pair agrees. If that cell were 4, it would disagree with the 1 above-left, and the grid would not be Toeplitz.

The four directions are down, up, right, and left: add 1 or subtract 1 from the row, or add 1 or subtract 1 from the column. You call `inBounds` before the read. The neighbor of a border cell is often not a cell.

Perimeter

Island perimeter is a count, not a search. Each land cell starts with 4 sides. If the cell above is also land, both cells counted that shared edge, so you subtract 2. You do the same when the cell to the left is land. You do not look down, and you do not look right. The cell below, and the cell to the right, will subtract that edge when you reach them.

Two land cells side by side, with water everywhere else, show the count. The left land has no land above it and no land to its left, so it contributes 4. The right land has land on its left, so it contributes 4 minus 2, which is 2. The total is 6, the outline of a domino. If you subtracted for all four directions, both cells would subtract the same edge and you would report 4.

Reshape

Reshape uses a flat index. You count cells in row-major order, starting at 0. Chapter 22 searches this same numbering. You meet it here, before the search. Cell number i in the old matrix sits at row `floor(i / n)` and column `i % n`, where n is the old width. In the new matrix it sits at row `floor(i / c)` and column `i % c`, where c is the new width.

Take [[1, 2], [3, 4]] and a new shape of 1 row by 4 columns. i = 0 is old row 0 column 0, value 1, and it lands at new column 0. i = 1 is 2, and it lands at new column 1. i = 2 is old row 1 column 0, value 3, because `floor(2 / 2)` is 1 and `2 % 2` is 0. It lands at new column 2. i = 3 is 4, at new column 3. The result is one row: 1, 2, 3, 4. If the new row count times the new column count is not m times n, the cells do not match, and you return the original matrix. A request for 3 columns cannot be filled by 4 cells, so you refuse.

Count negatives in a sorted matrix can be a double loop on this pass. Each row and each column is non-increasing. Chapter 22 walks that grid from a corner and does not revisit a cell. The double loop is the right tool until you have practiced that staircase.

Time and extra memory

Touching every cell takes time proportional to m times n. A new matrix takes that much extra memory, and each of its rows must be its own array. A read that only keeps a running total uses a handful of numbers besides the input. The shared-row line does not save a legal allocation. It aliases the rows.

Mistakes

On a 2 by 3 zero grid, `Array(2).fill(Array(3).fill(0))` then `broken[0][0] = 9` also stores 9 at row 1 column 0. The write hit one object, and both rows are that object.

An empty matrix has length 0. Reading `matrix[0].length` asks for row 0, and there is no row 0, so the read throws. You check `matrix.length` first and you return before you ask for a column count.

Copying column 0 of the 2 by 3 example by walking c while you read `matrix[c][0]` pulls 1, then 2, then 3. That is row 0, not column 0. Column 0 is 1 then 4, which you get only while r changes and c stays 0. On a non-square grid the swapped indexes also walk off the edge.

From row 0 column 0, the up neighbor is row minus 1. Without `inBounds`, that read is not a cell. The same miss happens on the right edge when you add 1 to a column that is already n minus 1.

Practice

Stay before chapter 19. Then before you leave the chapter.

1672. Richest Customer Wealth EASY STAY
Each row is a customer. Sum the row, keep the max.

1572. Matrix Diagonal Sum EASY STAY
Add both diagonals. On an odd n, the center was added twice. Subtract it once.

766. Toeplitz Matrix EASY STAY
Every cell with a parent must equal matrix[r - 1][c - 1].

566. Reshape the Matrix EASY STAY
Refuse when the counts differ. Map flat i through the old width and the new width.

1380. Lucky Numbers in a Matrix EASY THEN
A lucky number is the min of its row and the max of its column. Brute force is appropriate.

118. Pascal's Triangle EASY THEN
Row 0 is [1]. Each later cell is the sum of the two parents, and a missing parent is 0.

463. Island Perimeter EASY THEN
4 per land, minus 2 for each land neighbor above or to the left.

1351. Count Negative Numbers in a Sorted Matrix EASY THEN
A double loop is fine here. Chapter 22 revisits it as a staircase from the top-right.

REWRITE FROM MEMORY
From a blank file, write the fresh-row allocation and the four loops. Then on paper, set the corner of a shared-row matrix and write what the other row becomes.

Carry forward. Chapter 19 walks those cells in an order: a diagonal, a spiral, a transpose, a rotation.

CHAPTER 19 · BUILDS ON CHAPTER 18

Walk the matrix

You visit the cells in a chosen order. A spiral keeps four edges, and it skips a leg that would walk the same row or the same column again.

What this pattern is

You can already touch `matrix[r][c]`. This chapter is about the order of the visits. A spiral, a rotation, and a diagonal walk follow a shape. They do not finish row 0 and then row 1.

A layer of a spiral is the frame still inside four edges: the top row, the right column, the bottom row, and the left column that you have not walked yet. You peel one layer, then the frame inside it. The bug is visiting a cell twice when the matrix is a single row or a single column, because the next leg walks back over the leg you just finished.

What you already know

You can allocate a fresh row with `Array.from`. You can reverse a one-dimensional array. A transpose swaps `matrix[r][c]` with `matrix[c][r]`. You already know to test an index before you read it. Here the new test is whether a leg still has a row, or still has a column, before you walk it.

A worked example

Use this square. Row 0 is 1, 2, 3. Row 1 is 4, 5, 6. Row 2 is 7, 8, 9. Start with top 0, bot 2, left 0, and right 2. The first layer is the whole frame.

The rightward leg keeps the row at top and runs the column from left to right. You touch `matrix[0][0]`, `matrix[0][1]`, and `matrix[0][2]`, values 1, 2, 3. You do this because that top edge has not been taken. After the leg, you add 1 to top. Top is now 1. You do this before the downward leg, so the downward leg does not push the 3 a second time.

The downward leg keeps the column at right, which is still 2, and runs the row from the new top down to bot. You touch `matrix[1][2]`, the 6, then `matrix[2][2]`, the 9. After that leg you subtract 1 from right. Right is now 1. The old right column is outside the layer.

The leftward leg runs only if top is still less than or equal to bot. Here 1 is less than or equal to 2, so the leg runs. The row stays at bot, which is 2, and the column runs from the new right back to left. You touch `matrix[2][1]`, the 8, then `matrix[2][0]`, the 7. Then you subtract 1 from bot. Bot is now 1.

The upward leg runs only if left is still less than or equal to right. Here 0 is less than or equal to 1, so the leg runs. The column stays at left, and the row runs from the new bot up to top. You touch `matrix[1][0]`, the 4. Then you add 1 to left. Left is now 1.

One cell remains: row 1, column 1. Top 1, bot 1, left 1, and right 1 still form a layer, so the loop continues. The rightward leg touches `matrix[1][1]`, the 5. You then add 1 to top, and top is 2. The downward range from 2 to bot 1 is empty, and right becomes 0. Top is no longer less than or equal to bot, so the leftward leg does not run, and you do not walk the 5 backward. The output is 1, 2, 3, 6, 9, 8, 7, 4, 5.

Now a single row, [1, 2, 3]. Top and bot are both 0. Left is 0. Right is 2. The rightward leg pushes `matrix[0][0]`, `matrix[0][1]`, and `matrix[0][2]`. The output is 1, 2, 3. After that leg, top becomes 1. Top is past bot.

The downward loop runs from row 1 to row 0. That range is empty, so it pushes nothing. Then right becomes 1. The leftward leg runs only if `top <= bot`. Here 1 is greater than 0, so you skip it. If you ran it anyway, the row is bot, which is still 0, and the column would run from 1 down to 0. You would push 2, then 1. The row would be in the output a second time, walked backward.

The upward leg runs only if `left <= right`. Here 0 is still less than or equal to 1, so the leg is entered. Its row range runs from bot 0 down to top 1. That range is empty, so it pushes nothing. Then left becomes 1. The outer test fails because top is past bot. You stop. The output stays 1, 2, 3. The row was not walked backward.

How you recognise it

You are given a matrix, and the answer is a sequence of cells, or the matrix turned. A spiral means the outside frame, then the frame inside it. A clockwise quarter turn is a transpose, then a reverse of each row. Diagonal order alternates an up-right step and a down-left step, and the wall you test first decides the corner. A score that flips rows, then flips whole columns, is the same family: you fix column 0, then you choose each later column. A plain sum of each row never needs these edges.

- Use chapter 19 when the visit order is the problem: a spiral, a rotation, or a diagonal.
- Use chapter 18 when a row-by-row loop already visits the cells you need.
- Use chapter 20 when you encode a new state in the cell, instead of collecting cells into a list.

What to say out loud

The invariant is that top, bot, left, and right are the edges of the layer I have not finished. Every cell already in the output sits outside that rectangle. I move an edge as soon as its leg is done. I run the leftward leg only while top is less than or equal to bot, and the upward leg only while left is less than or equal to right.

The template

THE LAST TWO LEGS ARE GUARDED, SO A SINGLE ROW IS NOT WALKED BACKWARD.

```
function spiralOrder(matrix) {
  const out = [];
  let top = 0;
  let bot = matrix.length - 1;
  let left = 0;
  let right = matrix[0].length - 1;
  while (top <= bot && left <= right) {
    for (let c = left; c <= right; c++) out.push(matrix[top][c]);
    top += 1;
    for (let r = top; r <= bot; r++) out.push(matrix[r][right]);
    right -= 1;
    if (top <= bot) {
      for (let c = right; c >= left; c--) out.push(matrix[bot][c]);
      bot -= 1;
    }
    if (left <= right) {
      for (let r = bot; r >= top; r--) out.push(matrix[r][left]);
      left += 1;
    }
  }
  return out;
}
```

```
1 2 3
4 5 6
7 8 9
right along top → 1 2 3, top becomes 1
down the right → 6 9, right becomes 1
left along bot, top 1 <= bot 2 → 8 7, bot becomes 1
up the left, left 0 <= right 1 → 4, left becomes 1
right along the middle row → 5
[1, 2, 3, 6, 9, 8, 7, 4, 5]

a single row [1, 2, 3]: the rightward leg takes all three,
top moves past bot, and the leftward leg is skipped
```

The order of the updates is the invariant's proof. You push the top row from left to right, then you add 1 to top. That add happens even when the row you just pushed was the only row. You push the right column from the new top down to bot, then you subtract 1 from right. Those two updates are not inside a test. The leftward leg runs only if top is still less than or equal to bot, and only then do you subtract 1 from bot. The upward leg runs only if left is still less than or equal to right, and only then do you add 1 to left. On the single row above, top moves past bot after the rightward leg, the leftward test fails, and [1, 2, 3] is not walked backward.

Spiral II writes these same four legs into a fresh square, placing the numbers 1 through n squared. You allocate that square with `Array.from`, one new row per call. A shared inner array would make a write on row 0 appear on every row.

Rotate

A clockwise quarter turn of a square is a transpose, then a reverse of each row. You swap only when c is greater than r, which the inner loop does by starting c at r + 1. A swap with c less than r would undo the pair you already swapped. A swap with c equal to r would swap a diagonal cell with itself.

Start from the same 1..9 square. At row 0 column 1 you swap `matrix[0][1]` with `matrix[1][0]`. The 2 and the 4 trade, so row 0 begins 1, 4, 3 and row 1 begins 2. At row 0 column 2 you swap `matrix[0][2]` with `matrix[2][0]`. The 3 and the 7 trade. At row 1 column 2 you swap `matrix[1][2]` with `matrix[2][1]`. The 6 and the 8 trade. The diagonal cells stay where they are: 1, 5, and 9. The grid is now 1 4 7 / 2 5 8 / 3 6 9.

Then you reverse each row. Row 0 becomes 7, 4, 1. Row 1 becomes 8, 5, 2. Row 2 becomes 9, 6, 3. The old top row 1, 2, 3 is now the right column, from top to bottom. That is the clockwise turn.

TRANSPOSE ACROSS THE MAIN DIAGONAL, THEN REVERSE EACH ROW.

```
function rotate(matrix) {
  const n = matrix.length;
  for (let r = 0; r < n; r++) {
    for (let c = r + 1; c < n; c++) {
      const tmp = matrix[r][c];
      matrix[r][c] = matrix[c][r];
      matrix[c][r] = tmp;
    }
  }
  for (const row of matrix) row.reverse();
}
```

```
1 2 3       1 4 7       7 4 1
4 5 6   →   2 5 8   →   8 5 2
7 8 9       3 6 9       9 6 3
transpose swaps (r, c) with (c, r) only when c > r
reverse each row
```

The update order is transpose first, reverse second. During the transpose you only visit c greater than r, so each pair moves once and the diagonal is never written. The reverse then runs left to right on each finished row. If you reversed first, you would be reversing a grid that is not yet transposed, and the quarter turn would be the other way.

Diagonal order

Diagonal order alternates an up-right step and a down-left step. On the 1..9 square the visit order is 1, 2, 4, 7, 5, 3, 6, 8, 9. You start at row 0 column 0, heading up-right. You are on the top edge and not on the right edge, so you step to column 1 and head down. That cell is 2.

Going up, you test the right edge before the top edge. The cell 3 is row 0 column 2, the top-right corner, and you are heading up. Both edges are true. The right-edge test hits first, so you step down to row 1 column 2, the 6, and you head down. If you tested the top first, you would add 1 to the column and land on column 3, which is not a cell.

Going down, you test the bottom edge before the left edge. A 2 by 2 grid, 1 2 / 3 4, shows the corner the 3 by 3 path does not decide. After 1 and 2 you are at row 1 column 0, the 3, heading down. That cell is the bottom edge and the left edge. The bottom test steps you right, onto the 4. The left test would add 1 to the row and step off the board.

You do those tests in that order on every step, not only at the corner you happen to remember. The far edge of the current direction comes first. The near edge comes second. The ordinary diagonal step runs only when both tests fail.

Flip each row

Flipping an image reverses each row and inverts each bit. You can do both in one swap from the ends. The new left value is the old right value XOR 1. The new right value is the old left value XOR 1. XOR 1 flips a bit: 0 becomes 1, and 1 becomes 0. When the two indexes meet, you XOR that single cell and you do not swap it with itself twice.

On [1, 1, 0], the ends are index 0 and index 2. The new left is 0 XOR 1, which is 1. The new right is 1 XOR 1, which is 0. The middle index meets itself, and 1 XOR 1 is 0. The row becomes [1, 0, 0]. That matches a reverse to [0, 1, 1] followed by an invert.

Score after flipping

The first column is worth the most, so you make it all ones before you touch the later columns. Flip any row that starts with 0. That flip also flips every later bit in the row. A later column can then be flipped as a whole.

Take a 2 by 2 grid whose rows are 0, 1 and 1, 0. Row 0 starts with 0, so you flip it and it becomes 1, 0. Row 1 starts with 1, so you leave it as 1, 0. Column 0 is now all ones. Its place value is 2 to the power of (n minus 1 minus the column), which is 2. Both rows contribute, so column 0 adds 2 times 2, which is 4. That is m times the highest place, because column 0 is all ones.

A cell equals `grid[r][0]` exactly when the row flip you already chose leaves that cell as 1. In column 1, row 0's 1 does not equal the original row start 0, and row 1's 0 does not equal the original row start 1. The count of ones is 0. The complement, if you flip the whole column, is 2. You take the larger one, 2, times the place value 1, and you add 2. The total is 6. The grid you scored is 1, 1 / 1, 1, whose rows are 3 and 3.

Time and extra memory

A spiral, a diagonal walk, and a score each touch every cell a constant number of times, so the time is proportional to m times n. The spiral's output list is the size of the grid, and the extra memory besides that list is a few edges. Rotate reverses and swaps inside the given square, so the extra memory is a handful of indexes. Spiral II allocates a fresh n by n square, which is the output the problem asks for.

Mistakes

Drop the leftward test on the single row [1, 2, 3]. The rightward leg still pushes 1, 2, 3, and top still becomes 1. Right then becomes 1. The unguarded leftward leg walks row 0 from column 1 down to column 0 and pushes 2, then 1. The output is 1, 2, 3, 2, 1. The row was emitted, then walked backward.

On the diagonal walk, test the top edge before the right edge at row 0 column 2 of the 1..9 square. You are heading up, the top test hits, and you step to column 3. That column is not a cell. You leave the board, or you loop.

Transpose the 1..9 square with a loop that also visits c less than r. Each pair swaps twice and returns home, so the transpose does nothing. The following reverse then only mirrors each row: 3 2 1 / 6 5 4 / 9 8 7. That is not the clockwise turn 7 4 1 / 8 5 2 / 9 6 3.

Build the spiral-II square with one shared inner array. The first write, a 1 at row 0 column 0, also writes row 1 column 0 and row 2 column 0. Later legs read those aliased cells and the numbers are already wrong.

Practice

Stay before chapter 20. Then before you leave the chapter.

54. Spiral Matrix MEDIUM STAY
Four legs. Guard the leftward and upward legs with the updated edges.

48. Rotate Image MEDIUM STAY
Transpose, then reverse each row. Square matrix, in place.

498. Diagonal Traverse MEDIUM STAY
Up-right and down-left. Hit the far edge of the current direction before the near edge.

867. Transpose Matrix EASY THEN
out[c][r] = matrix[r][c]. The result has n rows and m columns. Allocate it fresh.

832. Flipping an Image EASY THEN
Reverse and invert in one end-to-end swap. XOR 1 flips a bit.

59. Spiral Matrix II MEDIUM THEN
The same four legs, writing the next integer. One row and one column guards still apply.

861. Score After Flipping Matrix MEDIUM THEN
Force column 0 to ones by row flips. Then each later column takes the majority bit.

REWRITE FROM MEMORY
Rewrite the spiral, including both guards, and the rotate: transpose only above the diagonal, then reverse each row.

Carry forward. Chapter 20 writes a mark into the cell itself, because you are not allowed a second matrix.

CHAPTER 20 · BUILDS ON CHAPTERS 18 AND 19

In-place marks

You store a second fact in the cell or in the first row. You do not erase a mark while a later cell still has to read it.

What this pattern is

In-place means you change the matrix you were given. You do not allocate a second matrix of the same size. The cell then has to hold two facts: the value it has now, and the value it will have, or the mark a later cell still needs to read.

Sometimes the second fact is a bit inside the number. Sometimes it is a sign. Sometimes the first row and the first column are a bulletin board, and one extra boolean holds the fact those cells cannot hold.

What you already know

You can visit every cell, and you can allocate a fresh grid when the problem allows it. Some problems forbid that second grid. You already know that a shared row is one object. Here the danger is different: you erase a mark that a cell you have not reached still has to read.

A worked example

Set every cell to 0 when its row or its column contains a 0. Use constant extra memory. The grid starts as three rows of 1, 1, 1, except row 1 column 2, which is 0. That zero is not in row 0, and it is not in column 0. Call the rows [1, 1, 1], [1, 1, 0], and [1, 1, 1].

The first pass stores marks. Column 0 will mean "this row must become zeros." Row 0 will mean "this column must become zeros." Column 0 cannot do both jobs. The cell `matrix[0][0]` would have to mean "row 0 dies" and also "column 0 dies," and one cell cannot say both. A separate boolean, col0, remembers whether column 0 itself contained a zero.

Row 0 has no zero. col0 stays false, and you write no marks. Row 1 column 0 is 1, so col0 stays false. Row 1 column 2 is 0, so you set `matrix[1][0] = 0` and `matrix[0][2] = 0`. You touch column 0 of this row because the row must die. You touch row 0 of column 2 because the column must die. Row 2 has no zero. The marked grid is [1, 1, 0], [0, 1, 0], [1, 1, 1].

The write-back starts at the bottom-right, so the rows below row 0 still see `matrix[0][c]`. On row 2, column 2 reads `matrix[0][2]`, which is 0, so `matrix[2][2]` becomes 0. Column 1 reads `matrix[0][1]`, which is still 1, and `matrix[2][0]` is 1, so `matrix[2][1]` stays 1. col0 is false, so you do not write column 0 from the flag. Row 2 is now [1, 1, 0].

On row 1 you read the row mark `matrix[1][0]` before you touch column 0. The mark is 0, so column 2 and column 1 both become 0. Only after those reads do you apply the flag. The flag is false, and the row mark stays 0, which is what this row needs. Row 1 is now [0, 0, 0].

Row 0 is last. `matrix[0][0]` is still 1, so the row itself does not force every column to die. Column 2 is already 0 because that cell is the column mark. Column 1 stays 1. The flag does not zero `matrix[0][0]`. The finished grid is [1, 1, 0], [0, 0, 0], [1, 1, 0]. Row 1 died because it held the zero. Column 2 died because it held the zero. The 1 at row 2 column 1 shares neither, so it stays.

If you zero row 0 first, those column marks vanish. The marked row 0 is [1, 1, 0]. Clearing it makes [0, 0, 0]. `matrix[0][1]` was 1, which meant column 1 lives. It is now 0. Row 2 then sees a mark on column 1 and sets `matrix[2][1]` to 0. The wrong grid is [0, 0, 0], [0, 0, 0], [1, 0, 0]. The 1 that should have survived at row 2 column 1 is gone, and row 0 column 0 is gone with it.

How you recognise it

The problem tells you to change the board, and the extra memory is constant, or a second board would only copy what the cell can remember. Set zeroes uses the first row and the first column as the board of marks, plus one boolean. Game of life needs the old value for the neighbors and the new value for later, in the same cell. Battleships can be counted from the top-left cell of each ship, with no search. Sudoku validity is a set per row, per column, and per box. You are checking a filled board. A search that paints a whole region is a different chapter.

- Use chapter 20 when the second fact lives in the cell or in the first row, and a later read still needs the old fact.
- Use chapter 18 when a second matrix is allowed and the write is easier in that copy.
- Use chapter 23 when the mark means a whole region is visited and you search outward from a cell.

What to say out loud

The invariant is that I do not destroy a mark until every cell that still needs it has read it. For zeroes, the write starts at the bottom-right, and column 0 is written from the flag after the row mark is read. For life, neighbors read the low bit, and I shift only after the whole board is decided.

The template

COLUMN 0 HAS ITS OWN FLAG. THE WRITE-BACK STARTS AT THE BOTTOM-RIGHT.

```
function setZeroes(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  let col0 = false;
  for (let r = 0; r < m; r++) {
    if (matrix[r][0] === 0) col0 = true;
    for (let c = 1; c < n; c++) {
      if (matrix[r][c] === 0) {
        matrix[r][0] = 0;
        matrix[0][c] = 0;
      }
    }
  }
  for (let r = m - 1; r >= 0; r--) {
    for (let c = n - 1; c >= 1; c--) {
      if (matrix[r][0] === 0 || matrix[0][c] === 0) matrix[r][c] = 0;
    }
    if (col0) matrix[r][0] = 0;
  }
}
```

The first loop only writes marks. The inner column loop starts at 1, so a zero in column 0 sets the boolean and does not also pretend to be a row mark stored in row 0. The second loop starts at row m minus 1 and at column n minus 1. Inside a row you decide columns n minus 1 down through 1 while `matrix[r][0]` is still the row mark and `matrix[0][c]` is still the column mark. After that inner loop, and only then, you write `matrix[r][0]` from col0. Row 0 is the last row of this loop, so every lower row has already read it.

Life in two bits

Each cell is 1 when it is live and 0 when it is dead. The neighbors must keep seeing that old value while you decide the next one. The low bit is the old state. You read it with `board[r][c] & 1`, which keeps the bit worth 1 and clears everything else. You store the next state by `|= 2`, which turns on the bit worth 2 and leaves the low bit alone. You shift with `>> 1` only after the whole board is decided. The shift moves the new state into the low bit and drops the old one.

A live cell with 2 or 3 live neighbors stays live. A dead cell with 3 live neighbors becomes live. Every other cell is dead next. You only turn on the bit worth 2 in those two surviving cases. A cell you do not touch keeps a clear

next-bit, so the shift makes it 0.

Use a board whose left column is live and everything else is dead: [1, 0, 0], [1, 0, 0], [1, 0, 0]. The middle-left cell, row 1 column 0, is live. Its in-bound neighbors include the live cell above and the live cell below, so the count is 2. A live cell with 2 stays. You OR in 2. The cell becomes 3, and the low bit is still 1, so a later neighbor still sees it as live.

The dead cell at row 1 column 1 sees three live neighbors: above-left, left, and below-left. A dead cell with 3 becomes live. The cell was 0. You OR in 2. It becomes 2. The low bit is still 0. A neighbor that masks with `& 1` still sees dead, which is the old truth.

The top-left cell is live with one neighbor, the cell below, so it dies. You leave it as 1. The bottom-left cell dies the same way and stays 1. Every other cell stays 0. After the whole board is decided, the shift turns 3 into 1, 2 into 1, and 1 into 0. The result is [0, 0, 0], [1, 1, 0], [0, 0, 0].

Do not shift a row early. If you shift row 0 before row 1 is scored, the top-left 1 becomes 0. Row 1 column 0 then counts only the live cell below. The count is 1, a live cell with 1 neighbor dies, and the final shift makes it 0. It should have stayed live. Row 1 column 1 now counts 2 live neighbors, not 3, so it stays dead. It should have been born. The row below saw the new state.

NEIGHBORS READ BIT 0. BIT 1 IS THE NEXT STATE. SHIFT ONLY AFTER THE WHOLE BOARD IS DECIDED.

```
function gameOfLife(board) {
  const m = board.length;
  const n = board[0].length;
  const dirs = [
    [-1, -1], [-1, 0], [-1, 1],
    [0, -1], [0, 1],
    [1, -1], [1, 0], [1, 1],
  ];
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      let live = 0;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= m || nc >= n) continue;
        if ((board[nr][nc] & 1) === 1) live += 1;
      }
      const cur = board[r][c] & 1;
      if ((cur === 1 && (live === 2 || live === 3)) || (cur === 0 && live === 3)) {
        board[r][c] |= 2;
      }
    }
  }
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) board[r][c] >>= 1;
  }
}
```

The update order is decide, then shift. Inside the first pass you read only the low bit, you count, and you OR in 2 when the cell survives. You do not shift inside that pass. The second pass runs only after every cell has its next bit. Each `>> 1` then drops the old state together. A shift of one row in the middle of the first pass is the failure the column example just showed.

Ships

Battleships lie in a straight line, horizontal or vertical, and they do not touch. You count ships. You do not flood. The top-left cell of a ship is an X whose upper neighbor is not X and whose left neighbor is not X. Every other X of that ship has an X above it or an X to its left, so you skip it.

On a board with an X at row 0 column 0, and a vertical ship in column 3 on all three rows, you count two ships. The lone X has no upper neighbor and no left neighbor, so you count it. The top of column 3 has empty cells beside it, so you count it. The X at row 1 column 3 has an X above it, so you skip it. The X at row 2 column 3 has an X above it, so you skip it. If you counted every X you would report 4, and there are 2 ships.

A filled board

Valid sudoku uses nine sets for rows, nine for columns, and nine for boxes. The box index is `floor(r / 3) * 3 + floor(c / 3)`. Row 4 column 5 is box `floor(4 / 3) * 3 + floor(5 / 3)`, which is 3 + 1, so box 4, the center box. You skip a dot. A digit already in that row's set, that column's set, or that box's set makes the board invalid.

You are checking a filled board. You are not searching for a way to fill the dots. The solver is chapter 23, and it uses these same three families of sets.

Time and extra memory

Set zeroes and life each make a constant number of passes over the board, so the time is proportional to m times n. The extra memory is the boolean, or nothing beyond the bits already in the cells. Battleships is one pass and a handful of neighbor checks. Sudoku touches a fixed 9 by 9 board, and the sets hold at most nine digits each.

Mistakes

On the marked grid [1, 1, 0], [0, 1, 0], [1, 1, 1], zero row 0 before the lower rows read it. Row 0 becomes [0, 0, 0]. Column 1's mark, which was 1, is now 0. The write then kills column 1 in every row. You finish at [0, 0, 0], [0, 0, 0], [1, 0, 0]. The correct board keeps the 1 at row 0 column 0, the 1 at row 0 column 1, and the 1 at row 2 column 1.

Take [1, 2] over [0, 3]. The only zero is row 1 column 0, not row 0. col0 becomes true, and no column mark is written into row 0, because the inner loop starts at column 1. The correct finish is [0, 2] over [0, 0]: column 0 dies from the flag, and row 1 dies because its row mark is the zero itself. The 2 stays. If you write column 0 from the flag before you read the row mark, you set `matrix[0][0]` to 0 first. Row 0 then looks marked, and the 2 becomes 0. The whole grid dies.

In the life example, a dead cell that will be born is stored as 2. If a neighbor reads the whole number and treats any value above 0 as live, that 2 looks live. The mask `& 1` makes 2 look dead and 3 look live, which is the old state. Without the mask, the neighbor sees the future.

Shift row 0 of [1, 0, 0], [1, 0, 0], [1, 0, 0] as soon as you finish that row. The top-left cell becomes 0 while row 1 still has to count it. The middle-left cell dies, and the cell to its right is not born. Both of those results are wrong.

Count every X on the battleship board and you report 4. The vertical ship is three cells and one ship. Only the top-left X of each ship is a count.

Practice

Stay before chapter 21. Then before you leave the chapter.

73. Set Matrix Zeroes MEDIUM STAY
First row and column are marks, plus a flag for column 0. Write back from the bottom-right.

289. Game of Life MEDIUM STAY
Next state in bit 1. Neighbors use board[r][c] & 1. Shift after the full pass.

36. Valid Sudoku MEDIUM STAY
A set per row, per column, and per 3×3 box. Dots are empty.

419. Battleships in a Board MEDIUM THEN
Count an X only when the cell above and the cell to the left are not X.

REWRITE FROM MEMORY

Rewrite set-zeroes from the bottom-right, and the life update that reads the low bit and shifts only after the whole board.

Carry forward. Chapter 21 goes back to prefix sums. The prefix becomes a grid, and a rectangle is four corners.

CHAPTER 21 · BUILDS ON CHAPTERS 8 AND 18

2D prefix

A rectangle sum is four corners of a prefix grid. You add the top-left corner back, because both subtractions removed it.

What this pattern is

A prefix sum at index i is the total of every value from the start through i. In one dimension, the sum from i through j is the prefix just after j minus the prefix just before i. A matrix prefix does that for a rectangle. One grid, built once, answers any rectangle with four reads.

What you already know

Chapter 8 seeds a map with 0, asks for sum minus the target, and updates the map after the query. You still do that when a rectangle has been collapsed into one array. Chapter 18 says each row of a new grid is its own array. The prefix grid is one larger than the matrix in each direction, and its border is zeros.

A worked example

Take the 2 by 2 matrix [[1, 2], [3, 4]]. Let `p[i + 1][j + 1]` be the sum of the rectangle from row 0 column 0 through row i column j, inclusive. `p[r][c]` is the sum of every original cell whose row is less than r and whose column is less than c. The grid p is 3 by 3, because of the zero border, and you build each of its rows with its own array.

The build is `p[i + 1][j + 1] = mat[i][j] + p[i][j + 1] + p[i + 1][j] − p[i][j]`. You add the cell, the rectangle above, and the rectangle to the left. Those two rectangles both include the top-left rectangle. The minus removes that double-counted piece.

Cell row 0 column 0 is 1. You write `p[1][1] = 1 + p[0][1] + p[1][0] − p[0][0]`, which is 1 + 0 + 0 - 0 = 1. You touch `p[1][1]` because the prefix index is one past the cell. The value 1 is the rectangle that holds only the top-left cell.

Cell row 0 column 1 is 2. You write `p[1][2] = 2 + p[0][2] + p[1][1] − p[0][1]` = 2 + 0 + 1 - 0 = 3. That is the whole first row, 1 + 2. The rectangle above this cell is still on the zero border, so the minus removes nothing.

Cell row 1 column 0 is 3. You write `p[2][1] = 3 + p[1][1] + p[2][0] − p[1][0]` = 3 + 1 + 0 - 0 = 4. That is the first column, 1 + 3.

Cell row 1 column 1 is 4. You write `p[2][2] = 4 + p[1][2] + p[2][1] − p[1][1]` = 4 + 3 + 4 - 1 = 10. The rectangle above is the first row, sum 3. The rectangle to the left is the first column, sum 4. Both include the top-left cell 1, which is `p[1][1]`. You added that 1 twice, so you subtract it. 4 + 3 + 4 - 1 is 10, which is 1 + 2 + 3 + 4. The finished prefix, with its zero border, is 0 0 0 / 0 1 3 / 0 4 10.

A query for the whole matrix reads `p[2][2] − p[0][2] − p[2][0] + p[0][0]` = 10 - 0 - 0 + 0 = 10. A query for the bottom-right cell only, rows 1 through 1 and columns 1 through 1, reads `p[2][2] − p[1][2] − p[2][1] + p[1][1]` = 10 - 3 - 4 + 1 = 4.

You subtract `p[1][2]`, which is 3, to remove the first row. You subtract `p[2][1]`, which is 4, to remove the first column. Both of those regions contain the top-left 1, so both subtractions removed it. The total is short by 1. You add `p[1][1]` back. That is why the last term is a plus. The result is 4, the cell itself.

How you recognise it

The matrix does not change, and you answer sum questions about rectangles. One prefix at the start makes each query four array reads. A square around each cell is that same prefix, with the edges clamped to the grid. You do not build a new prefix per cell. A count of submatrices whose sum equals a target is not four reads. You fix the top row and the bottom row, add those rows into one array, and run chapter 8. If the work is to add a value onto many ranges and then read, the matrix is not a fixed prefix. That is the difference array.

- Use chapter 21 when the rectangle sums are on a matrix that stays still.
- Use chapter 8 when the data is already one array. You keep the seed 0 and the query before the update.
- Use chapter 9 when you add a value onto many ranges and then read the line.

What to say out loud

The invariant is that p[r][c] is the sum of every original cell whose row is less than r and whose column is less than c. I subtract the top-left rectangle in the build because the rectangle above and the rectangle to the left both included it. On a query I add that corner back, because both subtractions removed it.

The template

INCLUSIVE RECTANGLE (R1, C1) THROUGH (R2, C2). THE LAST TERM IS THE ONE YOU SAY OUT LOUD.

```
function buildPrefix(mat) {
  const m = mat.length;
  const n = mat[0].length;
  const p = Array.from({ length: m + 1 }, () => Array(n + 1).fill(0));
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      p[i + 1][j + 1] = mat[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j];
    }
  }
  return p;
}

function rectSum(p, r1, c1, r2, c2) {
  return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
}
```

```
mat
1 2
3 4

p, with a zero border
0  0  0
0  1  3
0  4  10

whole matrix: p[2][2] - p[0][2] - p[2][0] + p[0][0] = 10
bottom-right cell only, (1,1)-(1,1):
10 - p[1][2] - p[2][1] + p[1][1] = 10 - 3 - 4 + 1 = 4
the +1 is the top-left rectangle that both subtractions removed
```

You fill p in row-major order, so the rectangle above and the rectangle to the left already exist when you write a cell. The minus is the third term, and it runs in the same assignment, before you move on. On a query, the two subtractions come first and the add-back is last. For the bottom-right cell of the example, that last plus is `p[1][1]`, the 1 both subtractions removed. The inclusive end of the matrix lives at `r2 + 1` and `c2 + 1`, because the prefix is one larger than the matrix in each direction.

Matrix block sum asks each cell for the sum of the square that extends k cells out, clipped to the matrix. You clamp each edge so it stays inside the grid, then you call `rectSum`. You built p once. You do not rebuild it per cell.

Submatrices that sum to a target

You fix two rows, the top and the bottom, inclusive. You collapse the columns between those rows into one array `col`. `col[c]` is the sum of `matrix[top][c]` through `matrix[bot][c]`. A contiguous segment of `col` is exactly one submatrix between those two rows. You count the segments whose sum is the target with chapter 8: seed 0 to 1, query sum minus the target, then update the map.

Use [[1, 2], [3, 4]] and target 3. The hits are the first row, 1 + 2, and the single cell 3. With top 0 and bot 0, col becomes [1, 2]. The map starts as 0 to 1. Sum becomes 1. You ask for 1 minus 3, which is absent, then you record sum 1. Sum becomes 3. You ask for 3 minus 3, which is 0, and the seed answers 1. That hit is the row [1, 2]. Then you record sum 3.

You do not clear col when the bottom row moves down. You add the new row into it. Bot becomes 1, and col becomes [1 + 3, 2 + 4], which is [4, 6]. You build a new map and you seed 0 again. Sum 4 asks for 1 and misses. Sum 10 asks for 7 and misses. The full matrix is 10, not 3.

When the top row moves to 1, you need a fresh col, because the old top row has to leave the sum. Bot is 1, so col becomes [3, 4]. Sum becomes 3, the query for 0 hits the seed, and you count the cell 3. Sum 7 misses. The count is 2. The matrix only chose which one-dimensional array you handed to chapter 8.

SECOND PASS, AND THE TEMPLATE IS COMPLETE. TWO ROWS, THEN CHAPTER 8.

```
function numSubmatrixSumTarget(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let count = 0;
  for (let top = 0; top < m; top++) {
    const col = new Array(n).fill(0);
    for (let bot = top; bot < m; bot++) {
      for (let c = 0; c < n; c++) col[c] += matrix[bot][c];
      const freq = new Map();
      freq.set(0, 1);
      let sum = 0;
      for (const x of col) {
        sum += x;
        count += freq.get(sum - target) ?? 0;
        freq.set(sum, (freq.get(sum) ?? 0) + 1);
      }
    }
  }
  return count;
}
```

The update order inside the column scan is the chapter 8 order. You add x into sum, you query sum minus the target, and only then you increment the frequency of sum. The seed 0 is written before that loop, once per bottom row, because a segment that starts at column 0 is a prefix equal to the target. You build that map again when bot moves. The previous map counted a shorter band.

Time and extra memory

The build is proportional to m times n. Each rectangle query is a constant number of reads. A block sum over every cell is proportional to m times n, after the one build. Counting submatrices that sum to a target, by fixing two rows and scanning the columns, is proportional to m squared times n. The prefix grid is one extra row and one extra column, each row its own array. The frequency map is rebuilt for each pair of rows.

Mistakes

Drop the added-back term on the bottom-right cell of [[1, 2], [3, 4]]. You compute 10 - 3 - 4 and you get 3. The cell is 4. Both subtractions removed the top-left 1, and you never put it back.

Allocate p with one shared inner array. The write `p[1][1] = 1` also writes `p[0][1]` and `p[2][1]`. The zero border is gone on the first cell, and every later formula adds that polluted 1.

Read `p[r2][c2]` for an inclusive end. For the whole 2 by 2 matrix that is `p[1][1]`, which is 1, not 10. The inclusive end lives at row r2 + 1 and column c2 + 1.

Rebuild the frequency map inside the two row loops and forget the seed. On the target-3 walk, both hits ask for 0. The map does not contain 0, so the count stays 0. The first row and the cell 3 are both prefixes that start at column 0, and the seed is what makes them visible.

Practice

Stay before chapter 22. The collapse to 1D is a second pass, and the template above is the one to adapt.

304. Range Sum Query 2D - Immutable MEDIUM STAY
Build p in the constructor. sumRegion is the four-term formula.

1314. Matrix Block Sum MEDIUM STAY
One prefix. Each cell clamps [i − k, j − k] through [i + k, j + k] to the matrix.

1074. Number of Submatrices That Sum to Target HARD SECOND PASS
Fix two rows, collapse to a 1D array, run the chapter 8 count.

REWRITE FROM MEMORY
Rewrite the prefix build and the rectangle sum until you can say why the top-left corner is added back.

Carry forward. Chapter 22 searches a sorted matrix. One version treats it as a flat sorted array. The other starts at a corner and walks one direction.

CHAPTER 22 · BUILDS ON CHAPTERS 14 AND 18

Search a sorted matrix

One kind of sorted matrix is a single list, searched from the middle. The other kind is sorted by row and by column, and you start at the top-right.

What this pattern is

A sorted matrix lets you throw away many cells after one look. You are not scanning every cell.

Picture a page of names in a dictionary. The names run in one order. You open the middle. If that name is too late in the alphabet, the word you want is in the left half. Some matrices are that kind of page. Each row is sorted, and the first cell of a row is greater than the last cell of the row above. The rows are one sorted list, written with line breaks. You binary-search a flat index.

Other matrices are sorted along each row and along each column, and the rows do not stack into one list. A cell at the end of a row can be larger than a cell at the start of the next row. A flat index is then the wrong tool. You start at the top-right and walk a staircase. A staircase means one step makes the number smaller and the other step makes it larger. A miss tells you which step to take.

A third job asks for the kth smallest number, not for a yes or a no. You binary-search the number itself, and you count how many cells are small enough. The three jobs look alike on the page. They are three different searches. The flat search, in particular, must not be aimed at a matrix that is only row-sorted and column-sorted. That reading order is not one sorted list.

What you already know

Chapter 14 searches a sorted list with a half-open range. Half-open means lo is included and hi is not. While lo is less than hi, you test the middle index. If that value is already large enough, you set hi to the middle. If it is too small, you set lo to the middle plus 1. You never set lo to the middle itself, or two indexes can sit still forever. Chapter 18 is the grid. `matrix[r][c]` is row then column, there are m rows and n columns, and each new row must be its own array.

A worked example

Use this matrix. It has 2 rows and 3 columns. Row 0 is 1, 3, 5. Row 1 is 7, 9, 11. You want 9. Each row is sorted left to right, and 7 is greater than 5, so the rows stack into one sorted list. That list has 6 cells.

The flat indexes run from 0 through 5. Index 0 holds 1, index 1 holds 3, index 2 holds 5, index 3 holds 7, index 4 holds 9, and index 5 holds 11. You search the half-open range from 0 up to 6. lo starts at 0. hi starts at 6, which is m times n, one past the last index.

The first middle is 3. You turn a flat index into a cell with the row width n, here 3. The row is `Math.floor(3 / 3)`, which is 1. The column is `3 % 3`, which is 0. The cell is 7. 7 is smaller than 9, so this index is too small. You set lo to mid plus 1, which is 4. Indexes 0 through 3 are finished.

The range is now 4 up to 6. The middle is 5. The row is `Math.floor(5 / 3)`, which is 1. The column is `5 % 3`, which is 2. The cell is 11. 11 is at least 9, so this index is large enough, and the answer might be 5 itself. You set hi to 5. Index 4 is still inside the range.

The range is now 4 up to 5. The middle is 4. The row is 1 and the column is 1, so the cell is 9. 9 is at least the target, so you set hi to 4. lo and hi meet. The loop stops. The cell at index 4 equals 9, so you return true. If the target were 12, lo would land on 6, past the last cell, and you would return false.

```
1  3  5
7  9  11
target 9, n = 3, flat length 6
[0, 6) mid 3 → row 1 col 0 → 7, 7 < 9, lo = 4
[4, 6) mid 5 → row 1 col 2 → 11, 11 >= 9, hi = 5
[4, 5) mid 4 → row 1 col 1 → 9, hi = 4
lo === 4, matrix[1][1] === 9, true
```

How you recognise it

Use the flat search when each row is sorted and the first cell of a row is greater than the last cell of the row above. The matrix is one sorted list, so a flat index has a real middle. Use the staircase when each row is sorted and each column is sorted, and that row-to-row promise is missing. One comparison must then discard a whole row or a whole column. Search the value itself when the question is the kth smallest entry in a row-and-column sorted matrix, because a larger candidate never holds fewer small entries. Count negatives is the same corner, on a matrix that stays the same or gets smaller as you move right or down.

- Use chapter 22 when one comparison on a sorted matrix discards a row, a column, or half of a flat index.
- Use chapter 14 when the input is already one sorted array, with no rows to flatten.
- Use chapter 18 when you visit every cell and the order does not let you discard any.

What to say out loud

SAY THIS BEFORE YOU CODE

An invariant is the fact that is true every time the loop is about to run. For the flat search, the answer index sits from lo up to hi, and hi is not included. If the middle cell is at least the target, set hi to mid, and if it is too small, set lo to mid plus 1. For the staircase, cells above your row are too small and cells to the right of your column are too large, so the target is still in the rectangle you kept.

The template

CHAPTER 14'S LOWER BOUND, ON AN INDEX THAT RUNS FROM 0 TO M*N.

```
function searchMatrix(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let lo = 0;
  let hi = m * n;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    const val = matrix[Math.floor(mid / n)][mid % n];
    if (val >= target) hi = mid;
    else lo = mid + 1;
  }
  if (lo === m * n) return false;
  return matrix[Math.floor(lo / n)][lo % n] === target;
}
```

The invariant is the half-open range on the flat index, from 0 up to m times n. Every index below lo is too small. The first index that might hold the target is lo, and every later candidate is below hi. hi is not a candidate. When lo and hi meet, lo is the first index whose cell is at least the target, or m times n if every cell is smaller.

The off-by-one is the exclusive hi. It starts at m times n, not at the last index. The middle is lo plus the whole number of (hi minus lo) divided by 2, the same form as chapter 14. The row is `Math.floor(mid / n)`. The column is `mid % n`. Divide by the row count m and you read the wrong cell. The update matches chapter 14. If val is at least the target, hi becomes mid. If val is smaller, lo becomes mid plus 1. The failing side must move past

mid. After the loop you return true only when lo is still inside the matrix and that cell equals the target. Leave this function alone when the matrix is only row-sorted and column-sorted. That sequence is not one sorted list.

The staircase from the top-right

Use this matrix. Row 0 is 1, 4, 7. Row 1 is 2, 5, 8. Row 2 is 3, 6, 9. You want 6. There are 3 columns, so you start at the top-right: row 0, column `n - 1`, the cell 7. Left decreases, because each row grows toward the right. Down increases, because each column grows toward the bottom. One direction decreases. One direction increases.

7 is bigger than 6, so you move left with `c -= 1`. You stand on 4. 4 is smaller than 6, so you move down with `r += 1`. You stand on 5. 5 is smaller than 6, so you move down again. You stand on 6. Equal means you found it.

Start at the top-left, on 1, and keep those same moves. 1 is smaller than 6, so you move down onto 2, then onto 3, then off the board. You return false, and 6 is in the matrix. At the top-left both the right neighbor and the lower neighbor are larger, so a miss does not tell you which way to go. The bottom-left also works only if you restate it: too small goes right, and too big goes up. Memorize one corner. This chapter memorizes the top-right.

TOP-RIGHT. TOO BIG GOES LEFT. TOO SMALL GOES DOWN.

```
function searchMatrix(matrix, target) {
  const m = matrix.length;
  const n = matrix[0].length;
  let r = 0;
  let c = n - 1;
  while (r < m && c >= 0) {
    if (matrix[r][c] === target) return true;
    if (matrix[r][c] > target) c -= 1;
    else r += 1;
  }
  return false;
}
```

r starts at 0 and c starts at `n - 1`. The loop runs while the row is still inside and the column is at least 0, so column 0 is a real cell. A test written as c greater than 0 would skip the leftmost column and miss a target that sits there. Equal returns true at once. A larger cell updates the column, `c -= 1`. A smaller cell updates the row, `r += 1`. You do not move up or right from this corner. Those steps walk back into cells you have already rejected. When the row falls off the bottom or the column falls off the left, you return false.

```
1  4  7
2  5  8
3  6  9
target 6, start at 7
7 > 6, left to 4
4 < 6, down to 5
5 < 6, down to 6
found

start at 1 instead: 1 < 6, and both the right neighbor and the lower neighbor are larger
you would be guessing
```

Count the negatives

Problem 1351 is this corner on a matrix that is non-increasing. Non-increasing means each step right stays the same or gets smaller, and each step down does the same. You still start at the top-right. The 240 tests are for a matrix that does not decrease, so you do not reuse them. If the cell is negative, every cell below it in that column is negative too. You add `m - r`, the number of rows from here to the bottom, and you move left. If the cell is not negative, you move down.

Use 4 rows. Row 0 is 4, 3, 2, −1. Row 1 is 3, 2, 1, −1. Row 2 is 1, 1, −1, −2. Row 3 is −1, −1, −2, −3. You start at row 0, column 3, on −1. It is negative, so you add 4 − 0, which is 4, and you move left. Row 0, column 2 is 2, so you move down. Row 1, column 2 is 1, so you move down. Row 2, column 2 is −1, so you add 4 − 2, which is 2, and you move left. Row 2, column 1 is 1, so you move down. Row 3, column 1 is −1, so you add 1 and you move left. Row 3, column 0 is −1, so you add 1 and you move left. The column falls off the board. The total is 4 + 2 + 1 + 1, which is 8, and that is every negative cell.

The kth smallest value

You want the 8th smallest value, not a yes or a no. The matrix is 3 by 3. Row 0 is 1, 5, 9. Row 1 is 10, 11, 13. Row 2 is 12, 13, 15. If you listed every cell in order you would see 1, 5, 9, 10, 11, 12, 13, 13, 15, so the 8th is 13. The search finds 13 without building that list. lo starts at the top-left value, 1. hi starts at the bottom-right value, 15. You binary-search the value with the same half-open shrink, while lo is less than hi.

The first middle is 8. Count the cells that are at most 8, walking each row from the right. A column cursor starts at 2. In row 0 the cell 9 is greater than 8, so the cursor moves left onto 5, which is small enough. That row contributes 2 cells. The cursor is now at column 1, and it only moves left. Row 1's cell at column 1 is 11, then column 0 is 10, so the cursor falls off and the row contributes 0. Row 2 contributes 0. The count is 2, which is under 8, so 8 is too small. You set lo to 9.

The next middle is 12. Six cells are at most 12, still under 8, so lo becomes 13. The next middle is 14. Eight cells are at most 14, and 8 is at least k, so hi becomes 14. A smaller value might still be large enough. The next middle is 13. Eight cells are at most 13, so hi becomes 13. lo and hi meet, and you return 13. The test ok means the count of entries at most mid is at least k, so hi becomes mid. If the count is short, lo becomes mid plus 1.

SEARCH THE VALUE. THE SAME LO < HI PROGRESS AS CHAPTER 14.

```
function kthSmallest(matrix, k) {
  const n = matrix.length;
  let lo = matrix[0][0];
  let hi = matrix[n - 1][n - 1];
  const countAtMost = (mid) => {
    let count = 0;
    let c = n - 1;
    for (let r = 0; r < n; r++) {
      while (c >= 0 && matrix[r][c] > mid) c -= 1;
      count += c + 1;
    }
    return count;
  };
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (countAtMost(mid) >= k) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
```

lo and hi are values, not indexes. They are integers, and the loop stops when they are equal, which covers a matrix whose smallest and largest cells are already the same number. The middle uses the chapter 14 form, so the range still shrinks. `countAtMost` keeps one column cursor for the whole test. The cursor moves left while the cell is greater than mid, then the row adds `c + 1`. A later row is larger, so the cursor never needs to move right, and one test stays linear in n. The update is the lower-bound update. A count that is at least k sets hi to mid. A short count sets lo to mid plus 1.

Time and extra memory

The flat search is O(log (mn)) comparisons and a handful of variables. The staircase reads at most O(m + n) cells and stores a handful of variables. On an n by n matrix the kth search tests O(log V) values, and each test is O(n), so the time is O(n log V). V is the range from the

smallest value to the largest. None of the three allocates another matrix.

Mistakes

- Flat search on row 1, 4, 7, then 2, 5, 8, then 3, 6, 9, looking for 2. The flat list is 1, 4, 7, 2, which is not sorted. The half-open search stops on 4 and returns false. The 2 is at row 1, column 0.
- Staircase from the top-left on 1, 4, 7 / 2, 5, 8 / 3, 6, 9, looking for 6, moving down whenever the cell is too small. You walk 1, then 2, then 3, then off the board, and you return false. Both right and down were larger at the start.
- Bottom-left on that same matrix, still using too-big-goes-left and too-small-goes-down. You start at 3. It is smaller than 6, so you move down and fall off, and you return false. Restated, too small goes right and too big goes up. Memorize the top-right.
- Count negatives on 1, −1 / −1, −1, using the 240 move that goes down when the cell is smaller than 0 and counting one cell at a time. You count the two cells in the last column and return 2. The bottom-left −1 is missed. The real total is 3, from adding m minus r and moving left.

Practice

Stay before chapter 23. Then before you leave the chapter. Revisit 1351 with the staircase once 240 is boring.

74. Search a 2D Matrix MEDIUM STAY
Rows are sorted and stacked in order. Flat index mid / n and mid % n.

240. Search a 2D Matrix II MEDIUM STAY
Rows and columns each sorted. Start at the top-right.

378. Kth Smallest Element in a Sorted Matrix MEDIUM THEN
Binary-search the value. Count entries <= mid, one descending cursor per test.

REWRITE FROM MEMORY
Rewrite both searches. Say why the top-left corner does not tell you which way to step.

Carry forward. Chapter 23 treats the matrix as a graph. A cell has up to four neighbors, and you mark a cell when you enter it.

CHAPTER 23 · BUILDS ON CHAPTERS 18 AND 20

Regions and grid graphs

You mark a cell when you enter it. One round of the queue is one step.

What this pattern is

A grid is a building drawn on paper. Each cell is a room. A room has a door to the room above, below, left, or right, when that room is on the paper. You do not write a list of doors. The four neighbors are the doors.

A depth-first search follows one hallway as far as it goes, then backs up and tries the next hallway. You use it to paint a whole region: an island, a blob of land, a flood of one color. The function calls itself on a neighbor. That kind of call is recursion.

A breadth-first search visits every room one door away before any room two doors away. You use it when the question is minutes, or distance, or the nearest source. Waiting cells sit in a queue, a line you join at the back and leave from the front. To enqueue a cell means to put it at the back of that line.

A layer is one ring of that search: every cell that is the same number of steps from the sources. In rotting oranges, one layer is one minute. You finish every cell now in the queue before you add a minute.

You mark a cell when you enter it, before you walk its neighbors. If you mark it on the way out, the neighbor walks back in. A cell waiting in the queue counts as entered, so a breadth-first search marks at the moment it enqueues.

What you already know

Chapter 18 gave you m rows, n columns, the four directions, and the test that a neighbor is inside the grid. Chapter 20 stored a mark inside the cell so you would not allocate a second board. You still refuse to read a row or a column that is off the paper. Chapter 22 searched for a value. This chapter walks neighbors.

A worked example

Use this grid of characters. Row 0 is 1, 1, 0. Row 1 is 0, 1, 0. Row 2 is 0, 0, 1. A 1 is land. A 0 is water. You want the number of islands, and land connects only through the four doors.

You scan from the top-left. Row 0, column 0 is 1, so this is a new island and the count becomes 1. You enter that cell and set it to 0 before you look at any neighbor. The mark has to happen now. If the cell still said 1 on the way out, a neighbor could walk back in.

The doors are tried down, then up, then right, then left. Down from the top-left is water, so that call returns. Up is off the board. Right, row 0 column 1, is 1. You enter it and set it to 0 before its own neighbors. Its first door goes down to row 1 column 1, which is 1. You set that cell to 0 as soon as you enter it. Its up neighbor is the cell you just marked, so the walk does not go back. Its other neighbors are water or off the board.

The scan continues. Cells you already set to 0 do not start a new island. Row 2, column 2 is still 1. The count becomes 2. You set that cell to 0. Its neighbors do not spread. The answer is 2.

Picture the same first island if the mark waited until the function returned. You would be standing on row 0 column 1 while row 0 column 0 still said 1. The left door would enter that cell again, and that cell would enter the right cell again. The two calls would not end.

How you recognise it

How many regions, how large one region is, or which color to paint: that is a depth-first search, and you mark on entry. Minutes until the oranges rot, distance to the nearest 0, or the shortest bridge between two islands: that is a breadth-first search, and every source starts in the queue. A layer of that search is one minute, or one step. A cell that can reach both oceans is a walk uphill from the ocean borders, toward a neighbor of equal or greater height.

A word on the board marks a letter, goes one step deeper, and puts the letter back, because the cell is busy for this word and free for the next try. If the new flood color already equals the color under your feet, you stop, or the walk never ends.

- Use chapter 23 when the question is a region, a recolor, or a distance measured in steps on the grid.
- Use chapter 22 when you are hunting a value in a sorted matrix, not walking neighbors.
- Use chapter 24 when the number in a cell is a sum, a min, or a size taken from cells you have already filled.

What to say out loud

SAY THIS BEFORE YOU CODE

The invariant, the fact that is true before each step, is that a cell you have entered will not be entered again. You mark it before the function calls its neighbors, or at the moment you enqueue it. If this search must back up, the mark lasts only for this path. You write the old letter back before the next path runs.

The template

WRITE THE MARK BEFORE THE RECURSIVE CALLS.

```
function numIslands(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const dfs = (r, c) => {
    if (r < 0 || c < 0 || r >= m || c >= n || grid[r][c] !== "1") return;
    grid[r][c] = "0";
    for (const [dr, dc] of dirs) dfs(r + dr, c + dc);
  };
  let count = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === "1") {
        count += 1;
        dfs(r, c);
      }
    }
  }
  return count;
}
```

The invariant is the character in the cell. Once you have entered it, the cell is 0, so a later call returns at the bounds line. The update order is the line `grid[r][c] = "0"` sitting above the four calls. Move that write below the loop and the neighbor sees 1 and walks back in. The bounds reject a row or column below 0, and a row or column at m or n or beyond. The last real row is m minus 1, which must stay inside. A test that rejects r greater than m minus 1 is the same edge. A test that only rejects r greater than m reads one row past the end. The outer loops start a search only on a cell that is still the character 1.

Area, and a flood of color

Max area uses the same walk and returns a number. On the grid 1, 1, 0 / 1, 0, 0, you enter the top-left. The function returns 1 for this cell, plus what the four calls return. The right call finds land and returns 1. The down call finds land and returns 1. The other two are water or off the board and return 0. The area is 3. A rejected neighbor returns 0, so adding it is safe. You still set the cell to 0 before the four calls.

Flood fill is the same walk, painting a new color. Take the image 1, 1 / 1, 0, start at the top-left, and paint 2. You set the top-left to 2 before its neighbors, then the top-right, then the bottom-left. The bottom-right is 0, so you

stop. The image is 2, 2 / 2, 0. If the new color is already 1, you return the image immediately. Otherwise you paint 1 onto 1, the neighbor still matches, and the walk never ends.

Closed islands

Here land is 0 and water is 1. A closed island is land that does not touch the border. Take a frame of water around three land cells: row 0 is 1, 1, 1, 1, row 1 is 1, 0, 0, 1, row 2 is 1, 0, 1, 1, and row 3 is 1, 1, 1, 1. First you sink every land cell that touches the border, with the same flood. This border is all water, so the sink finds nothing. Then you count the land that remains. The three 0s touch, so the count is 1.

Now take row 0 as 0, 0, 1 and row 1 as 1, 1, 1. The two land cells touch the top border. If you count them at once, you report one closed island. They are not closed. You sink them first. Nothing remains, so the answer is 0.

Surrounded regions

You are given a board of the letters X and O. An O that can reach the border must stay O. An O that is sealed in must become X. Take this board. Row 0 is X, X, X, X. Row 1 is X, O, O, X. Row 2 is X, X, O, X. Row 3 is X, O, X, X. The O at row 3, column 1 sits on the border. You flood from every border O and mark those cells with a temporary letter, T. Its four neighbors are X or off the board, so the inner O cells are not reached.

You then walk the whole board. An O you never reached becomes X, so the three inner O cells flip. Each T becomes O again, so the border O stays. The order is the whole method. If you flipped every O first, the border O would be gone, and you would not know which region touched the outside.

One layer is one minute

Use the grid of numbers 2, 1, 1 / 1, 1, 0 / 0, 1, 1. A 2 is rotten, a 1 is fresh, and a 0 is empty. Every 2 starts in the queue, so the queue begins with row 0 column 0 only. You count 6 fresh oranges. A layer is one minute. You will set a fresh orange to 2 at the moment you enqueue it, so a second neighbor cannot enqueue it again.

The first layer is that single 2, and the minute counter is still 0 while you expand it. The down neighbor and the right neighbor are fresh. You set each to 2, subtract from the fresh count, and enqueue it. Fresh is now 4. The layer is finished, so you add 1 minute.

The next layer holds row 1 column 0 first, then row 0 column 1, because down was enqueued before right. Row 1 column 0 looks right and finds the center still fresh. You set the center to 2 and enqueue it. Row 0 column 1 then looks down, sees 2, and does not enqueue the center again. It does rot the top-right, which is still fresh. Fresh is 2. You add a minute. The total is 2.

The next layer rots the orange at row 2 column 1. Fresh is 1, and the minute total is 3. The last layer rots the bottom-right. Fresh is 0, and the minute total is 4. The loop stops because no fresh orange remains. You return 4.

A grid with no fresh orange, such as 0, 2, never enters the loop, because the fresh count is not positive. You return 0. A fresh orange walled off by empty cells, such as 2, 0 / 0, 1, leaves fresh at 1 when the queue runs out. You return −1.

EVERY ROTTEN ORANGE STARTS IN THE QUEUE. MARK A FRESH ORANGE WHEN YOU ENQUEUE IT.

```
function orangesRotting(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const q = [];
  let fresh = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === 2) q.push([r, c]);
      else if (grid[r][c] === 1) fresh += 1;
    }
  }
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
```

```
  let head = 0;
  let minutes = 0;
  while (head < q.length && fresh > 0) {
    const layer = q.length - head;
    for (let k = 0; k < layer; k++) {
      const [r, c] = q[head];
      head += 1;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= m || nc >= n || grid[nr][nc] !== 1) continue;
        grid[nr][nc] = 2;
        fresh -= 1;
        q.push([nr, nc]);
      }
    }
    minutes += 1;
  }
  return fresh === 0 ? minutes : -1;
}
```

Every 2 is pushed before the clock starts. The layer length is `q.length - head`, saved before the inner loop, because the loop pushes cells that belong to the next minute. You read `q[head]` and then add 1 to head. You do not call `Array.shift`, which would move every later cell on every pop. The update on a fresh neighbor is three steps in this order: set it to 2, subtract 1 from fresh, then push. The set is what makes the cell fail the test `!== 1` for the next neighbor. minutes gains 1 after the layer. If fresh is 0 at the start, the loop does not run and you return 0. If fresh is still positive when the queue is empty, you return −1.

```
minute 0, queue holds every 2
each loop processes exactly the current layer, then minutes += 1
an orange set to 2 before the push cannot be enqueued by a second neighbor
no fresh at the start → the while does not run → 0
fresh left and the queue is empty → -1
```

Distance to the nearest 0

The 01 matrix is the same multi-source search. On the grid 0, 0 / 0, 1, every 0 starts in the queue at distance 0. The first time you reach the 1, you stepped from a neighboring 0, so the distance is 1. Breadth-first search walks in order of distance, so that first visit is final. A later path cannot be shorter. You keep a head index, the same stand-in for `Array.shift`.

Both oceans

Pacific Atlantic asks which cells can flow to both oceans. Water flows to an equal or lower neighbor. You reverse the doors. From an ocean border you walk to a neighbor whose height is greater than or equal, which is uphill, and you mark on entry so one ocean does not expand a cell twice.

Use the heights 1, 2 / 4, 3. Pacific is the top row and the left column, so the border cells are 1, 2, and 4. Atlantic is the bottom row and the right column, so the border cells are 4, 3, and 2. From the Pacific cell 1 you can climb to 2 and to 4, and from 2 you can climb to 3. All four cells reach the Pacific. From the Atlantic borders you can stand on 2, 4, and 3, and you cannot climb from any of them into the 1, because 1 is smaller. The cell 1 reaches only the Pacific. The other three cells are in both sets, so they are the answer.

The shortest bridge

You paint one island as 2 and enqueue every cell of it, marking before you recurse, the same order as an island. Then you expand in layers. steps starts at 0. The painted island is the first layer, so during that layer steps is still 0. If a neighbor is still 1, the other island shares an edge with this one. You return steps, which is 0. You have flipped no water. Do not return 1 for that look.

A single water cell between the islands is a different grid. Take the row 1, 0, 1. You paint the left 1 as 2 and enqueue it. The right 1 stays 1. You expand the first layer while steps is 0. The only inside neighbor is the 0. You set that water to 2 and enqueue it. You do not see a 1 yet. The layer ends, and steps becomes 1. The next layer stands on the water, sees the other 1, and returns 1.

PAINT ONE ISLAND. THE FIRST TIME THE EXPANSION TOUCHES A 1, RETURN THE STEPS SO FAR.

```
function shortestBridge(grid) {
  const n = grid.length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const q = [];
  const paint = (r, c) => {
    if (r < 0 || c < 0 || r >= n || c >= n || grid[r][c] !== 1) return;
    grid[r][c] = 2;
    q.push([r, c]);
    for (const [dr, dc] of dirs) paint(r + dr, c + dc);
  };
  let found = false;
  for (let r = 0; r < n && !found; r++) {
    for (let c = 0; c < n && !found; c++) {
      if (grid[r][c] === 1) {
        paint(r, c);
        found = true;
      }
    }
  }
  let head = 0;
  let steps = 0;
  while (head < q.length) {
    const layer = q.length - head;
    for (let k = 0; k < layer; k++) {
      const [r, c] = q[head];
      head += 1;
      for (const [dr, dc] of dirs) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr < 0 || nc < 0 || nr >= n || nc >= n || grid[nr][nc] === 2) continue;
        if (grid[nr][nc] === 1) return steps;
        grid[nr][nc] = 2;
        q.push([nr, nc]);
      }
    }
    steps += 1;
  }
  return steps;
}
```

paint writes 2 and enqueues before the recursive calls, so the queue starts as one whole island. steps starts at 0 and increases only after a layer finishes. The moment a neighbor is 1, you return `steps`. On the first layer that value is 0, which is the answer when the other island shares an edge. You do not return 1 for that look. A water cell is set to 2 before it is pushed, so a later cell does not push it again. On the row 1, 0, 1 the water is pushed during the first layer, steps becomes 1, and the next layer returns 1. Returning `steps + 1` would report 2 for that single water cell.

Mark, recurse, unmark

The board is two rows: A, A / A, B. The word is AAA. The word is on the board. One path is the top-right A, left into the top-left A, then down into the bottom-left A. The code does not try the top-right first. The outer scan starts at the top-left.

You save A, write #, and try the doors in order: down, up, right, left. Down is the bottom-left A, which matches the second letter. You mark it #. It has no third A, because the top-left is already # and the bottom-right is B. That

call fails, and you write A back. Right is the top-right A. You mark it, fail to find a third A, and write A back. Then you write the top-left A back as well. The first start is over, and the board is whole.

The next start is the top-right A. You mark it. Down is B, which is not A. Left is the top-left A. You mark that, step down to the bottom-left A, and the index reaches the end of the word. You return true. On the way out each frame writes its letter back, so the board is clean even after a hit.

Delete the line that writes the letter back, and run the same first start. The top-left, the bottom-left, and the top-right all stay #. The later start looks at # where an A should be. You return false for a word that is on the board. Forgetting to unmark punches a hole for the next path.

THE MARK IS ONLY FOR THIS PATH. PUT THE LETTER BACK ON THE WAY OUT.

```
function exist(board, word) {
  const m = board.length;
  const n = board[0].length;
  const dfs = (r, c, i) => {
    if (i === word.length) return true;
    if (r < 0 || c < 0 || r >= m || c >= n || board[r][c] !== word[i]) return false;
    const saved = board[r][c];
    board[r][c] = "#";
    const ok = dfs(r + 1, c, i + 1) || dfs(r - 1, c, i + 1)
      || dfs(r, c + 1, i + 1) || dfs(r, c - 1, i + 1);
    board[r][c] = saved;
    return ok;
  };
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (dfs(r, c, 0)) return true;
    }
  }
  return false;
}
```

You test the letter before you mark. A cell that does not match returns while the board is unchanged. When i equals the word length, you are past the last letter and you return true before the bounds test. The update on the way in is `saved`, then #. The update on the way out is writing `saved` back, and it sits after the four calls, so it runs for a failed path and for a path that already returned true up the chain. The current cell stays # while the four doors are tried, which is correct, because this path is still using it. Sibling doors from the parent see the letter again only because this frame restored it before the parent tried the next door.

Sudoku, on the second pass

The solver is the same shape on digits, and it waits for the second pass. You already have the three families of sets from chapter 20: the row, the column, and the box. Suppose the first empty cell is row 0, column 0. The row already holds 2, the column already holds 3, and the box already holds 2, 3, and 5. You try 1. It is free in all three sets. You add 1 to the three sets, write it, and move to the next empty cell.

That next cell has no legal digit. You delete 1 from the three sets, clear the cell, and try the next free digit, 4. You keep placing and undoing. The first time you move past the last empty cell, the board is solved. The undo is the same job as writing the letter back in word search. Skip it, and a failed guess leaves a digit that blocks every later try.

Word search II, later

Word search II is the same board walk with a dictionary. A trie is a tree of prefixes, so a prefix that no word uses can stop the recursion. You build the trie, walk the board, mark and unmark as above, and record a word when a node says a word ends. Learn the trie on its own day. The link is Word Search II.

Time and extra memory

Islands, rotting oranges, the bridge, the oceans, the flood, and the surrounded-region walk visit each cell a constant number of times, so the time is O(mn). The queue or the call stack can hold every cell, so the extra memory is O(mn). You mark the grid in place. Word search can try many paths through one cell, so a long word costs more than O(mn), and the extra memory is the depth of the current path. The sudoku sketch tries digits on the empty cells, so its time is not one visit per cell.

Mistakes

- Two land cells side by side, the row 1, 1. You call the neighbor before you set the cell to 0. Each cell still sees 1 in the other, and the calls do not end.
- Rotting oranges on 2, 1 / 1, 1. You enqueue a fresh orange and you skip the write that sets it to 2. The center-bottom cell is reached by two neighbors in the same minute, fresh is subtracted twice, and you return −1. The true answer is 2.
- Calling Array.shift to take the next orange. On a grid where the rot spreads through every cell, each shift copies the rest of the queue. The minutes can still be right. The time is no longer one visit per cell. A head index is one visit per cell.
- Word AAA on the board A, A / A, B, with the restore line deleted. The first start leaves # in every A. Later starts see holes and you return false. The word is there.
- Flood fill on 1, 1 / 1, 0, starting at the top-left, when the new color is already 1. You skip the immediate return. Every neighbor still matches, and the walk never ends.
- Shortest bridge on the row 1, 0, 1, returning steps plus 1. You return 2. One water cell means the answer is 1. Returning 1 at the moment the first layer sees a 1 is the other miss: steps is still 0, the islands already share an edge, and the answer is 0.

Practice

Stay before chapter 24. Then before you leave the chapter. The solver and the trie are a second pass.

200. Number of Islands MEDIUM STAY
Count starts of a search. Mark the land as water when you enter.

733. Flood Fill EASY STAY
If the new color equals the start color, return. Otherwise paint as you enter.

994. Rotting Oranges MEDIUM STAY
Multi-source. A layer is a minute. Return −1 if fresh oranges remain.

79. Word Search MEDIUM STAY
Mark, recurse, unmark. The letter must match word[i] before you mark.

695. Max Area of Island MEDIUM THEN
The island function returns 1 plus the four neighbors, or 0 when rejected.

130. Surrounded Regions MEDIUM THEN
Flood border O cells first. Flip only the O cells you never reached.

542. 01 Matrix MEDIUM THEN
Every 0 is a source at distance 0. The first visit is the distance.

417. Pacific Atlantic Water Flow MEDIUM THEN
Walk uphill from each ocean. A cell reached by both searches is an answer.

934. Shortest Bridge MEDIUM THEN
Paint one island, expand, return 0 if the other island already touches.

1254. Number of Closed Islands MEDIUM THEN
0 is land. Sink the border lands, then count what remains.

37. Sudoku Solver HARD SECOND PASS
Empty cells, three sets, place a digit, recurse, remove it.

212. Word Search II HARD SECOND PASS
The board walk you know, pruned by a trie. Learn the trie later. The link is the note above.

REWRITE FROM MEMORY
Rewrite islands and rotting oranges. Include the mark before you enqueue, and the bridge returning 0.

Carry forward. Chapter 24 fills a cell only after the cells it reads. That order is the method.

CHAPTER 24 · BUILDS ON CHAPTERS 15 AND 23

Cells that depend on smaller cells

You fill a cell only after the cells it reads. The order you fill the grid is the method.

What this pattern is

Some grid answers are simpler than a search. The number you write in a cell depends on two or three neighbors, and those neighbors have to be finished first. You choose a filling order that makes that true. People call this dynamic programming. In this book that phrase means one thing: filling a cell after the cells it reads.

Picture a form. Each blank, past the top and the left edge, is decided by the blank above it and the blank to its left. You fill the top row first, because those blanks have no top inside the form. Then you go row by row, left to right. Chapter 23 visited cells in whatever order a stack or a queue produced. Here the order is fixed before you start, and the formula is small.

What you already know

Chapter 23 walked neighbors and marked a cell so you would not enter it twice. You still know m, n, and the four directions when a later problem searches. This chapter usually does not search. A prefix, from chapter 8, is a running total along an edge. Chapter 15 already computes the largest rectangle inside a histogram, with a 0 at both ends. You call that function. You do not rebuild the width. Chapter 18 is why each row of a new answer grid must be its own array.

A worked example

You want the number of ways to walk from the top-left cell to the bottom-right cell of a 3 by 3 grid. A step goes only right or down. Call the rows and columns 0, 1, and 2.

You build an answer grid of the same shape and fill every cell with 1. The first row stays 1. There is only one way to reach any cell there: all rights. The first column stays 1. There is only one way: all downs. Those cells have no top neighbor, or no left neighbor, inside the grid. You do not overwrite them.

Cell (1, 1) reads the cell above, which is 1, and the cell to the left, which is 1. You write 2. The two ways are right-then-down, and down-then-right. Cell (1, 2) reads above, 1, and left, 2. You write 3. Cell (2, 1) reads above, 2, and left, 1. You write 3. Cell (2, 2) reads above, 3, and left, 3. You write 6.

Six is the answer. You could write each cell only because the cells it reads were already numbers, not blanks. That is the filling order. If the target were a 1 by 1 grid, the single cell stays 1, and the inner loops never run.

```
m = 3, n = 3, start filled with 1s
r=1,c=1: 1+1 = 2
r=1,c=2: 1+2 = 3
r=2,c=1: 2+1 = 3
r=2,c=2: 3+3 = 6
six ways across a 3 by 3 grid of cells
```

How you recognise it

Unique paths asks how many ways reach a cell, and that number is the cell above plus the cell to the left. The first row and the first column stay 1, because the only route is along the edge. An obstacle writes 0, and it also cuts the rest of an edge that would have flowed through it. A minimum path takes the cheaper parent and then adds the cell you are standing on. A square of the character 1 grows by 1 plus the smaller of the three neighbors above-left, and the returned number is the area, side times side. A rectangle of those characters becomes a histogram on each row, and the histogram function is chapter 15.

- Use chapter 24 when the answer at a cell is a sum, a min, or a size taken from neighbors that are already written.
- Use chapter 23 when you are painting a region or counting steps from a source, and the order comes from a stack or a queue.
- Use chapter 15 when a row of heights asks for the largest rectangle, because that function is already written.

What to say out loud

SAY THIS BEFORE YOU CODE

The invariant, the fact that is true just before you write, is that every cell this formula reads is already written. You finish the first row and the first column before the inner loop, and you leave those edges in place. Each later cell reads the top, the left, or both. Those writes have already happened.

The template

EDGES STAY 1. EVERYONE ELSE ADDS THE TOP AND THE LEFT.

```
function uniquePaths(m, n) {
  const dp = Array.from({ length: m }, () => Array(n).fill(1));
  for (let r = 1; r < m; r++) {
    for (let c = 1; c < n; c++) {
      dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
    }
  }
  return dp[m - 1][n - 1];
}
```

The grid starts full of 1s, so the edges are already the right answer. The loops start at row 1 and column 1. That start index is the off-by-one that protects the edge. A loop that starts at 0 overwrites the first row, and a cell in row 0 has no top neighbor inside the grid. The update reads `dp[r - 1][c]` and `dp[r][c - 1]`, both written earlier, and stores the sum. You do not read the right neighbor or the cell below. Those are not filled yet. The answer is the bottom-right cell.

The cheaper parent

Minimum path sum uses the same order and a different formula. You may step only right or down, and you want the cheapest path. Take the grid 1, 3, 1 / 1, 5, 1 / 4, 2, 1. The start cell is 1, so the answer grid's top-left is 1.

The first row is a prefix, a running total from the left. The next cell is 1 + 3, which is 4. The cell after that is 4 + 1, which is 5. There is no up step along the top. The first column is a running total downward: 1 + 1 is 2, and 2 + 4 is 6. There is no left step along that column.

Every later cell adds its own value to the cheaper parent. The center is 5 plus the min of 4 above and 2 to the left, so you write 7. The cell to its right is 1 plus the min of 5 above and 7 to the left, so you write 6. The bottom-middle is 2 plus the min of 7 above and 6 to the left, so you write 8. The bottom-right is 1 plus the min of 6 above and 8 to the left, so you write 7. That 7 is the answer.

THE FIRST ROW AND THE FIRST COLUMN ARE PREFIXES. EVERY OTHER CELL ADDS THE CHEAPER PARENT.

```
function minPathSum(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const dp = Array.from({ length: m }, () => Array(n).fill(0));
  dp[0][0] = grid[0][0];
  for (let c = 1; c < n; c++) dp[0][c] = dp[0][c - 1] + grid[0][c];
  for (let r = 1; r < m; r++) {
    dp[r][0] = dp[r - 1][0] + grid[r][0];
    for (let c = 1; c < n; c++) {
```

```
      dp[r][c] = grid[r][c] + Math.min(dp[r - 1][c], dp[r][c - 1]);
    }
  }
  return dp[m - 1][n - 1];
}
```

The first row loop starts at column 1, so it does not overwrite the start. Inside each later row, the first column is written before the inner loop reads it as a left parent. The update adds `grid[r][c]` once, to the min of the top and the left. Add the cell into both parents first and you will double-count it. The answer is the bottom-right cell. On the sample, that cell is 7.

The largest square

The matrix stores characters, not numbers. A square cell is the character 1. The number 1 is a different value, and a comparison against the number will skip every cell. Use this character grid. Row 0 is 1, 0, 1. Row 1 is 1, 1, 1. Row 2 is 1, 1, 1. Each cell stores the side of the largest square whose bottom-right corner is that cell.

The top-left 1 has no up, no left, and no diagonal, so those three reads are 0. The side is 1 plus the min of 0, 0, and 0, which is 1. The 0 in the top middle stores nothing. The top-right 1 also has zeros for up and for the diagonal, so its side is 1. Row 1, column 0 is a 1 with no left neighbor, so its side is 1.

The center reads up 0, because the cell above is the character 0 and stores no side. It reads left 1 and diagonal 1. The min is 0, so the side is 1. The cell to its right reads up 1, left 1, and diagonal 0, so the side is 1. Row 2, column 1 reads up 1, left 1, and diagonal 1, so the side is 2. The bottom-right reads up 1, left 2, and diagonal 1, so the side is 2. The best side on the board is 2. You return side times side, the area, which is 4. Returning 2 would return the side, and the problem asks for the area.

THE CELLS ARE CHARACTERS. THE ANSWER IS THE AREA, SIDE TIMES SIDE.

```
function maximalSquare(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  const dp = Array.from({ length: m }, () => Array(n).fill(0));
  let side = 0;
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (matrix[r][c] !== "1") continue;
      const up = r > 0 ? dp[r - 1][c] : 0;
      const left = c > 0 ? dp[r][c - 1] : 0;
      const diag = r > 0 && c > 0 ? dp[r - 1][c - 1] : 0;
      dp[r][c] = 1 + Math.min(up, left, diag);
      if (dp[r][c] > side) side = dp[r][c];
    }
  }
  return side * side;
}
```

The test is `!== "1"`, a character. A 0 from the matrix is skipped, and the answer cell stays 0, which is the right side for a non-square. On the first row, up and the diagonal are forced to 0 because `r > 0` fails. On the first column, left and the diagonal are 0 because `c > 0` fails. Reading `dp[-1]` would step off the grid. The update is 1 plus the min of the three neighbors, and it happens only for a character 1. You keep the best side. The return is `side * side`. A board of all zeros leaves side at 0 and returns 0.

```
1 0 1
1 1 1
1 1 1
side at (0,0) = 1
(1,1) = 1 + min(1, 1, 0) = 1
(1,2) = 1 + min(0, 1, 1) = 1
(2,1) = 1 + min(1, 1, 1) = 2
(2,2) = 1 + min(1, 2, 1) = 2
area = 4
```

Obstacles cut the edge

Unique paths II puts obstacles on that grid. A 1 in the obstacle grid is blocked. A 0 is open. If the start cell is blocked, you return 0 immediately. There is no path to count. Take 1, 0 / 0, 0. The start is blocked, so the answer is 0. Skip that check and the rest of the fill pretends a path began.

An obstacle also cuts an edge, and the suffix of that edge stays 0. Take 0, 1, 0 / 0, 0, 0. The top-left is open, so it holds 1 way to be there. The next cell on the first row is blocked, so you write 0. The cell to its right, still on the first row, stays 0. You cannot cross the block by hugging the top. The first column is open, so row 1 column 0 holds 1. The bottom-middle adds the top, which is 0, and the left, which is 1, so it holds 1. The bottom-right adds 0 and 1, so it holds 1. That single path goes down first, then right, then right.

If you had filled the whole first row with 1s and never zeroed the suffix, the top-right would stay 1, and the bottom-right would count paths that cross the obstacle. On this grid that mistake produces 3. The real answer is 1.

Falling one row at a time

A falling path steps from one row to the next, into the cell directly below or either diagonal below. You read only the previous row, so you fill downward from row 1. Edge cells have one missing parent. You clip the look so you never read column −1 or a column past the end.

Take 2, 1, 3 / 6, 5, 4 / 7, 8, 9. Copy row 0 as 2, 1, 3. In row 1, the left cell 6 can see 2 and 1 above it, so you write 6 + 1, which is 7. The middle 5 can see 2, 1, and 3, so you write 5 + 1, which is 6. The right cell 4 can see 1 and 3, so you write 4 + 1, which is 5. In the last row, 7 plus the min of 7 and 6 is 13, 8 plus the min of 7, 6, and 5 is 13, and 9 plus the min of 6 and 5 is 14. The answer is the minimum of the last row, which is 13.

The rectangle uses chapter 15

Maximal rectangle, on the second pass, builds a histogram per row and calls the chapter 15 function, sentinels included. You do not re-derive the width. The cells are the character 1 again. If the cell is that character, the height at that column grows by 1. If the cell is the character 0, the height resets to 0.

Take the character grid whose rows are 1 0 1 0 0, then 1 0 1 1 1, then 1 1 1 1 1, then 1 0 0 1 0. After row 0 the heights are 1, 0, 1, 0, 0. After row 1 they are 2, 0, 2, 1, 1, because a 0 reset the second column and the three 1s on the right started from 0. After row 2 they are 3, 1, 3, 2, 2. The histogram on that row contains a bar of height 2 across the last three columns, area 6. Later rows do not beat 6. The answer is 6. Paste the histogram function. A check against the number 1 never grows a height, and the area stays 0.

A path that must rise

Longest increasing path, on the second pass, is a depth-first search with a memo. From a cell you may step to a strictly greater neighbor, in any of the four directions. The length at a cell is 1 plus the best neighbor, or 1 if no neighbor is greater. Strict means each step gets larger, so the walk cannot return to a cell it left. The memo is only a cache. You do not also need a mark for the current path.

Take 1, 2 / 4, 3. From 4, no neighbor is greater, so its length is 1. From 3, the left neighbor 4 is greater, so its length is 2. From 2, the down neighbor 3 has length 2, so its length is 3. From 1, the best neighbor is 2, with length 3, so

its length is 4. The path is 1, 2, 3, 4. A stored 0 means not computed yet. That sentinel is safe because every real length is at least 1. If the step allowed an equal neighbor, a board of all 1s would walk back and forth, and the search would not end. The problem is strict.

Time and extra memory

The path fillings, unique paths II, the falling path, and maximal square are O(mn) time. You store another m by n grid of answers, so the extra memory is O(mn). Maximal rectangle is O(mn) because each of the m histograms is linear. The increasing path is O(mn): each cell is expanded once, the memo is another grid, and the call stack is as deep as the path.

Mistakes

- Unique paths on a 3 by 3 grid, with both loops starting at 0. The top-left reads a row that is not in the grid. The sum is not a number of ways, and the bottom-right is not 6. The first row and the first column have to stay 1.
- Unique paths II on 0, 1, 0 / 0, 0, 0. You fill the board with 1s and you only zero an obstacle when both indexes are at least 1. The obstacle is on the first row, so it stays a fake 1, the cell to its right stays 1, and the bottom-right becomes 3. One path exists, down then right then right. On 1, 0 / 0, 0 the start is blocked. Skip the check and the same fill returns 2. The answer is 0.
- Minimum path on 1, 3 / 1, 5. You set the start to 1 and you skip the first-row and first-column prefixes. The bottom-right adds 5 to the min of two unread zeros and returns 5. The real cheaper path is 1, then 1, then 5, which totals 7.
- Maximal square on the character grid 1, 1 / 1, 1, compared with the number 1. No cell matches, the side stays 0, and you return 0. The square has side 2 and area 4. The same grid, compared correctly, still fails if you return the side. You return 2. The problem asks for side times side.
- Falling path on 2, 1, 3 / 6, 5, 4 / 7, 8, 9, reading three parents on the left edge. The left edge has no above-left column. The min is not a number, and the minimum of the last row is not 13.
- Longest increasing path on a grid of all 1s, stepping to a neighbor that is greater or equal. Each cell walks to its neighbor and back, and the calls do not end. Strict increase leaves every length at 1, so the answer is 1. A stored 0 means not computed yet, because a finished length is at least 1.

Practice

Stay before chapter 25. Then before you leave the chapter. The histogram rectangle and the rising path are a second pass.

62. Unique Paths MEDIUM STAY
First row and first column are 1. Each other cell adds top and left.

64. Minimum Path Sum MEDIUM STAY
Edges are prefixes. Each other cell adds the min of top and left.

221. Maximal Square MEDIUM STAY
Character "1". Side is 1 + min of three neighbors. Return side * side.

63. Unique Paths II MEDIUM THEN
An obstacle writes 0 and cuts the edge. Start obstacle returns 0.

931. Minimum Falling Path Sum MEDIUM THEN
Min of the three cells above, clipped at the ends. Answer is the min of the last row.

85. Maximal Rectangle HARD SECOND PASS
Each row is a histogram of consecutive 1s. Reuse chapter 15.

329. Longest Increasing Path in a Matrix HARD SECOND PASS

Memoized search, four directions, strict increase, no cycle mark beyond the memo.

REWRITE FROM MEMORY

Rewrite unique paths and maximal square. Say side times side as you return.

Carry forward. Chapter 25 puts the path on a calendar and then hides the chapter numbers.

CHAPTER 25 · BUILDS ON THE WHOLE PATH

Four-week order and a mixed drill

You walk the chapters on a four-week calendar. A mixed drill then hides the chapter number.

You have the templates. The work now is the order you practice them, and a mixed list that hides the chapter number.

Four weeks

The four weeks are a calendar. Each week has a band of chapters. You finish that band before you open the next band.

A weekday is one chapter. You read it. You close the page and rewrite its template the same day, from a blank file. Then you solve that chapter's Stay problems. A Stay problem is the template on a new input. You finish those before the next chapter.

An open Stay problem outranks a new chapter. If it is still unsolved, you do not have the template yet, and the next chapter builds on it. The next weekday stays on that problem. The calendar slips. A slip of one day is cheaper than a hole you carry into the following week.

The weekend is for the Then problems of that same week. A Then problem can wait until the week's chapters are in place. Saturday and Sunday are also when you rewrite a template that felt shaky on Friday. The weekend belongs to the week you just lived. You pick up the next week's first chapter on Monday.

Week 1 is chapters 0 through 6. You learn the index, one pass, a map, and both ways to move two indexes. Week 2 is chapters 7 through 11. You learn a window, a prefix, a difference array, two passes, and Kadane. Week 3 is chapters 12 through 17. You sort, place values at indexes, binary-search, use a stack, take a greedy scan, and you practice naming which pattern you are in. Week 4 is chapters 18 through 24. You treat the matrix as a grid, a walk, a set of marks, a rectangle sum, a staircase, a region, and a filling order.

Second pass starts the week after you finish chapter 24. The ladder marks some problems Second pass. The sudoku solver, word search II, the histogram rectangle, and the longest rising path are in that group. They wait until the four weeks are done. The last two evenings of week 4 are the card below and the mixed mock.

WEEK	CHAPTERS	WHAT YOU CAN DO BY SUNDAY
1	0–6	Index, classify, one pass, pairs, a map, both kinds of two indices.
2	7–11	A window, a prefix, a difference array, two passes, Kadane.
3	12–17	Sort, placement, binary search, a stack, a greedy scan, and you can name which one.
4	18–24	A matrix, a spiral, a mark, a rectangle sum, a staircase, a flood, a filling order.

The table is that same plan in one view. The Sunday sentence is what you can do when the week's Stay problems are done.

Fifteen templates, from a blank file

You drill these fifteen from a blank file. The chapter stays closed while you type. Before the first loop, you say the invariant out loud: the fact that stays true about the range, the mark, or the cell you are about to write. After the code is on the page, you check the update order. A prefix map queries before it increments. A stock profit uses the old minimum before the minimum moves. A window shrinks with while, and the length is the right index minus the left index plus 1. If you cannot say that order, the template is not yours yet. One template is one sitting. Fifteen sittings cover the list.

1. Stock I. Profit uses the old minimum. Then the minimum may move.

2. Two sum. Map lookup of the complement, then insert.
3. Two sum on a sorted array. Too small moves l.
4. Container. Move the shorter wall only.
5. Read r, write w. Duplicates II compares with w − 2.
6. Variable window. Shrink with while. Length is r − l + 1.
7. Prefix frequency. Seed 0 → 1. Query sum − k. Then increment. Negative modulo repaired.
8. Difference array. +v at L, −v at R + 1, then one scan.
9. Kadane from nums[0]. Product uses temps. Circular rejects the empty wrap.
10. 3Sum. Skip duplicate i, and skip both ends after a hit.
11. Dutch flag. A swap with hi does not advance mid.
12. lowerBound. while lo < hi. ok sets hi = mid. Otherwise lo = mid + 1.
13. Next greater, stack of indices. Histogram with a 0 on both ends.
14. 2D prefix build, and the rectangle with the added-back corner.
15. Islands, mark on entry. Rotting oranges, mark on enqueue. A layer is one minute.

The night before

Use this card the night before a mock, or the night before you sit down to be tested. Cover the chapter column with a hand or a sheet of paper. Read the signal. Say the pattern name, then the usual cost. Then uncover the chapter and check. A wrong name sends you back to one blank-file rewrite of that template, and then back to the card. The card is short because you can finish it in one sitting.

SIGNAL	PATTERN	USUAL COST	CH.
Summary of one side	One pass	n	**02**
Have I seen the complement	Map, insert after lookup	n	**04**
Sorted, discard one end	Two indices, ends	n	**05**
Overwrite in place	Read and write	n	**06**
Contiguous, shrink repairs it	Window	n	**07**
Sum K, negatives allowed	Prefix map	n	**08**
Range add, then read	Difference array	n + updates	**09**
Both sides, then combine	Left pass and right pass	n	**10**
Best non-empty subarray sum	Kadane	n	**11**
Pairs or intervals in order	Sort, then scan	n log n, or n^2	**12**
Value wants an index	Cyclic place, or XOR	n	**13**
Sorted half, or monotone answer	Half-open search	log n, or n log A	**14**
Nearest greater, window max	Stack or deque	n	**15**
Farthest reach, tank, one edit	Greedy scan	n	**16**
Rectangle sum	2D prefix	mn, query 1	**21**
Row-and-column sorted	Staircase from top-right	m + n	**22**
Component, minutes, word	Mark on entry, BFS layer	mn	**23**
Cell reads finished neighbors	Fill in order	mn	**24**

Mixed mock

The list below is twenty-four problems. The order hides the chapter, so the previous chapter is not a hint. Set a timer if you want the interview limit. Give one problem 45 minutes, or take one sitting of four problems.

Write the pattern name before you write any code. The name is the skill from chapter 17. The names are on the next page. Uncover that page only after each problem you attempted has a written name.

1. 1. Two Sum
2. 283. Move Zeroes
3. 200. Number of Islands
4. 209. Minimum Size Subarray Sum
5. 33. Search in Rotated Sorted Array
6. 53. Maximum Subarray
7. 54. Spiral Matrix
8. 11. Container With Most Water
9. 560. Subarray Sum Equals K
10. 739. Daily Temperatures
11. 75. Sort Colors
12. 56. Merge Intervals
13. 875. Koko Eating Bananas
14. 238. Product of Array Except Self
15. 1004. Max Consecutive Ones III
16. 73. Set Matrix Zeroes
17. 15. 3Sum
18. 994. Rotting Oranges
19. 167. Two Sum II - Input Array Is Sorted
20. 55. Jump Game
21. 304. Range Sum Query 2D - Immutable
22. 42. Trapping Rain Water
23. 128. Longest Consecutive Sequence
24. 74. Search a 2D Matrix

Mock answer key

Uncover this after you have named a pattern for each problem.

1. Map, lookup then insert. Chapter 4.
2. Read and write, then fill the zero tail. Chapter 6.
3. Depth-first search, mark on entry. Chapter 23.
4. Variable window, positive sums, shrink with while. Chapter 7.
5. Half-open search. One half of the rotation is sorted. Chapter 14.
6. Kadane, start at nums[0]. Chapter 11.
7. Four edges. Guard the last two legs. Chapter 19.
8. Pointers from the ends. Move the shorter wall. Chapter 5.
9. Prefix frequency. Seed 0, query before update. Chapter 8.
10. Monotonic stack of indices. Answer is the distance. Chapter 15.
11. Dutch flag. Do not advance mid after the swap with hi. Chapter 13.
12. Sort by start. Merge when start <= last end. Chapter 12.
13. Binary search the speed. Chapter 14.
14. Left products, then right products. Chapter 10.
15. Window of at most k zeros. The answer is the window length. Chapter 7.
16. First row and column as marks. Write back from the bottom-right. Chapter 20.
17. Sort, then two ends, skip duplicates. Chapter 12.
18. Multi-source breadth-first search. A layer is a minute. Chapter 23.
19. Pointers from the ends on a sorted array. Chapter 5.
20. Farthest reach. If i is past it, return false. Chapter 16.
21. 2D prefix. Add back the top-left corner. Chapter 21.
22. leftMax and rightMax, or the two-index collapse. Chapter 10.
23. Set. Start a run only when x − 1 is absent. Chapter 4.
24. Flat binary search. mid / n and mid % n. Chapter 22.

REWRITE FROM MEMORY

Rewrite the fifteen templates on blank paper. The mock is finished when each pattern name matches the key without a reread of the chapter.

Carry forward. There is no next template. When a new problem arrives, it enters through the recognition questions in chapter 17.